

Self-Supervised Learning

Prof. Jianguo Zhang
SUSTech

Slides were adapted from various sources

Recall: Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a *function* to map $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Unsupervised Learning

Data: x

Just data, no labels!

Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering, dimensionality reduction, feature learning, density estimation, etc.

Problem: Supervised Learning is Expensive!

Assume you want to label 1M images. How much will it cost?

Problem: Supervised Learning is Expensive!

Assume you want to label 1M images. How much will it cost?

(1,000,000 images)

× (10 seconds/image)

× (1/3600 hours/second)

× (\$15 / hour)

(Small to medium sized dataset)

(Fast annotation)

(Low wage paid to annotator)

Problem: Supervised Learning is Expensive!

Assume you want to label 1M images. How much will it cost?

(1,000,000 images)

(Small to medium sized dataset)

× (10 seconds/image)

(Fast annotation)

× (1/3600 hours/second)

× (\$15 / hour)

(Low wage paid to annotator)

= \$41,667

(Other assumptions: one annotator per image, no benefits / payroll tax / crowdsourcing fee for annotators; not accounting for time to set up tasks for annotators, etc. Real costs could easily be 3x this or more)

Problem: Supervised Learning is Expensive!

Assume you want to label **1B** images. How much will it cost?

(1,000,000,000 images)

(Large dataset)

× (10 seconds/image)

(Fast annotation)

× (1/3600 hours/second)

× (\$15 / hour)

(Low wage paid to annotator)

= **\$41,666,667**

(Other assumptions: one annotator per image, no benefits / payroll tax / crowdsourcing fee for annotators; not accounting for time to set up tasks for annotators, etc. Real costs could easily be 3x this or more)

Problem: Supervised Learning is Not How We Learn

Babies don't get supervision for everything they see!



Solution: Self-Supervised Learning

Lets build methods that learn from "raw" data – no annotations required

Unsupervised Learning: Model isn't told what to predict. Older terminology, not used as much today.

Self-Supervised Learning: Model is trained to predict some naturally-occurring signal in the raw data rather than human annotations.

Solution: Self-Supervised Learning

Lets build methods that learn from "raw" data – no annotations required

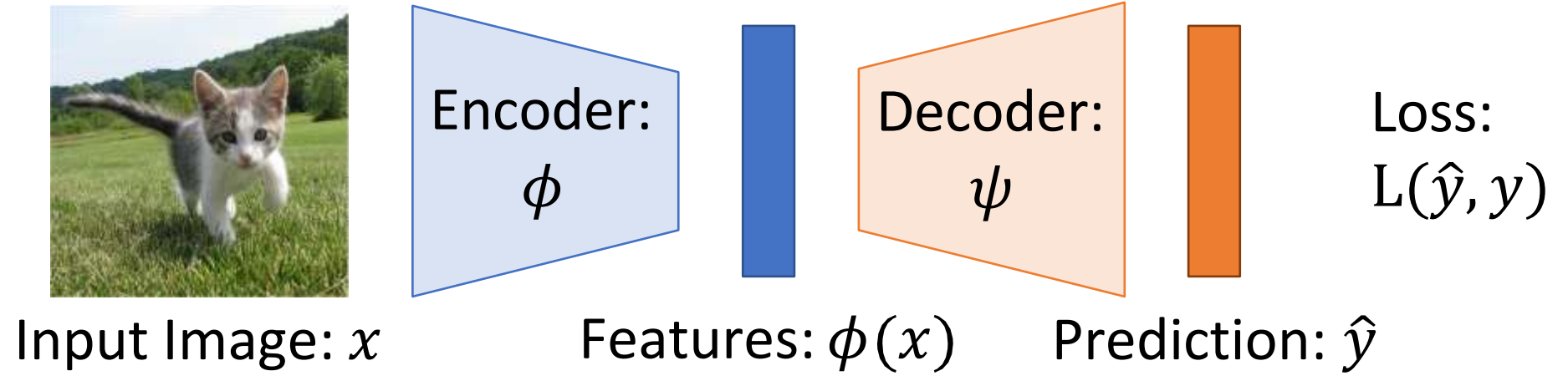
Unsupervised Learning: Model isn't told what to predict. Older terminology, not used as much today.

Self-Supervised Learning: Model is trained to predict some naturally-occurring signal in the raw data rather than human annotations.

Semi-Supervised Learning: Train jointly with some labeled data and (a lot) of unlabeled data.

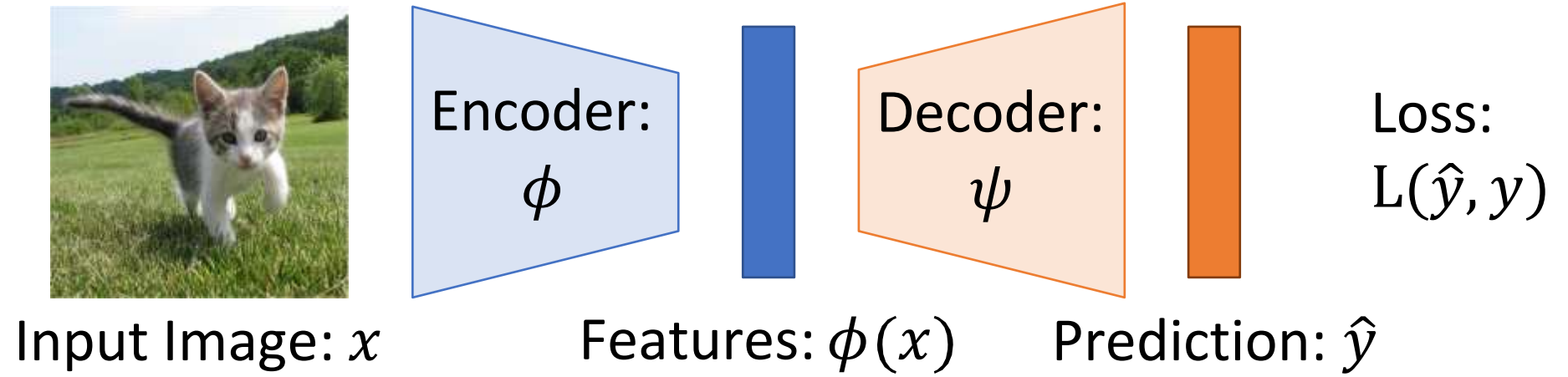
Self-Supervised Learning: Pretext then Transfer

Step 1: Pretrain a network on a pretext task that doesn't require supervision

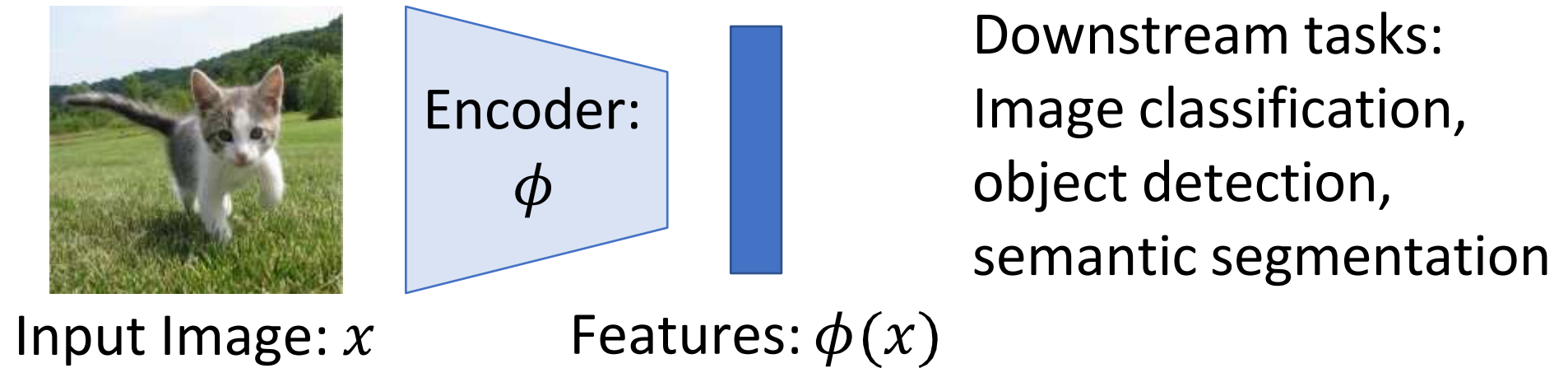


Self-Supervised Learning: Pretext then Transfer

Step 1: Pretrain a network on a pretext task that doesn't require supervision

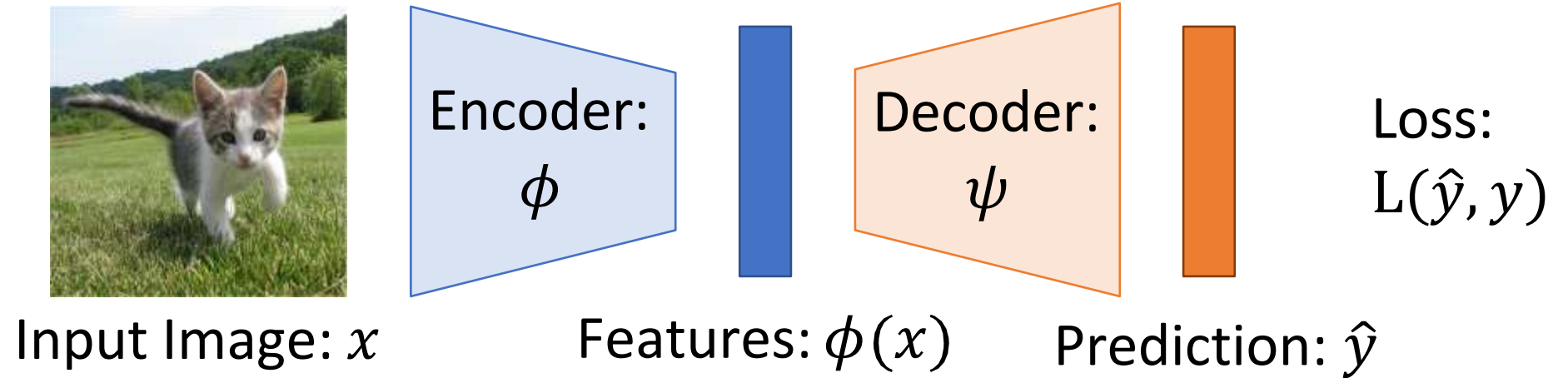


Step 2: Transfer encoder to downstream tasks via linear classifiers, KNN, finetuning

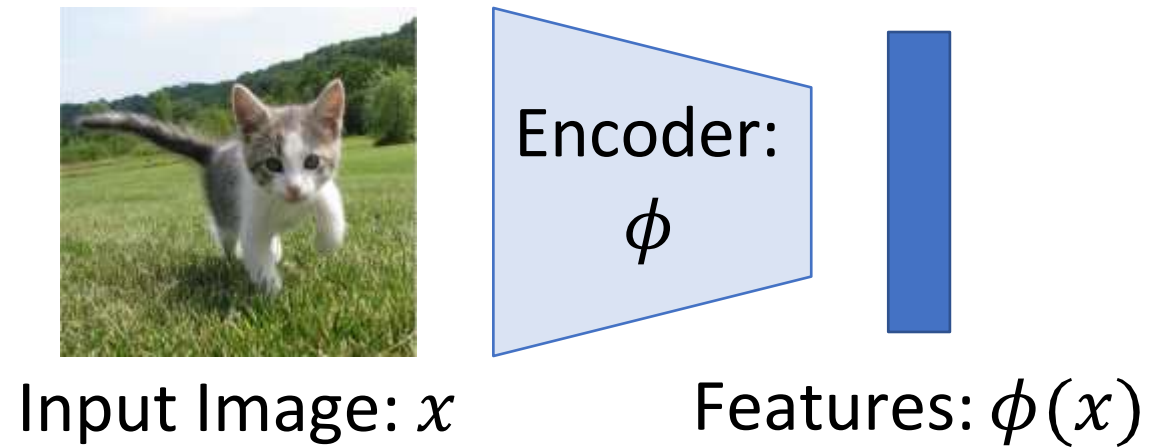


Self-Supervised Learning: Pretext then Transfer

Step 1: Pretrain a network on a pretext task that doesn't require supervision



Step 2: Transfer encoder to downstream tasks via linear classifiers, KNN, finetuning



Goal: Pretrain + Transfer does better than supervised pretraining, and better than directly training on downstream

Self-Supervised Learning: Pretext Tasks

Generative: often Predict part of the input signal

- Autoencoders (sparse, denoising, masked)
- Autoregressive
- GANs
- Colorization
- Inpainting

Discriminative: Predict something about the input signal

- Context prediction
- Rotation
- Clustering
- Contrastive

Multimodal: Use some additional signal in addition to RGB images

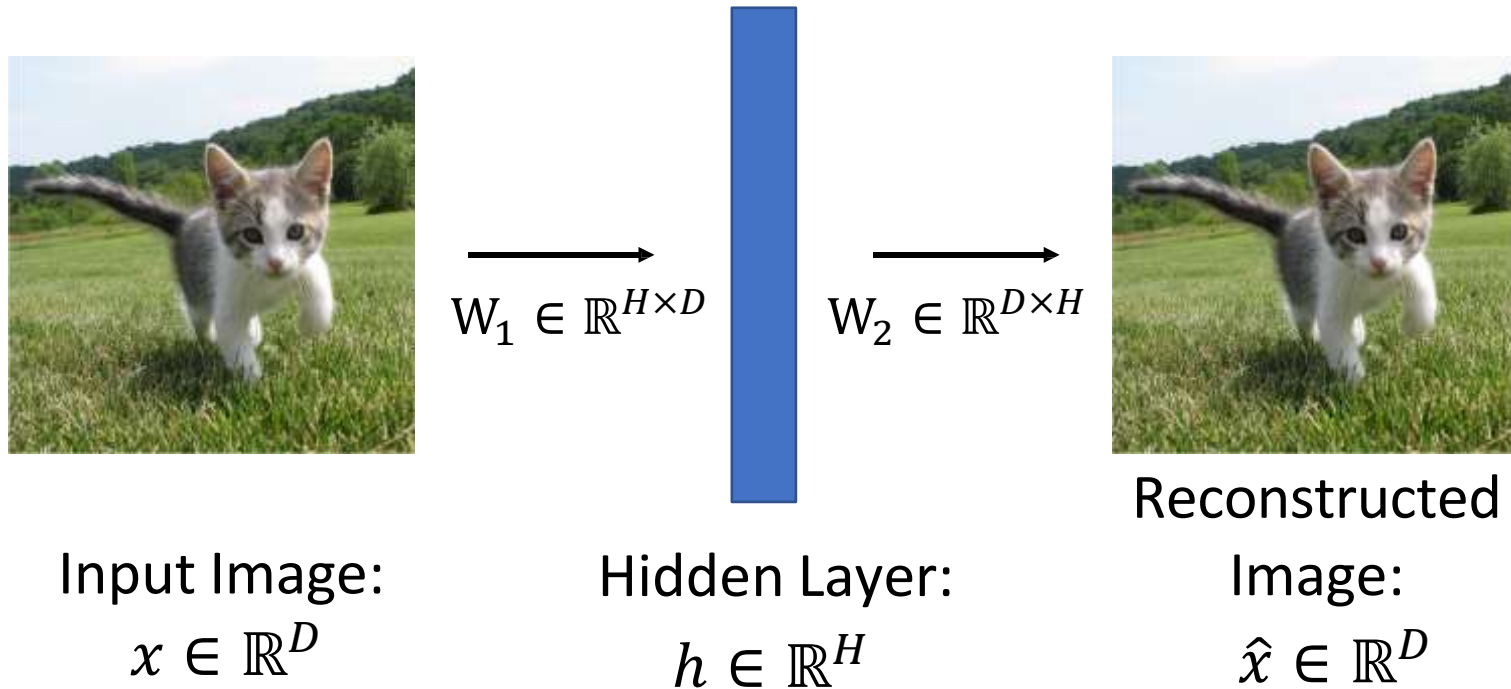
- Video
- 3D
- Sound
- Language

Recall: Autoencoder

Autoencoder tries to reconstruct inputs. Hidden layer (hopefully) learns good representations.

Generative pretraining task!

$$\begin{aligned} L(x) &= R(x, \hat{x}) \\ &= \|x - \hat{x}\|_2^2 \end{aligned}$$

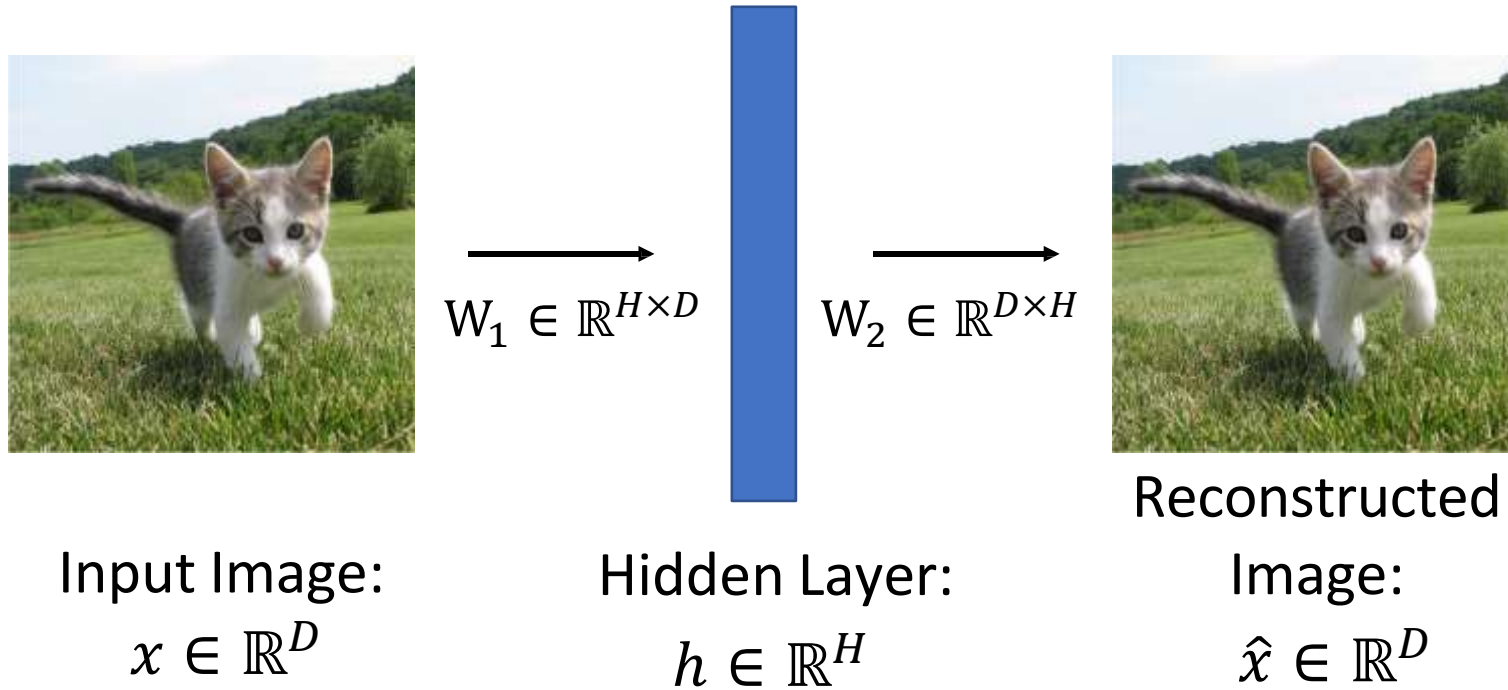


Recall: Autoencoder

Autoencoder tries to reconstruct inputs. Hidden layer (hopefully) learns good representations

$$L(x) = R(x, \hat{x}) \\ = \|x - \hat{x}\|_2^2$$

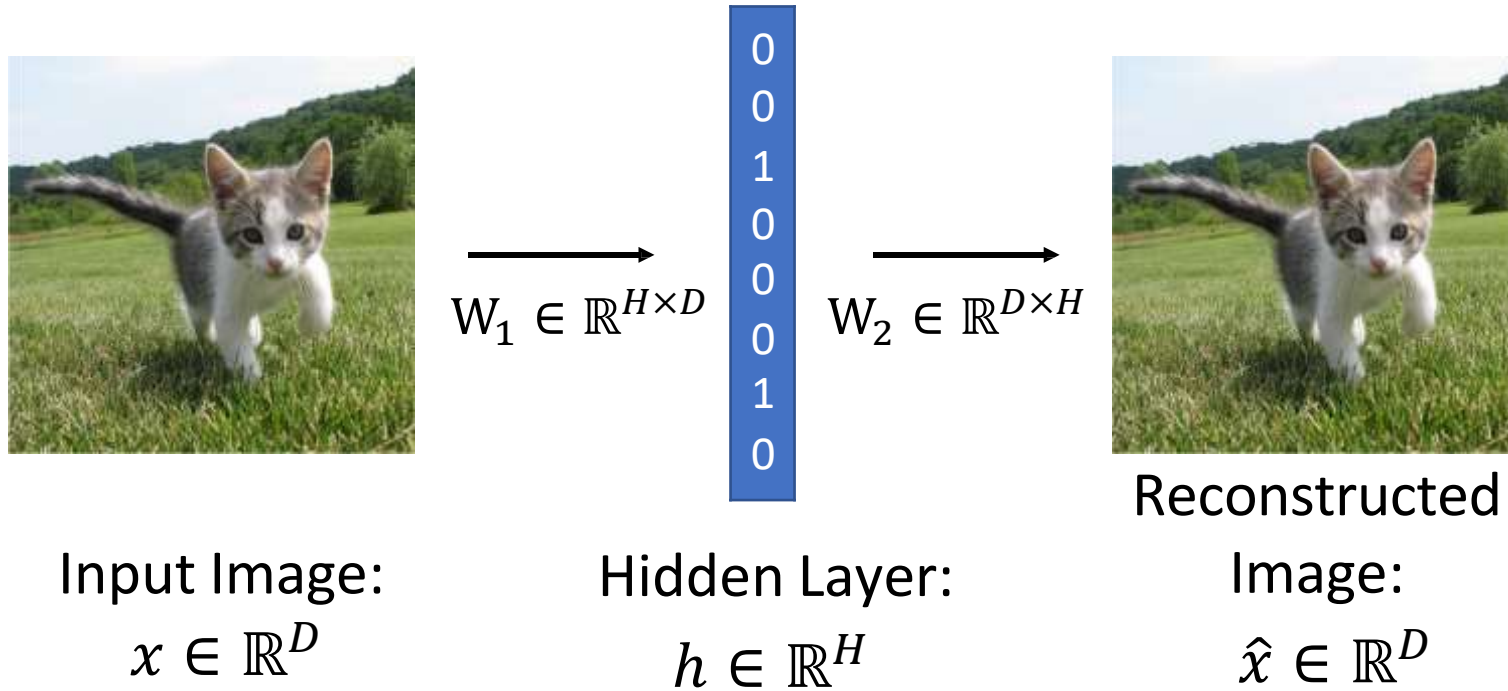
$H < D$ is the only thing forcing non-trivial hidden representations...



Sparse Autoencoder

Train an autoencoder to **reconstruct inputs** with **sparse activations** (mostly 0). Many ways to implement sparsity penalties!

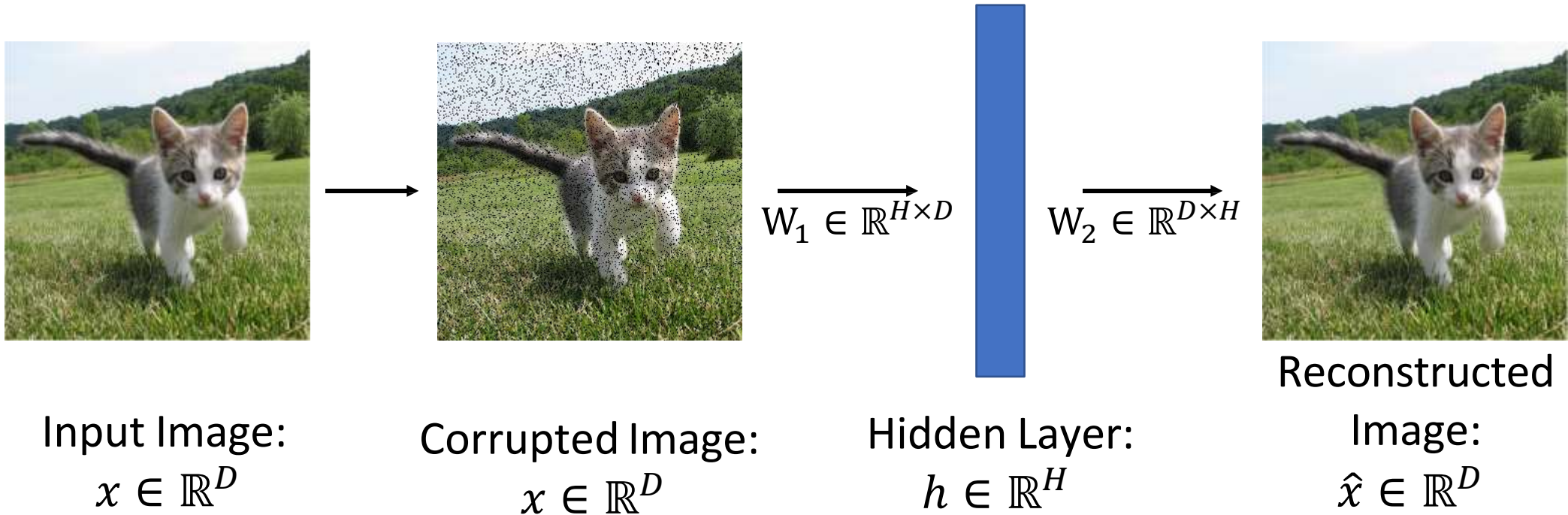
$$L(x) = R(x, \hat{x}) + \lambda S(h) \\ = \|x - \hat{x}\|_2^2 + \lambda \|h\|_1$$



Denoising Autoencoder

Train an autoencoder to reconstruct the clean version of the noisy inputs (pixels randomly set to zero)

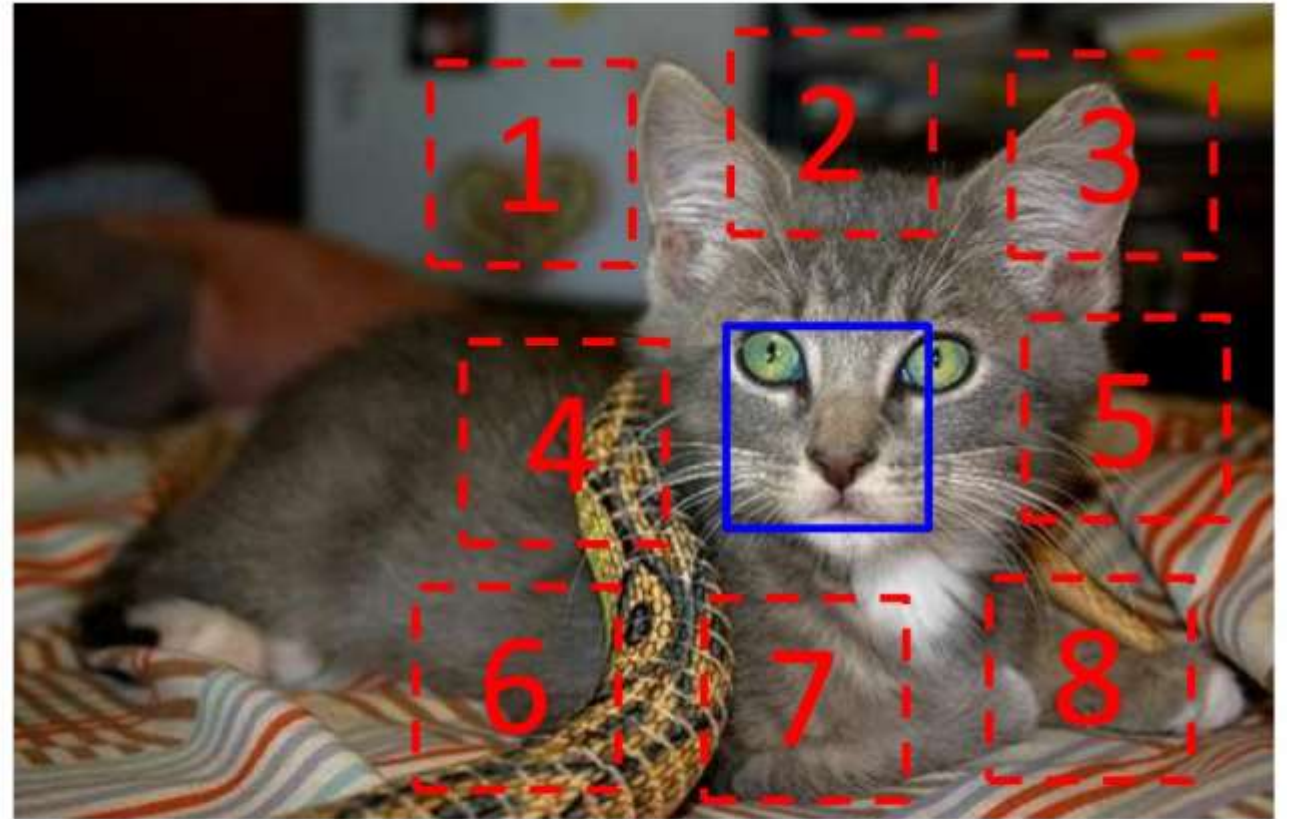
$$L(x) = R(x, \hat{x}) \\ = \|x - \hat{x}\|_2^2$$



Context Prediction

Model predicts relative location of two patches from the same image.
Discriminative pretraining task

Intuition: Requires understanding objects and their parts

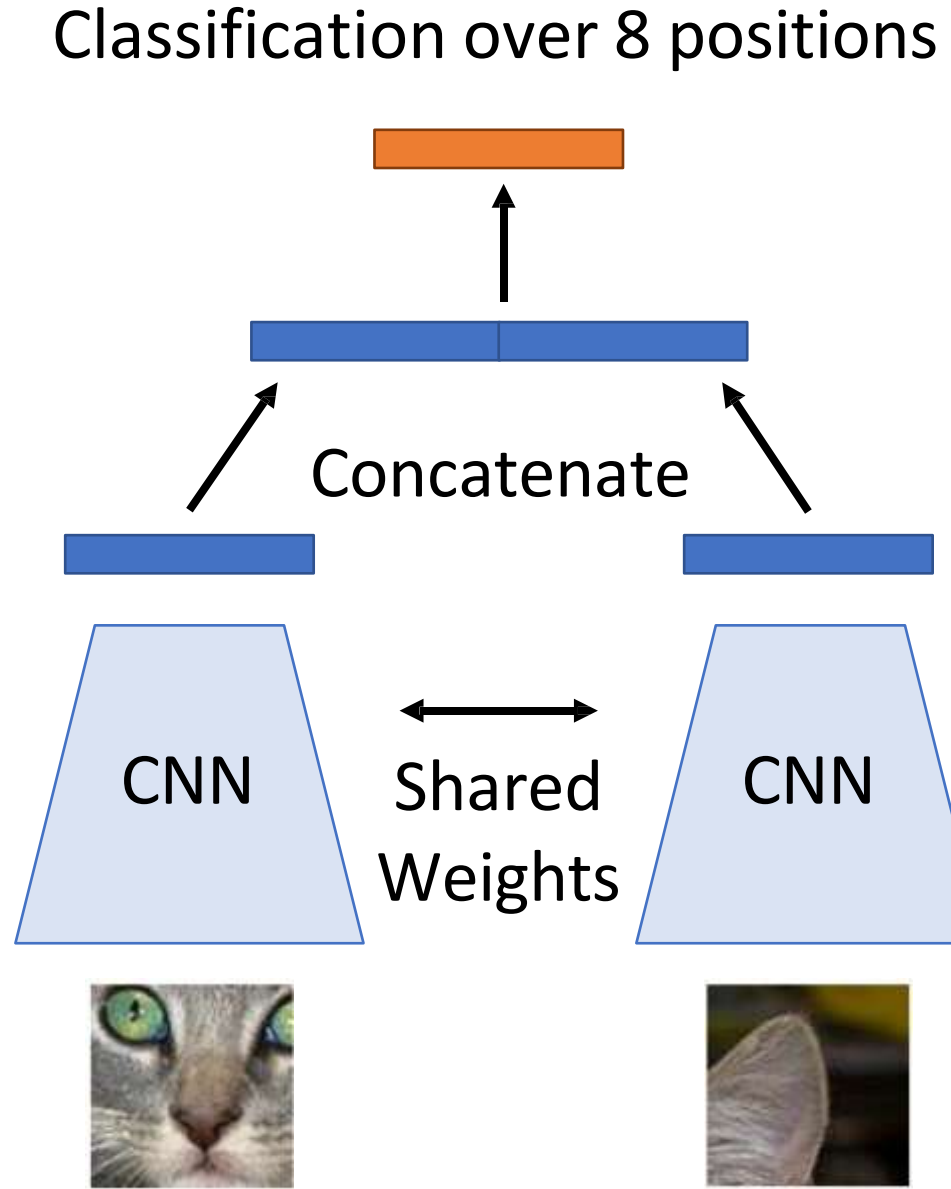


$$X = (\text{cat face patch}, \text{cat ear patch}); Y = 3$$

Context Prediction

Model predicts relative location of two patches from the same image.
Discriminative pretraining task

Intuition: Requires understanding objects and their parts

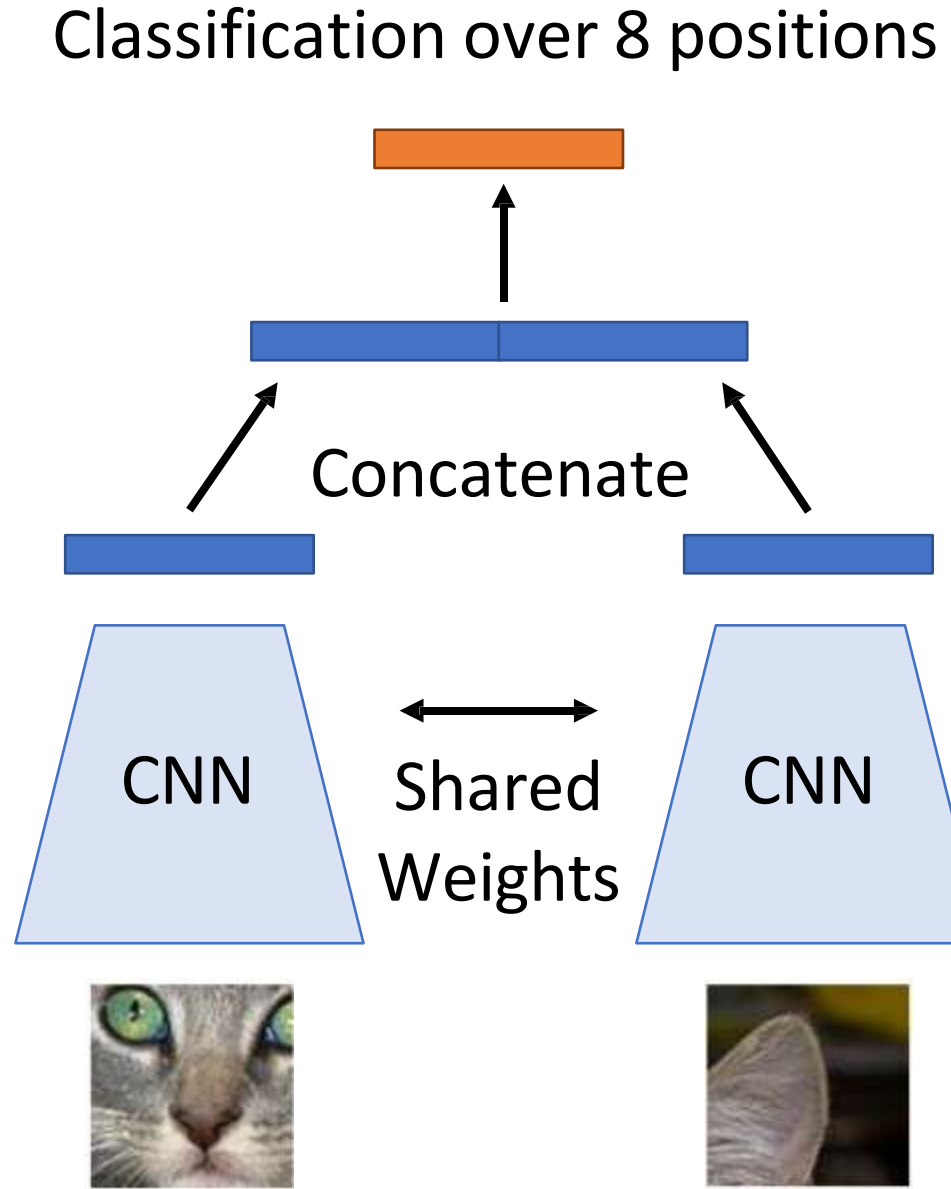


Context Prediction

Model predicts relative location of two patches from the same image.
Discriminative pretraining task

Intuition: Requires understanding objects and their parts

Two networks with shared weights sometimes called a "Siamese network"



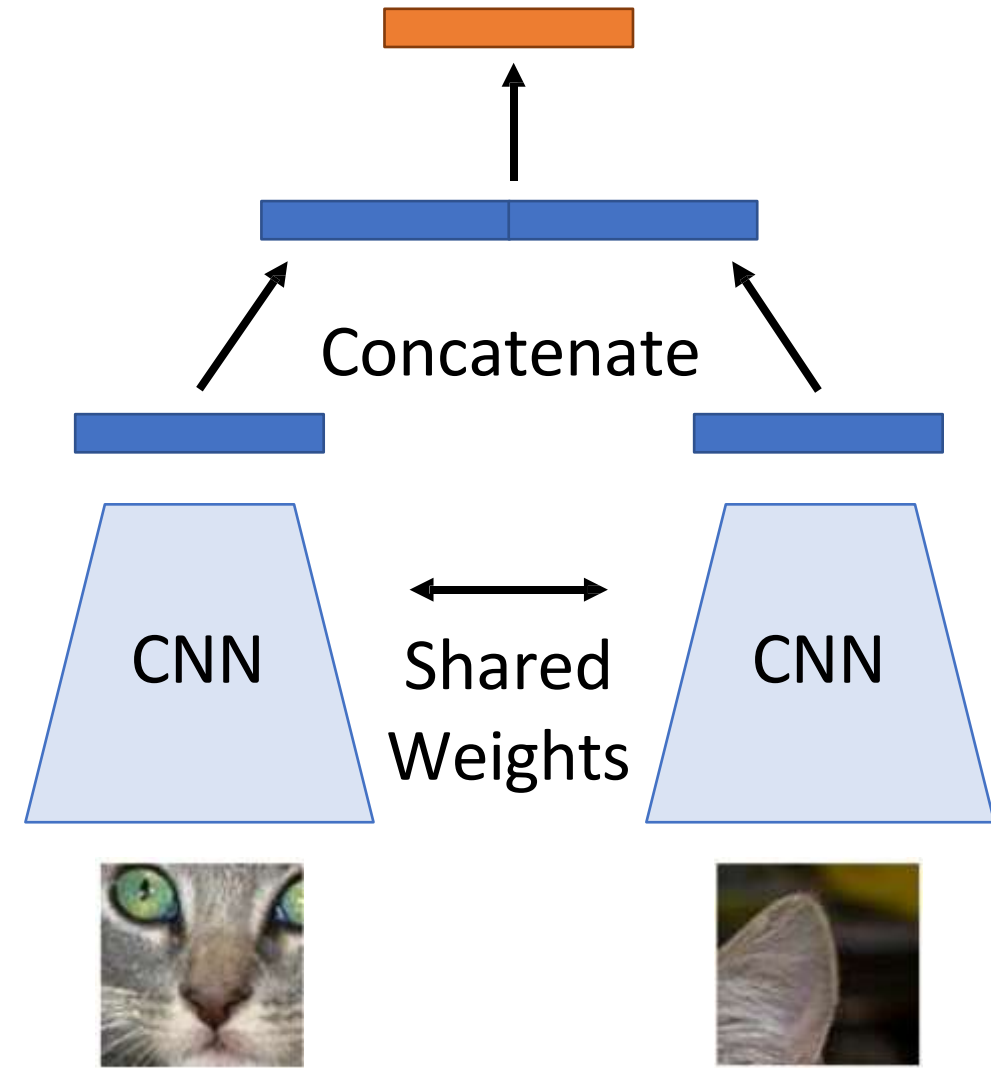
Context Prediction

Model predicts relative location of two patches from the same image.
Discriminative pretraining task

Intuition: Requires understanding objects and their parts

“For experiments, we use a ConvNet trained on a K40 GPU for approximately four weeks.”

Classification over 8 positions



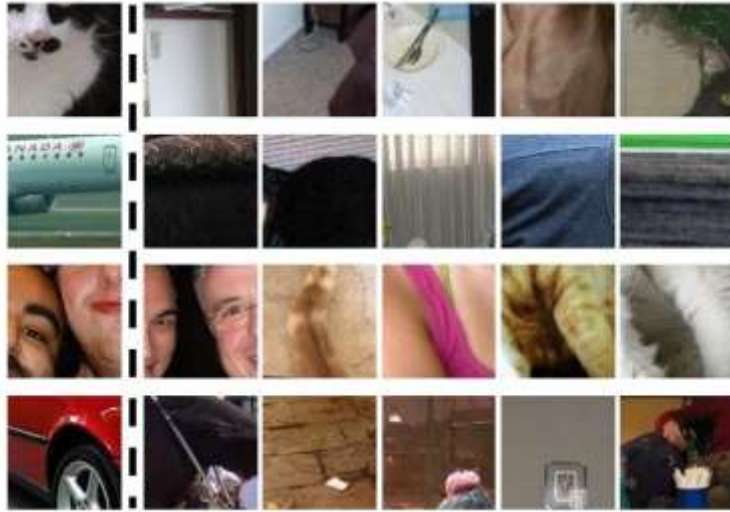
Context Prediction: Nearest Neighbors in Feature Space

Input Patch

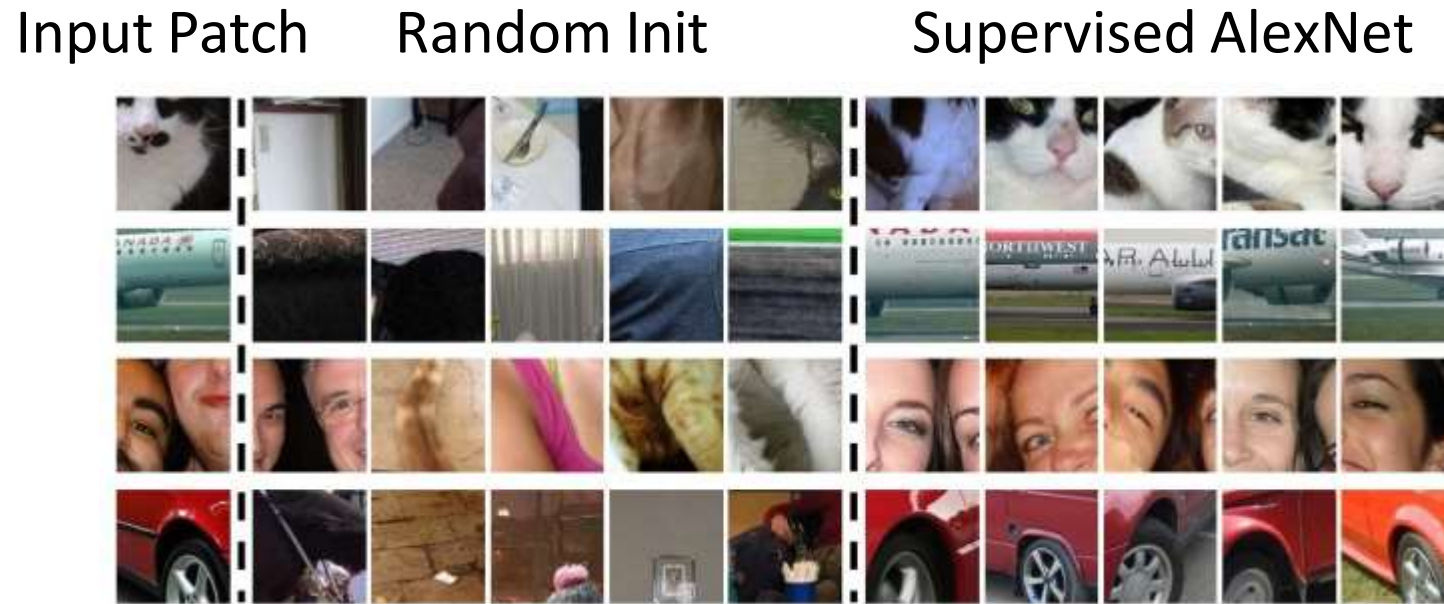


Context Prediction: Nearest Neighbors in Feature Space

Input Patch Random Init



Context Prediction: Nearest Neighbors in Feature Space



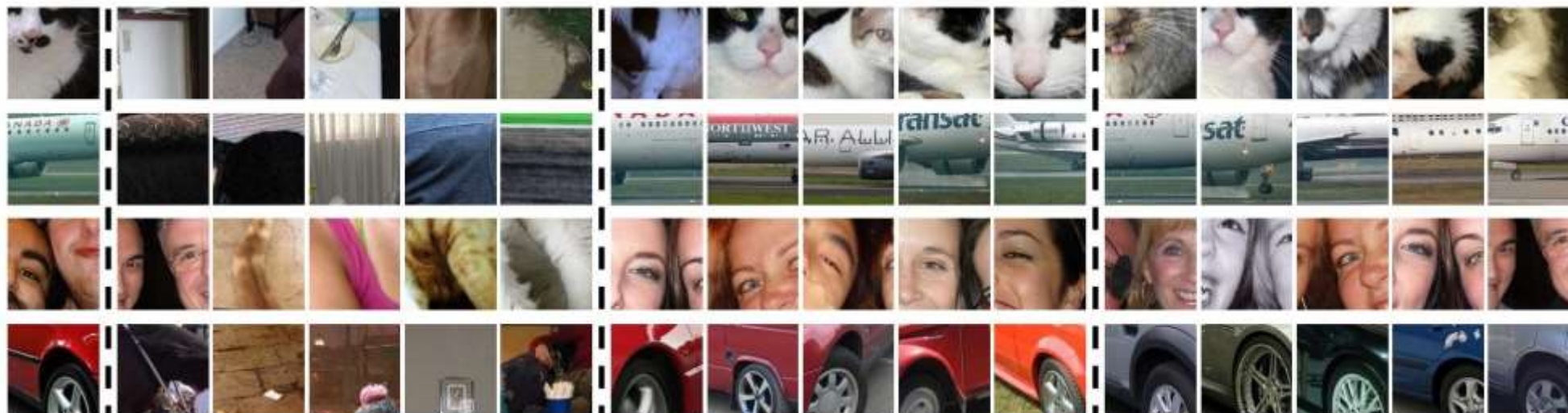
Context Prediction: Nearest Neighbors in Feature Space

Input Patch

Random Init

Supervised AlexNet

Their Features



Works
well!
Similar to
AlexNet

Context Prediction: Nearest Neighbors in Feature Space

Input Patch

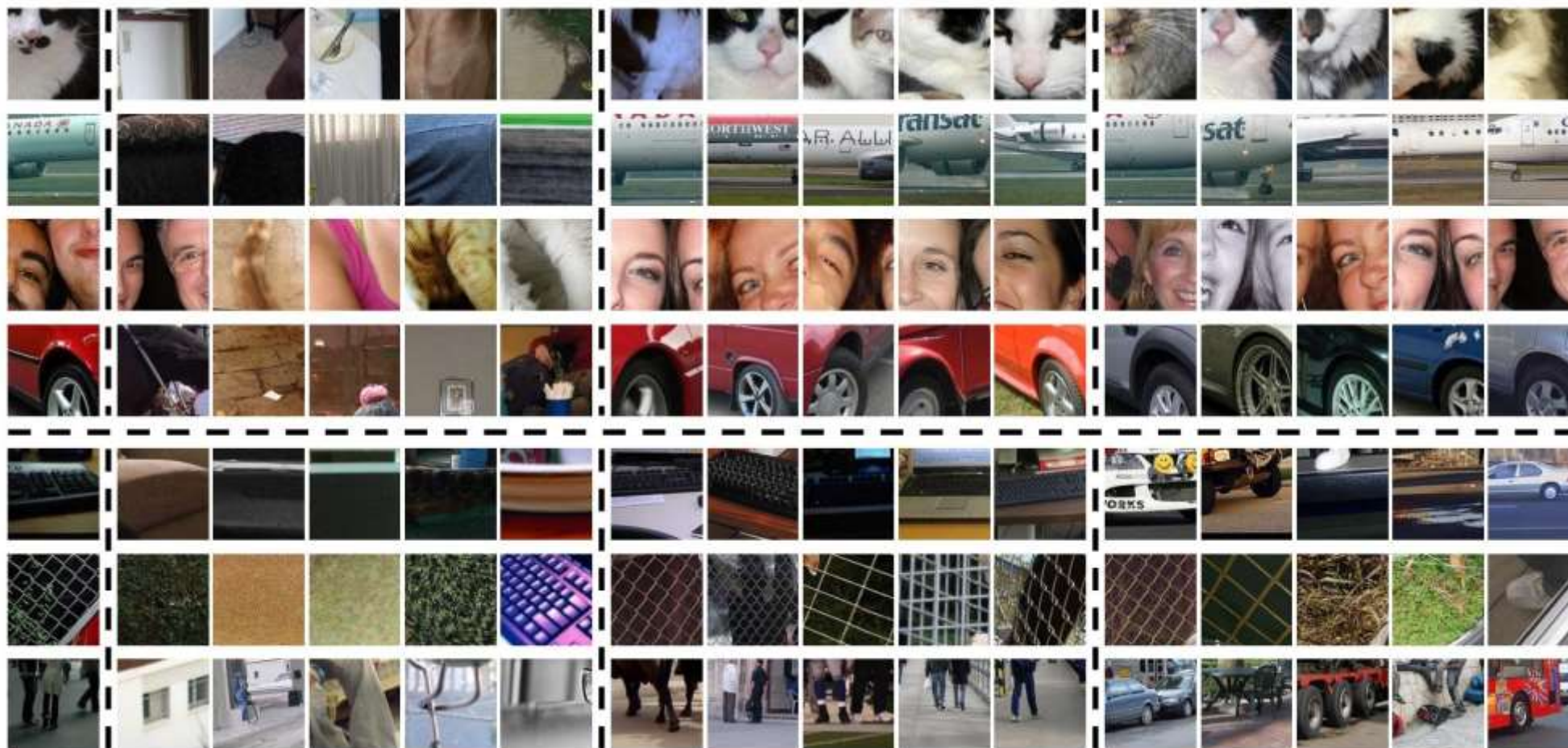
Random Init

Supervised AlexNet

Their Features

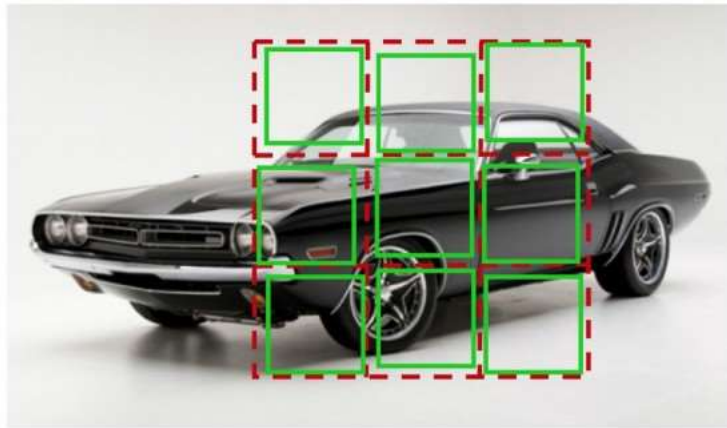
Works
well!
Similar to
AlexNet

Failure
modes



Extension: Solving Jigsaw Puzzles

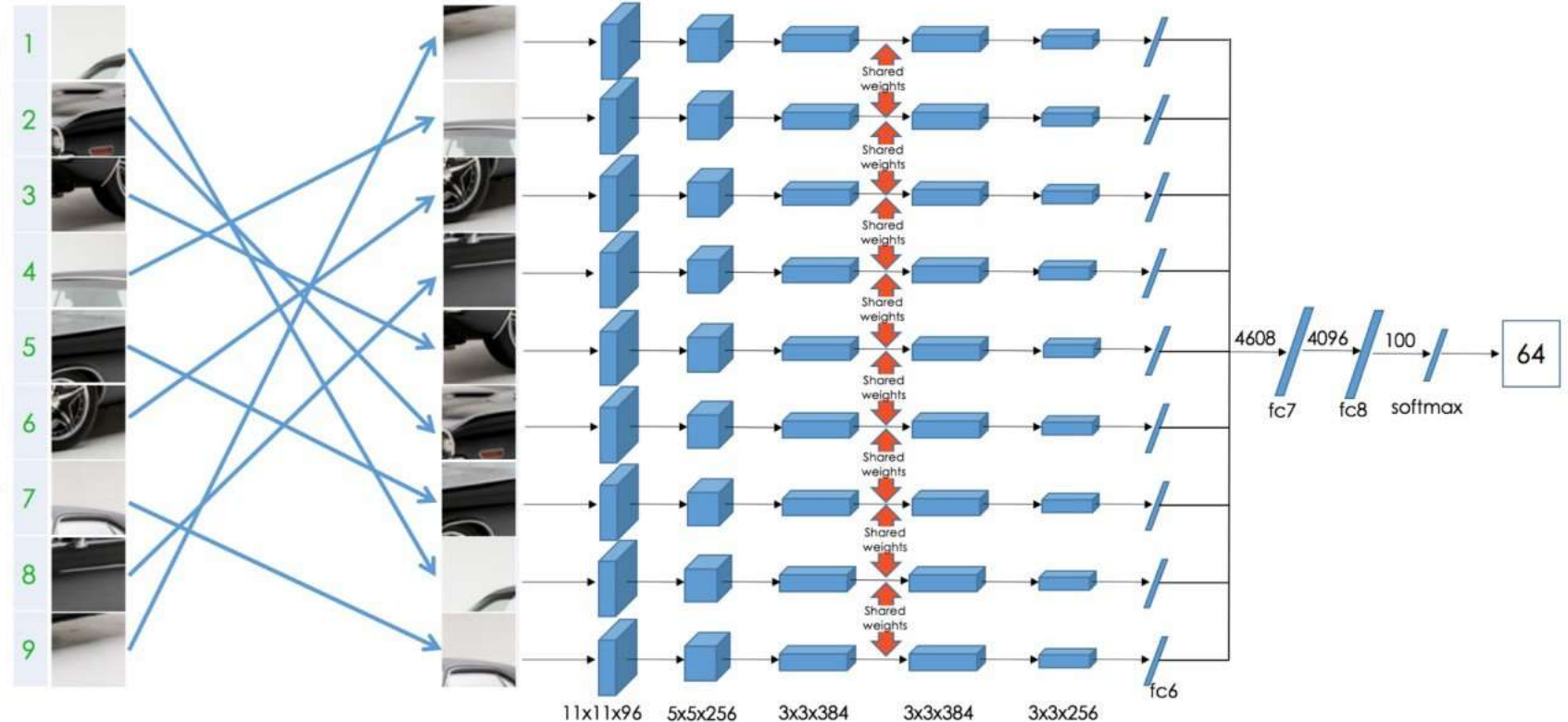
Rather than predict relative position of two patches, instead predict permutation to “unscramble” 9 shuffled patches



Permutation Set

index	permutation
64	9,4,6,8,3,2,5,1,7

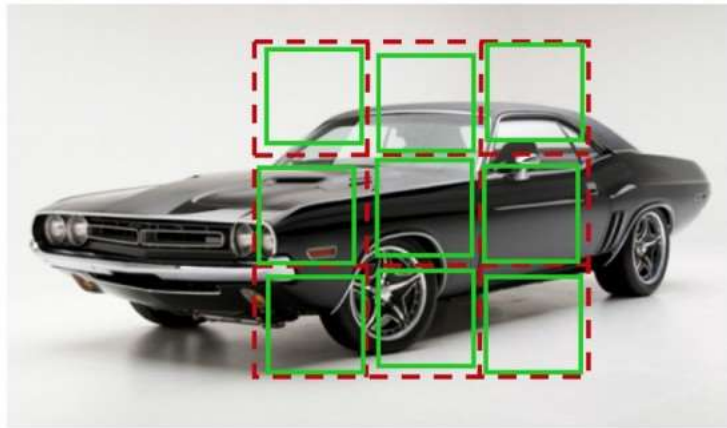
Reorder patches according to the selected permutation



Extension: Solving Jigsaw Puzzles

Problem: These methods
only work on patches,
not whole images!

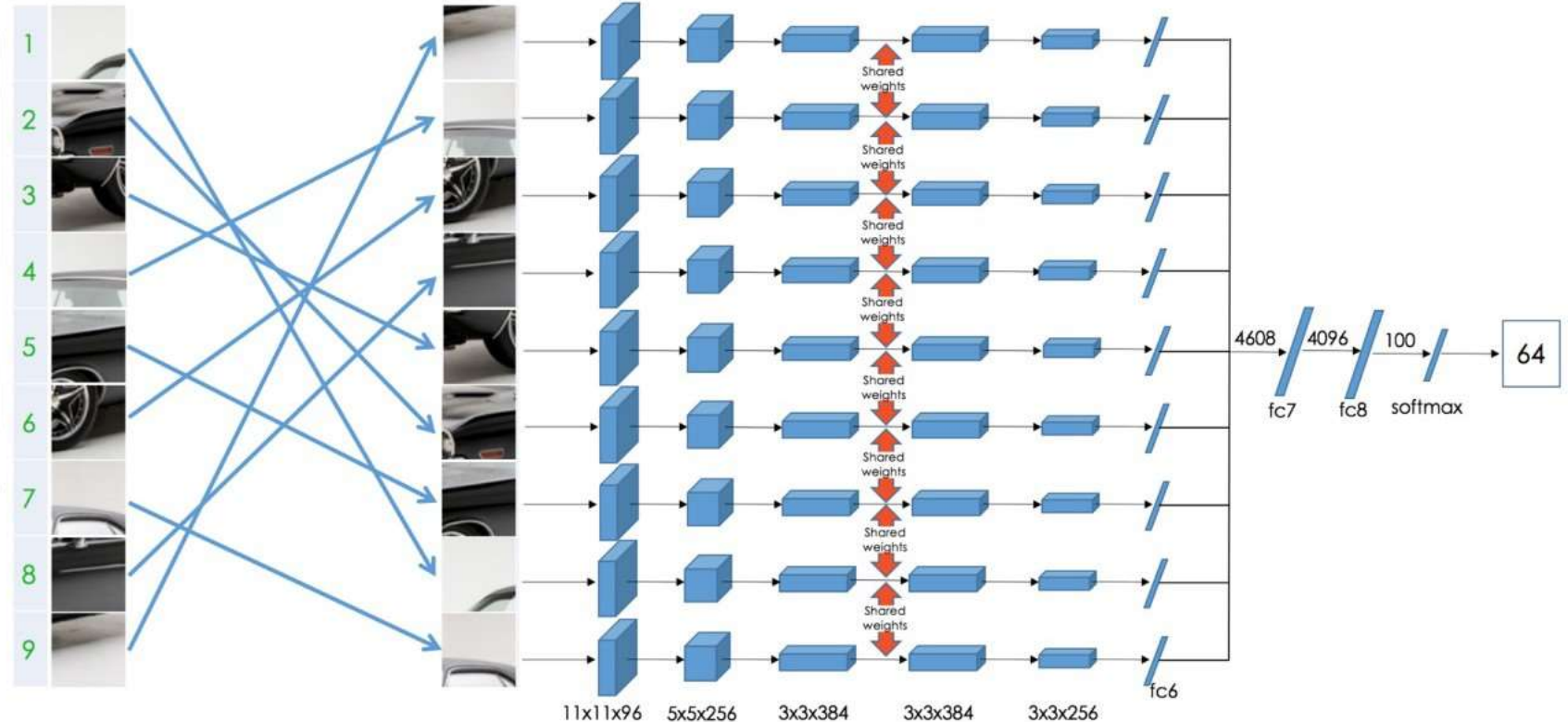
Rather than predict relative position of two patches, instead
predict permutation to “unscramble” 9 shuffled patches



Permutation Set

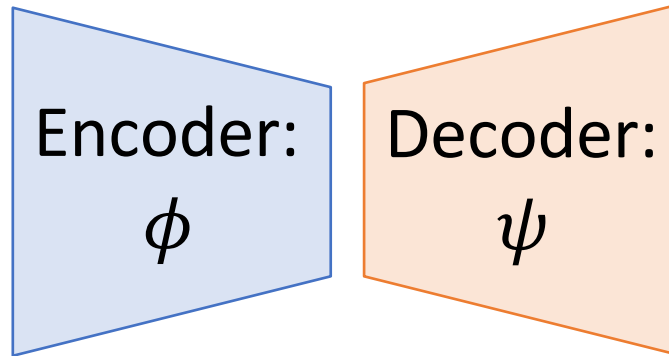
index	permutation
64	9,4,6,8,3,2,5,1,7

Reorder patches according to the selected permutation



Context Encoders: Learning by Inpainting

Input Image



Context Encoders: Learning by Inpainting

Input Image



Encoder:
 ϕ

Decoder:
 ψ

Predict Missing Pixels



Human Artist

Context Encoders: Learning by Inpainting

Input Image



Encoder:
 ϕ

Decoder:
 ψ

Predict Missing Pixels



L2 Loss
(Best for feature learning)

Context Encoders: Learning by Inpainting

Input Image



Encoder:
 ϕ

Decoder:
 ψ

Predict Missing Pixels



L2 + Adversarial Loss
(Best for nice images)

Colorization

Intuition: A model must be able to identify objects to be able to colorize them

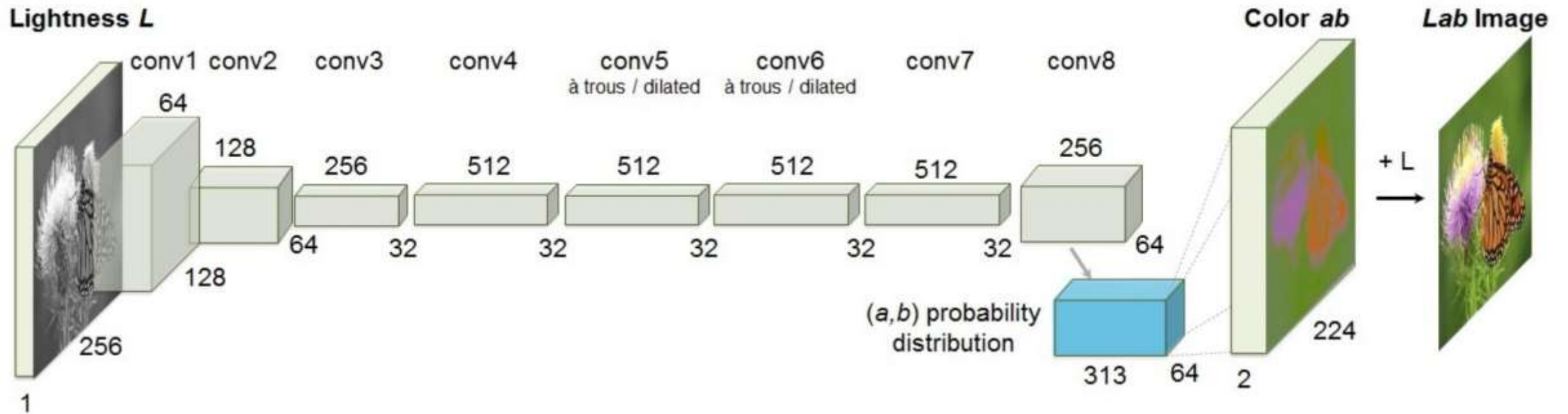


Input: Grayscale Image

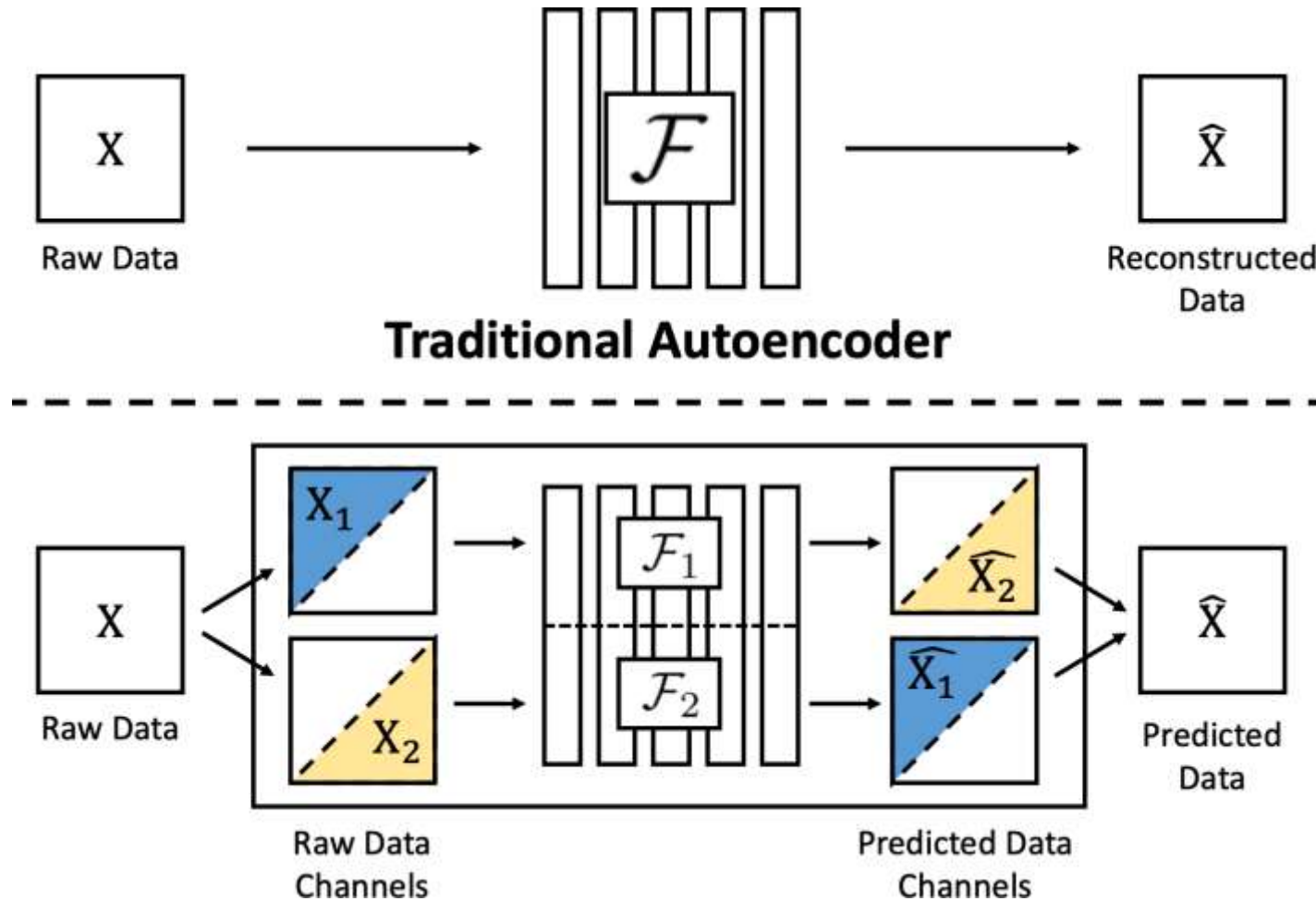


Output: Color Image

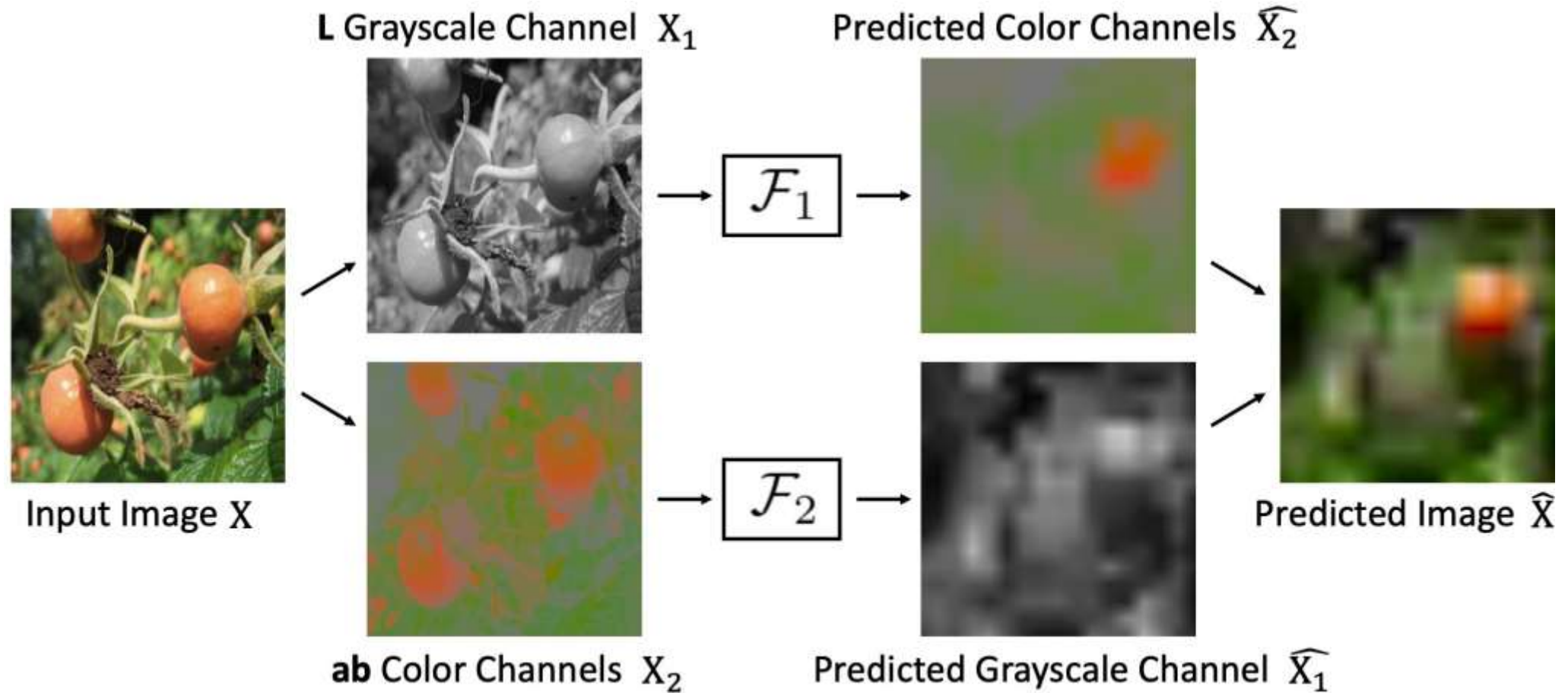
Colorization



Colorization Extension: Split-Brain Autoencoder

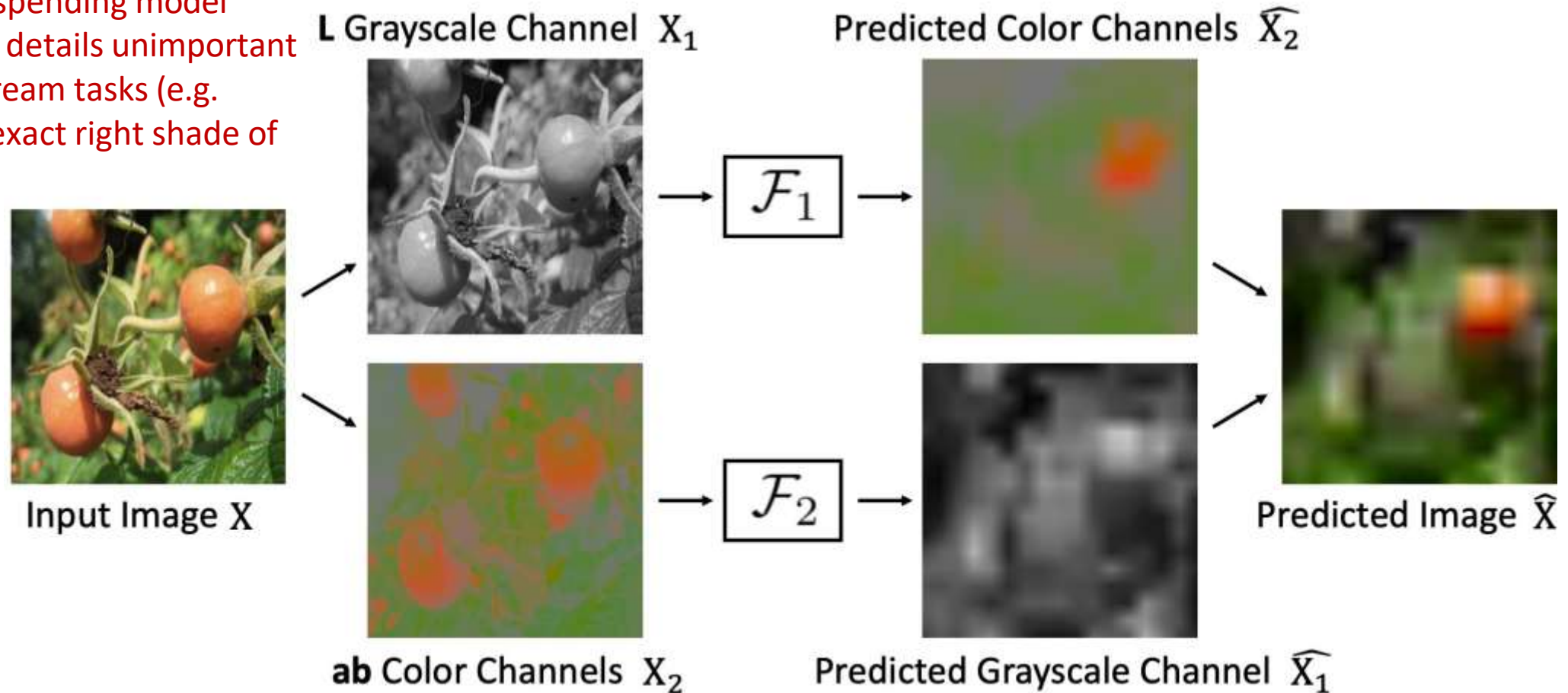


Colorization Extension: Split-Brain Autoencoder



Colorization Extension: Split-Brain Autoencoder

Concern: Generative pretexts encourage spending model capacity on details unimportant for downstream tasks (e.g. regressing exact right shade of orange)

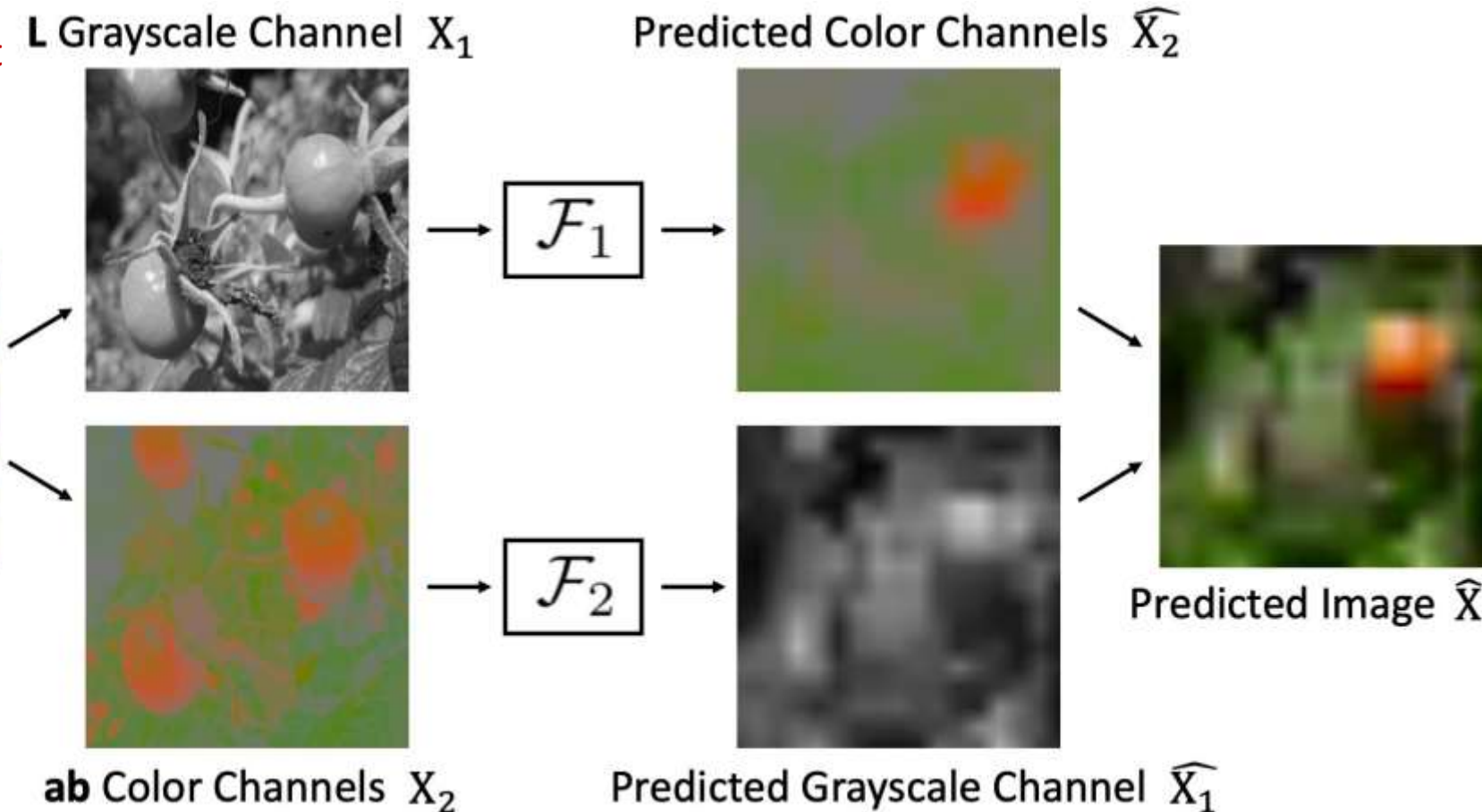


Colorization Extension: Split-Brain Autoencoder

Concern: Generative pretexts encourage spending model capacity on details unimportant for downstream tasks (e.g. regressing exact right shade of orange)



Input Image X



Solution: Discriminative pretext tasks that require classification, not generation

Deep Clustering

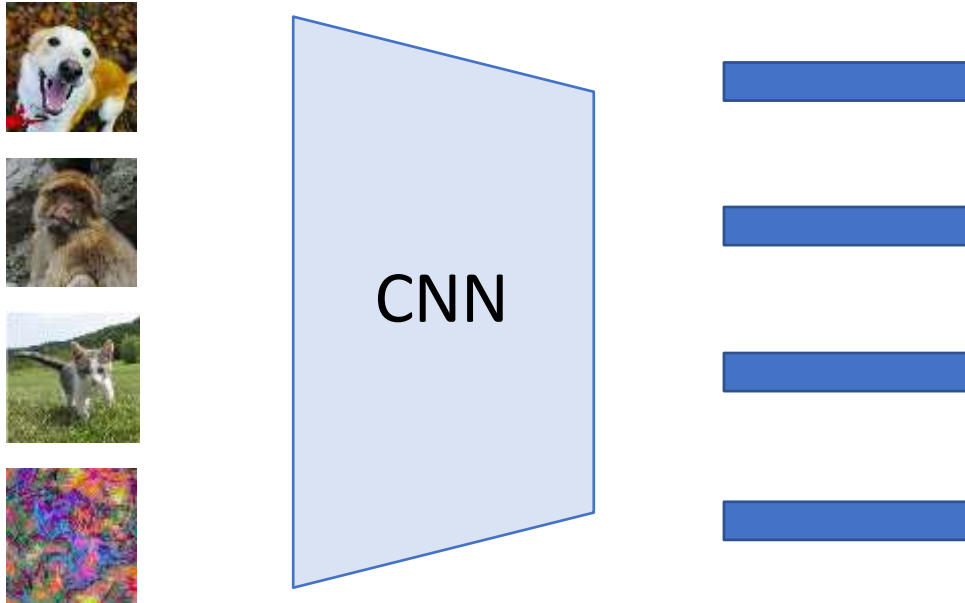
(1) Randomly initialize a CNN



Caron et al, "Deep Clustering for Unsupervised Learning of Visual Features", ECCV 2018
Caron et al, "Unsupervised Pre-Training of Image Features on Non-Curated Data", ICCV 2019
Yan et al, "ClusterFit: Improving Generalization of Visual Representations", CVPR 2020
Caron et al, "Unsupervised Learning of Visual Features by Contrasting Cluster Assignments", NeurIPS 2020

Deep Clustering

(1) Randomly initialize a CNN

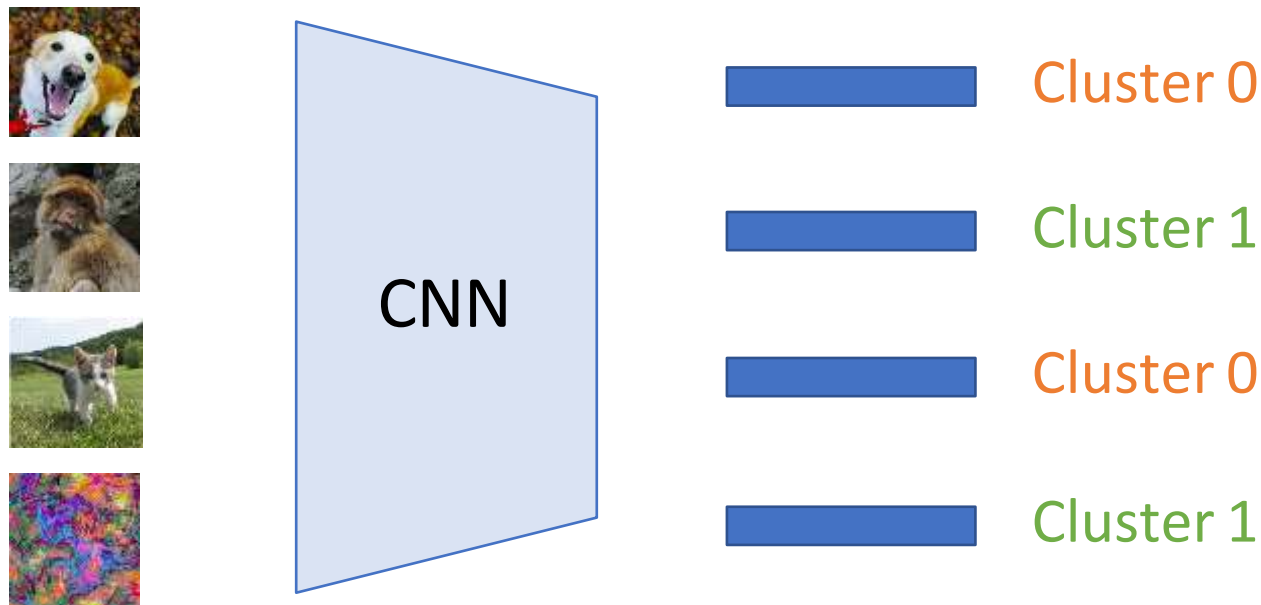


(2) Run many images through
CNN, get their final-layer features

Caron et al, "Deep Clustering for Unsupervised Learning of Visual Features", ECCV 2018
Caron et al, "Unsupervised Pre-Training of Image Features on Non-Curated Data", ICCV 2019
Yan et al, "ClusterFit: Improving Generalization of Visual Representations", CVPR 2020
Caron et al, "Unsupervised Learning of Visual Features by Contrasting Cluster Assignments", NeurIPS 2020

Deep Clustering

(1) Randomly initialize a CNN

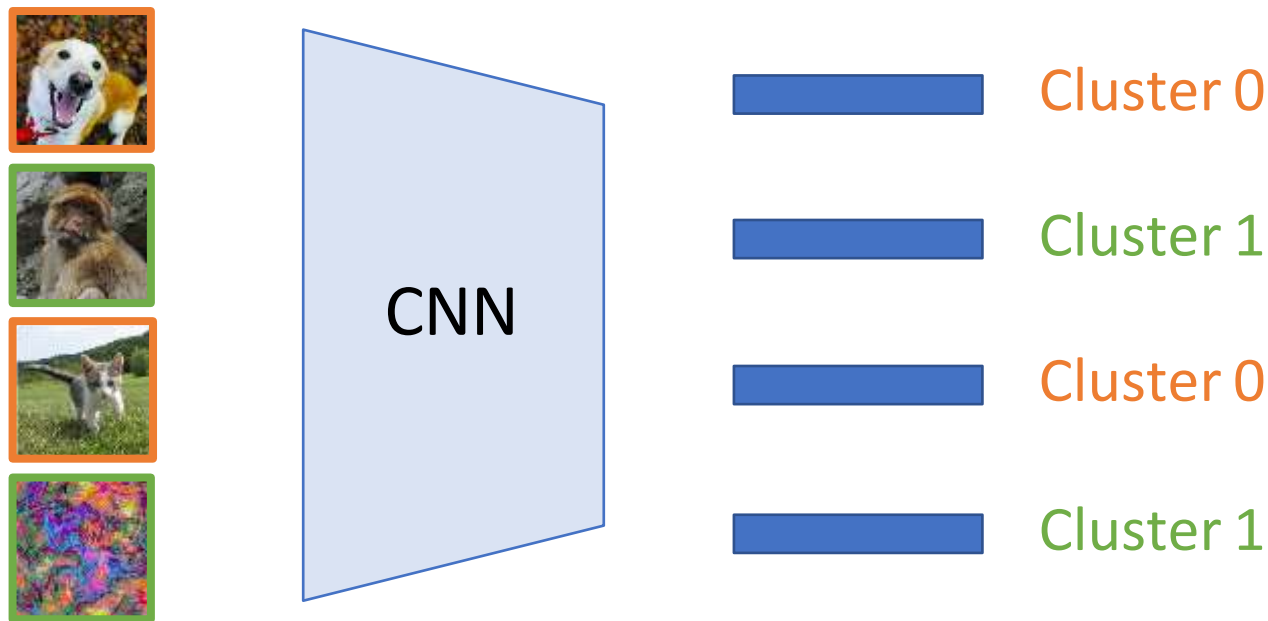


(3) Cluster the features with K-Means;
record cluster for each feature

(2) Run many images through
CNN, get their final-layer features

Deep Clustering

(1) Randomly initialize a CNN



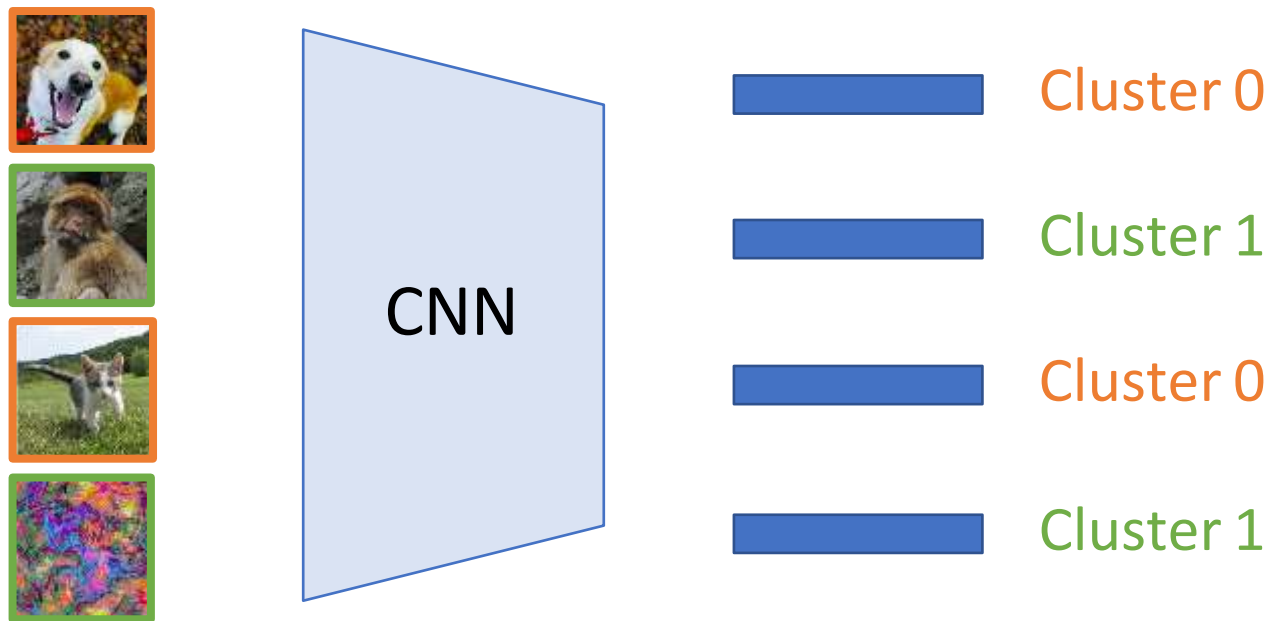
(2) Run many images through CNN, get their final-layer features

(3) Cluster the features with K-Means; record cluster for each feature

(4) Use cluster assignments as pseudo-labels for each image; train the CNN to predict cluster assignments

Deep Clustering

(1) Randomly initialize a CNN



(2) Run many images through CNN, get their final-layer features

(3) Cluster the features with K-Means; record cluster for each feature

(4) Use cluster assignments as pseudo-labels for each image; train the CNN to predict cluster assignments

(5) Repeat: GOTO (2)

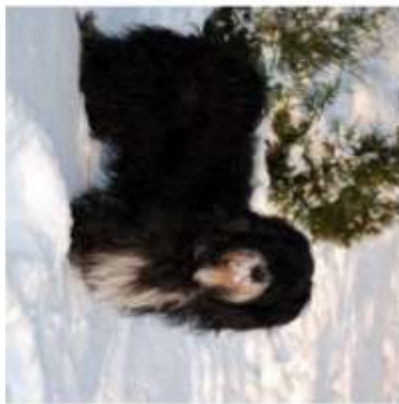
RotNet: Predict Rotation

4-way classification task: How much was each image rotated? (0, 90, 180, or 270 degrees)



RotNet: Predict Rotation

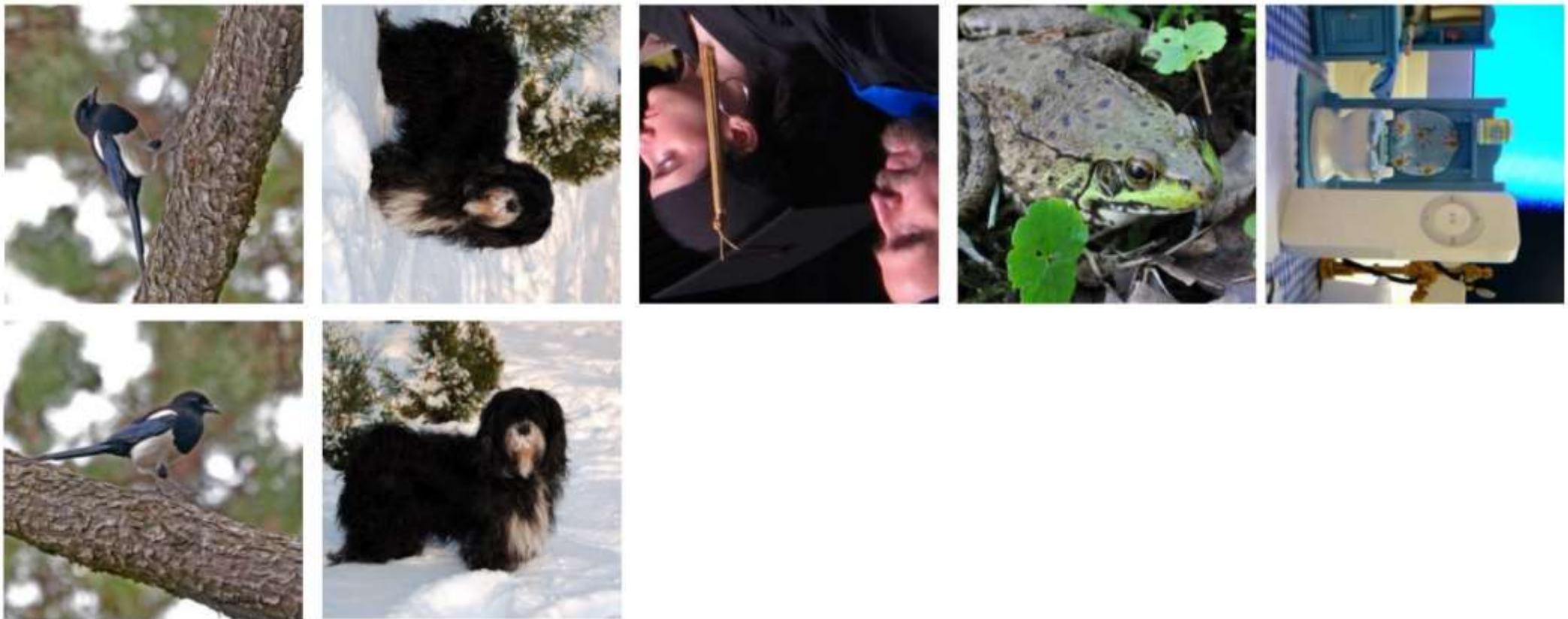
4-way classification task: How much was each image rotated? (0, 90, 180, or 270 degrees)



90

RotNet: Predict Rotation

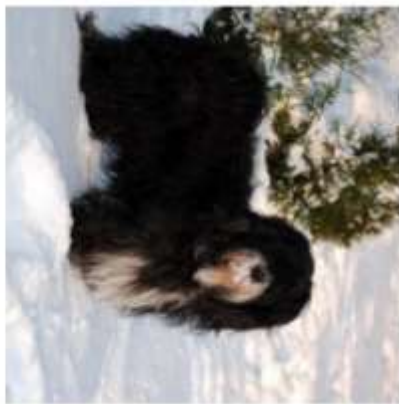
4-way classification task: How much was each image rotated? (0, 90, 180, or 270 degrees)



90

RotNet: Predict Rotation

4-way classification task: How much was each image rotated? (0, 90, 180, or 270 degrees)



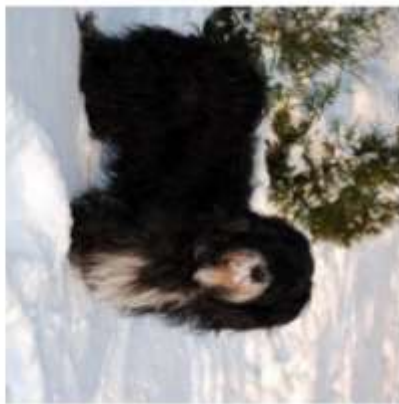
90

270

180

RotNet: Predict Rotation

4-way classification task: How much was each image rotated? (0, 90, 180, or 270 degrees)



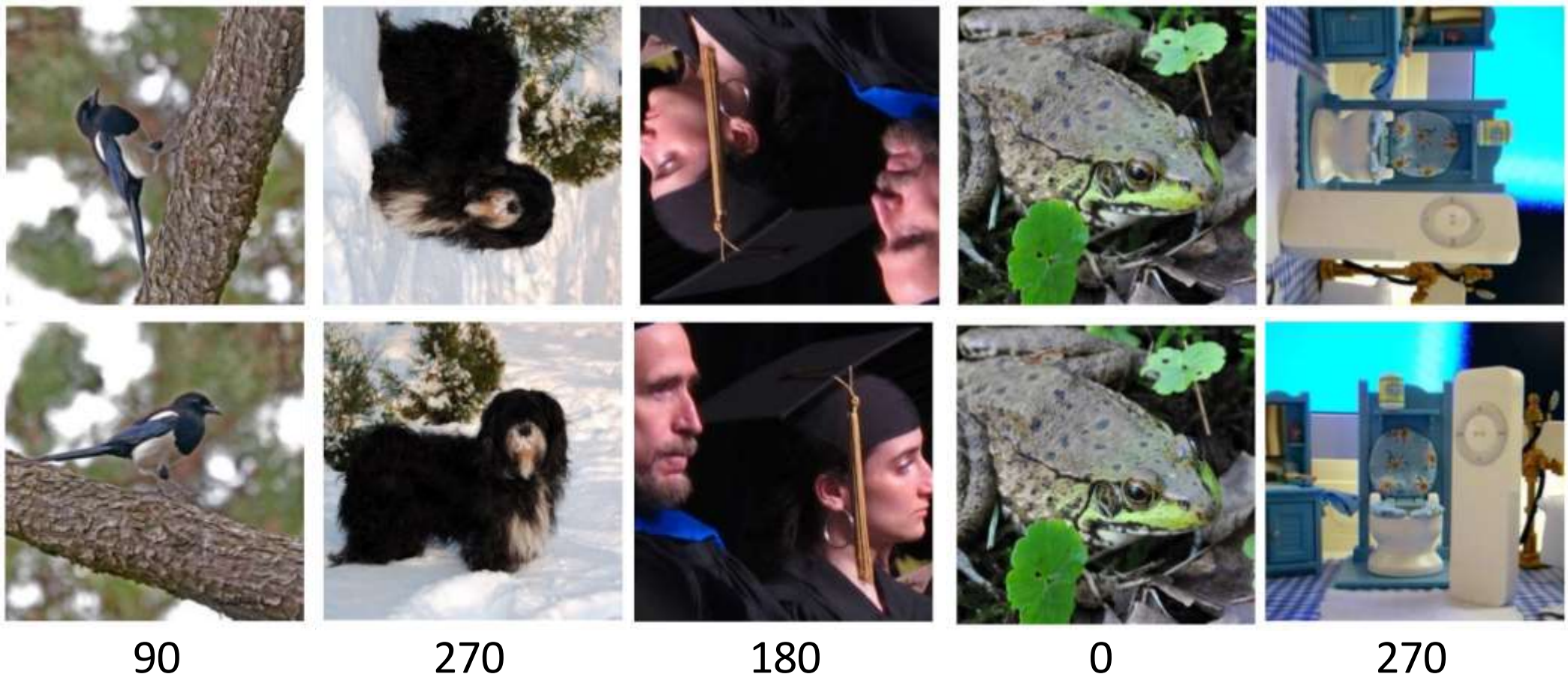
90

270

180

RotNet: Predict Rotation

4-way classification task: How much was each image rotated? (0, 90, 180, or 270 degrees)



Which SSL Method is best?

Fair evaluation of SSL methods is very hard! No theory, so we need to rely on experiment

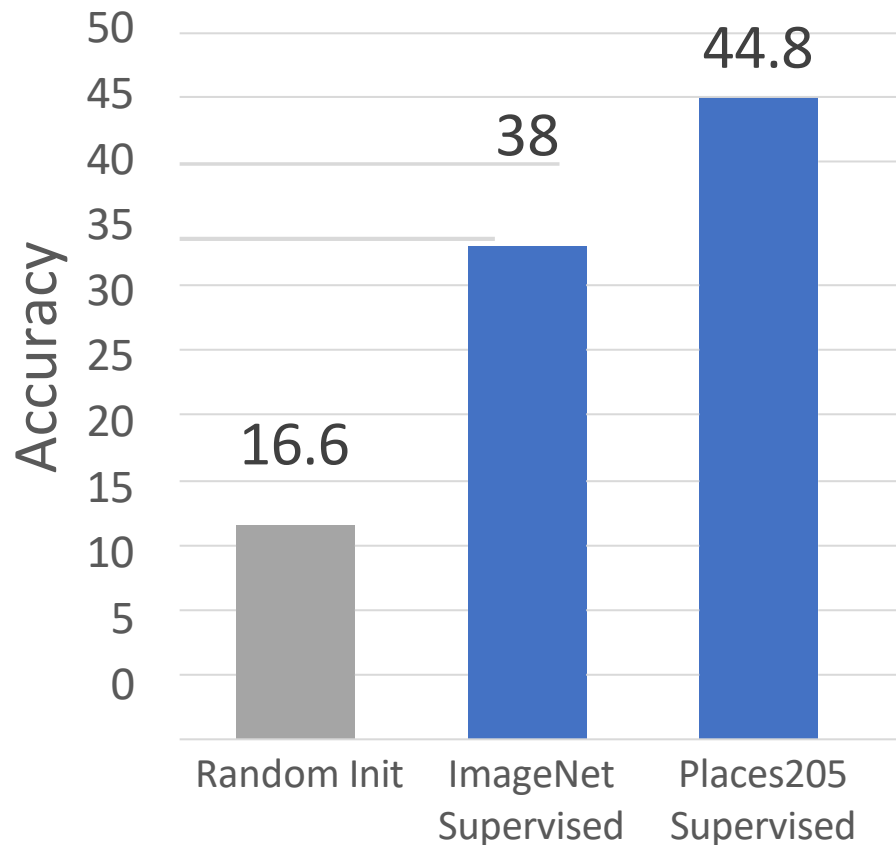
Many choices in experimental setup, huge variations from paper to paper:

- CNN architecture? AlexNet, ResNet50, something else?
- Pretraining dataset? ImageNet, or something else?
- Downstream task? ImageNet classification, detection, something else?
- Pretraining hyperparameters? Learning rates, training iterations, data augmentation?
- Transfer learning protocol?
 - Linear probe? From which layer? How to train linear models? SGD, something else?
Transfer learning hyperparameters? Data augmentation or BatchNorm during transfer learning?
 - Fine-tune? From which layer? Architecture of “head” you attach? Linear or nonlinear? Fine-tuning hyperparameters?
 - KNN? What value of K? Normalization on features?

Which SSL Method is best?

Some papers have tried to do fair comparisons of many SSL methods

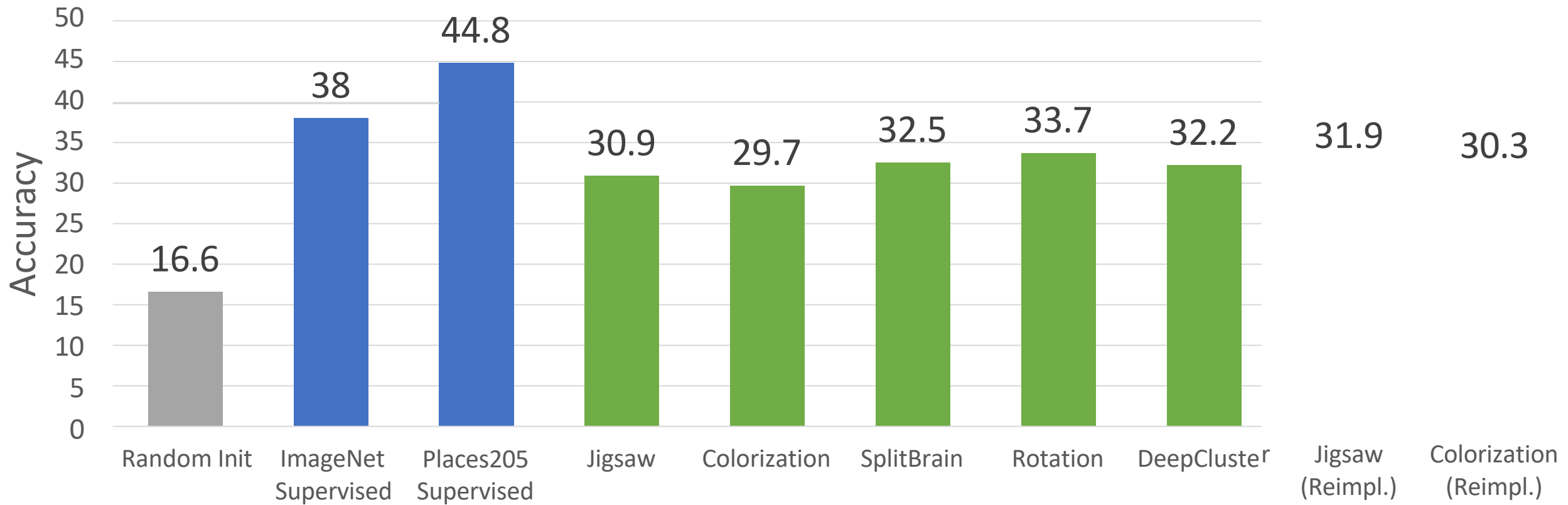
Places205 Linear Classification from AlexNet conv5



Which SSL Method is best?

Some papers have tried to do fair comparisons of many SSL methods

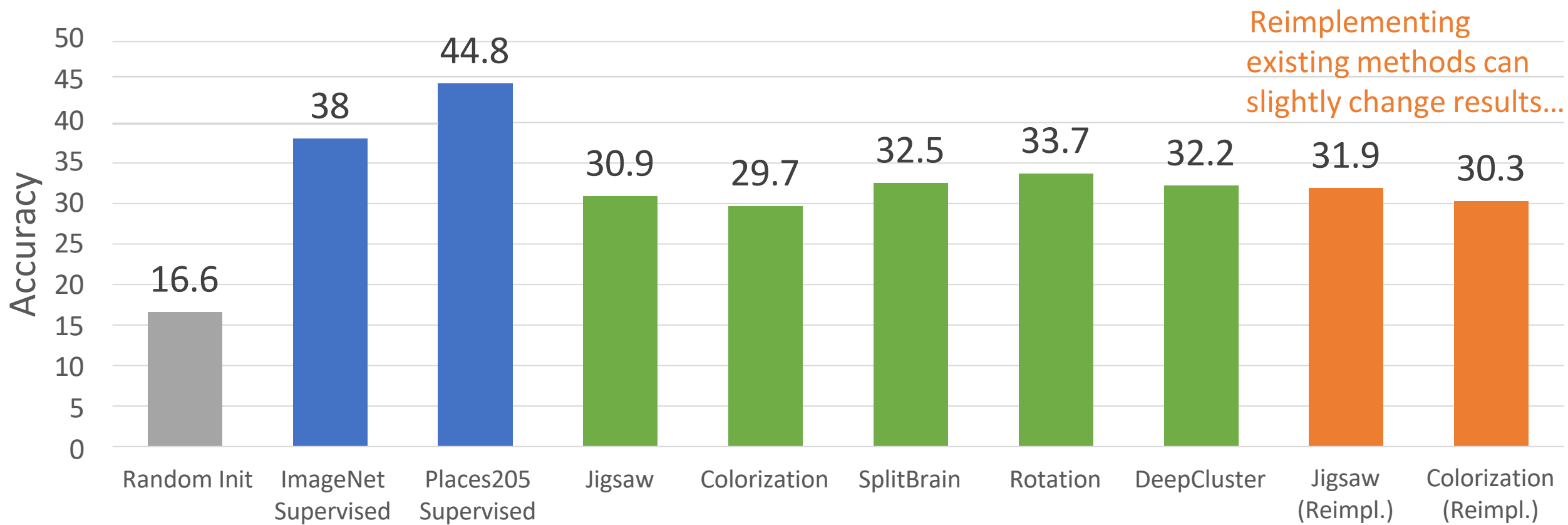
Places205 Linear Classification from AlexNet conv5



Which SSL Method is best?

Some papers have tried to do fair comparisons of many SSL methods

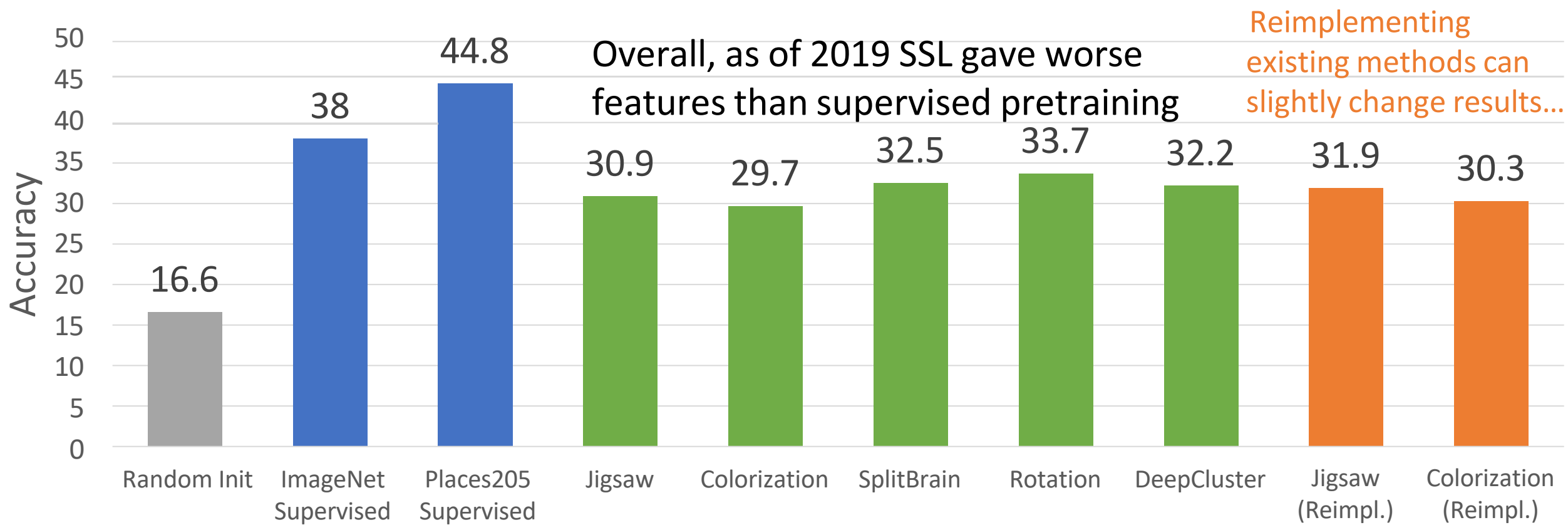
Places205 Linear Classification from AlexNet conv5



Which SSL Method is best?

Some papers have tried to do fair comparisons of many SSL methods

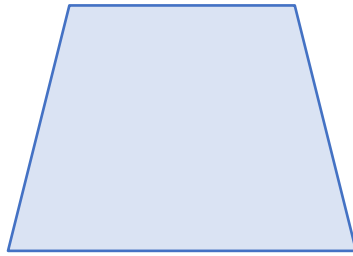
Places205 Linear Classification from AlexNet conv5



Self-Supervised Learning for Natural Language

Computer Vision

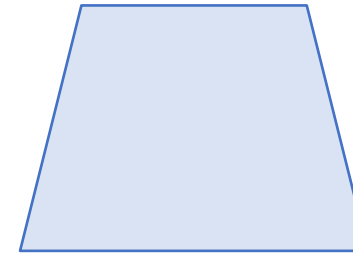
Image Features:
 $H \times W \times C$



Input Image

Natural Language Processing

Word Features
 $L \times C$



*A white and gray
cat standing outside
on the grass*

Input Sentence (L words)

Exemplar CNN: Invariance to Data Augmentation

Quiz: What is this?



Exemplar CNN: Invariance to Data Augmentation

Quiz: What is this?



Answer: Deer!

Exemplar CNN: Invariance to Data Augmentation

Quiz: What is this?

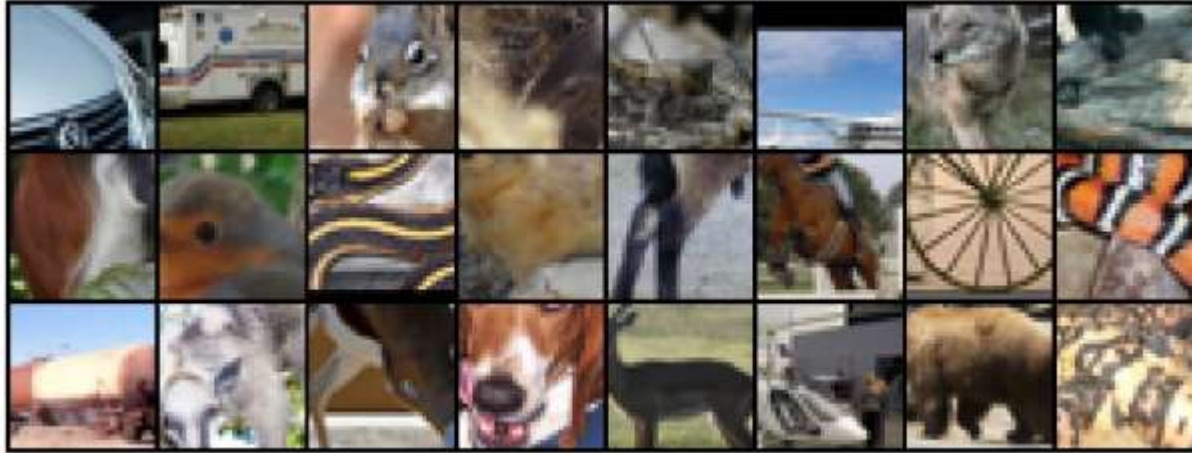


Different data augmentations (scale, shift, color jitter) of the same initial image patch

Answer: Deer!

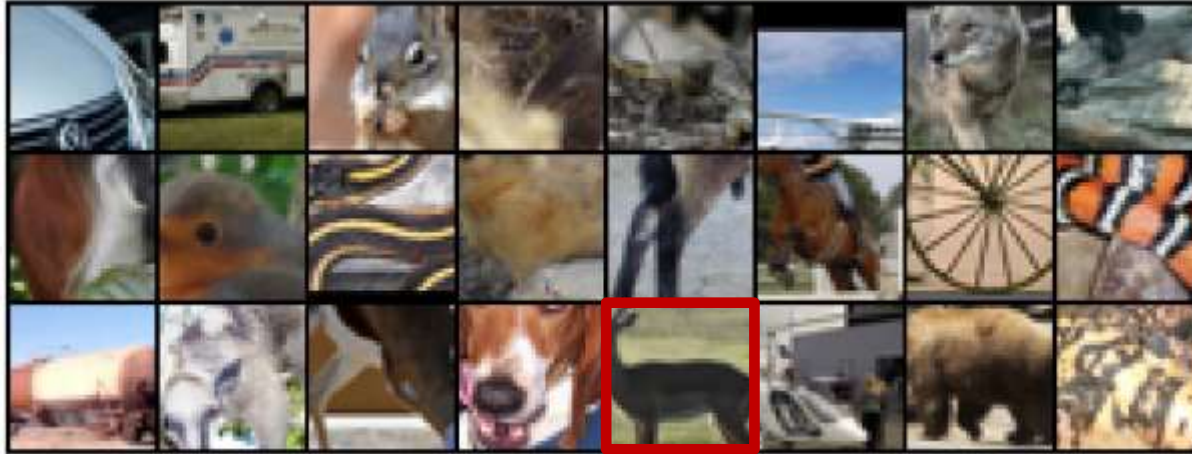
Exemplar CNN: Invariance to Data Augmentation

Given an initial
dataset of N
image patches

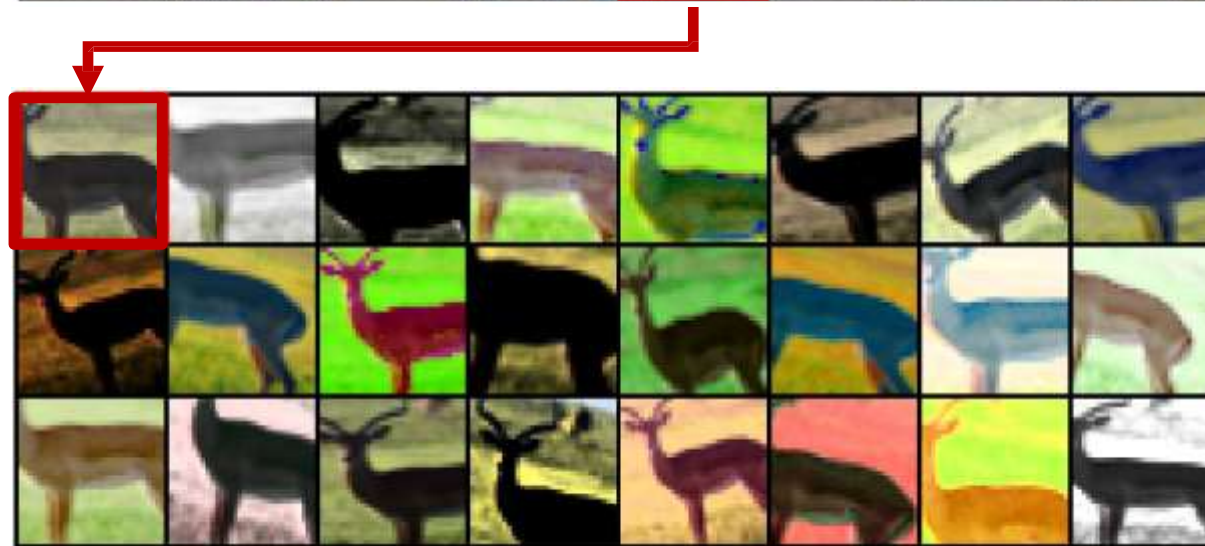


Exemplar CNN: Invariance to Data Augmentation

Given an initial dataset of N image patches

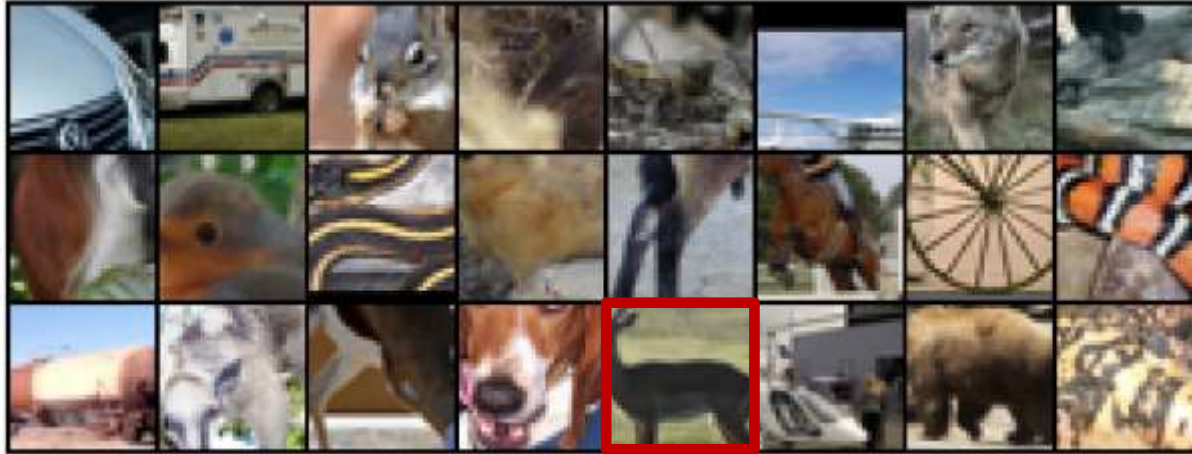


Sample K different augmentations for each; now have $K \times N$ total patches

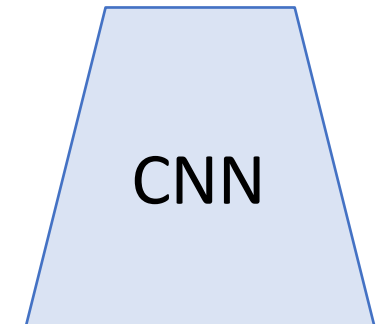
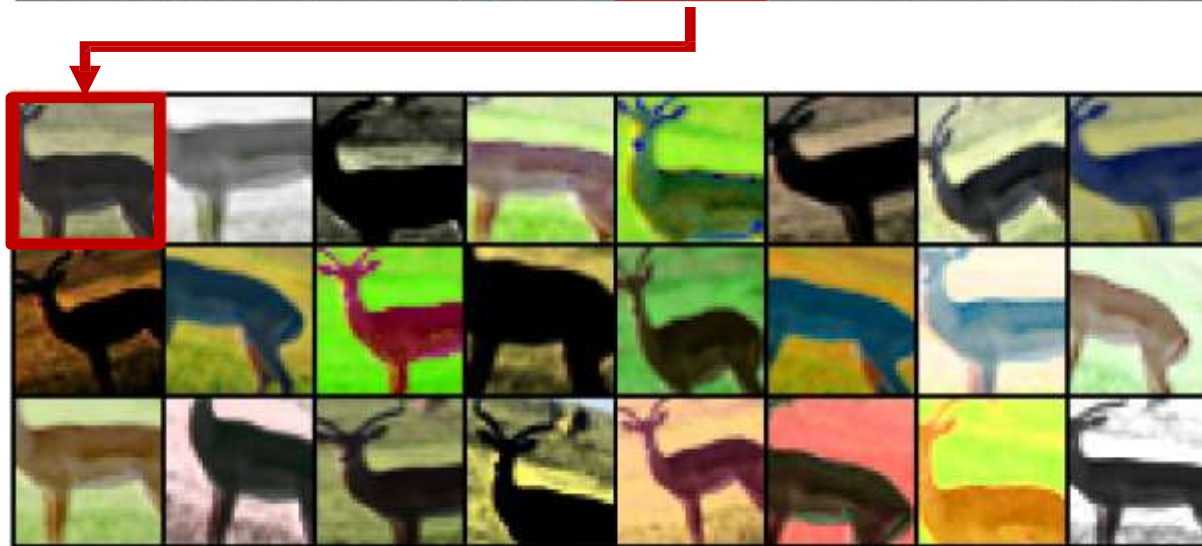


Exemplar CNN: Invariance to Data Augmentation

Given an initial dataset of N image patches



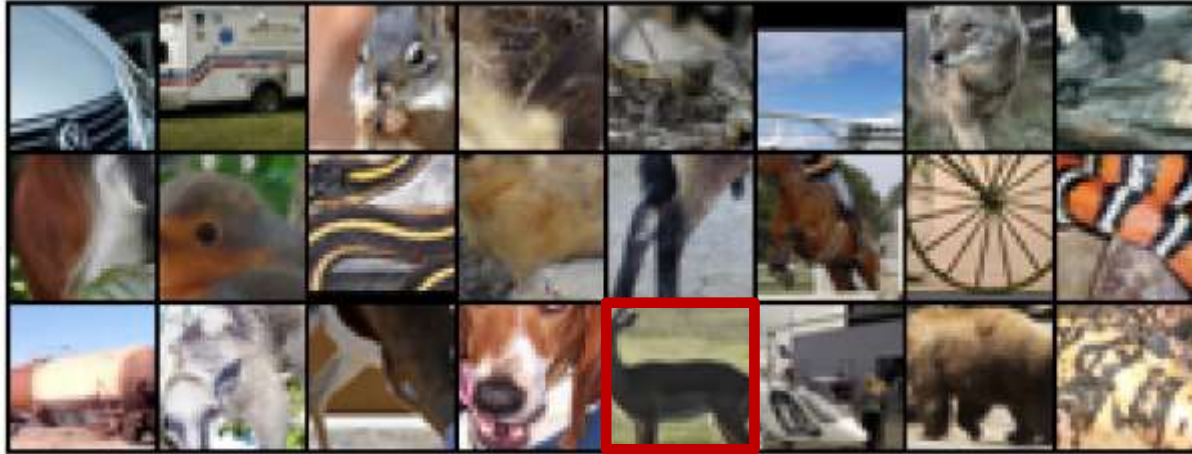
Sample K different augmentations for each; now have $K*N$ total patches



CNN inputs an augmented patch

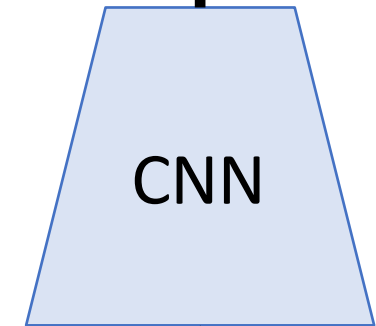
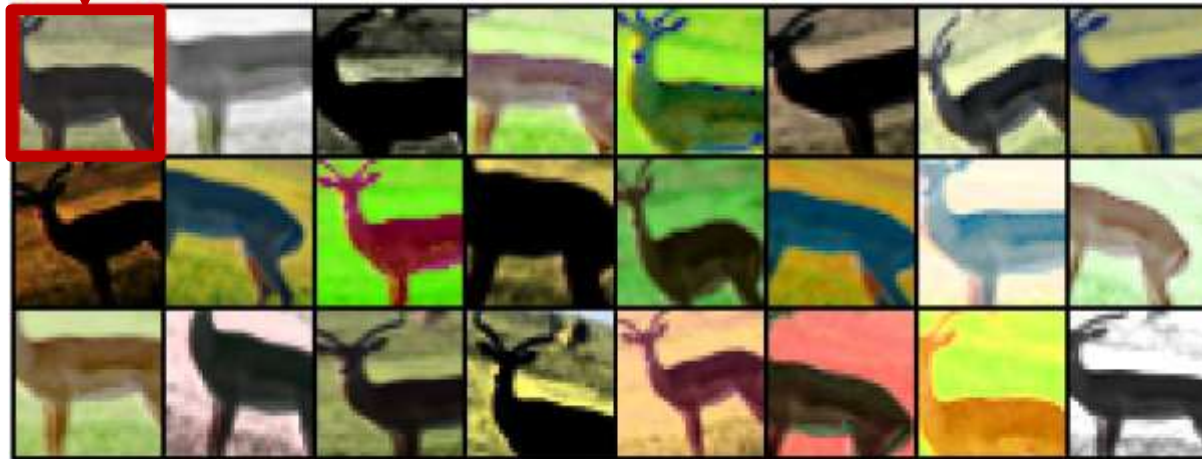
Exemplar CNN: Invariance to Data Augmentation

Given an initial dataset of N image patches



Predicts which of the N original images it came from (N-way classification)

Sample K different augmentations for each; now have $K*N$ total patches



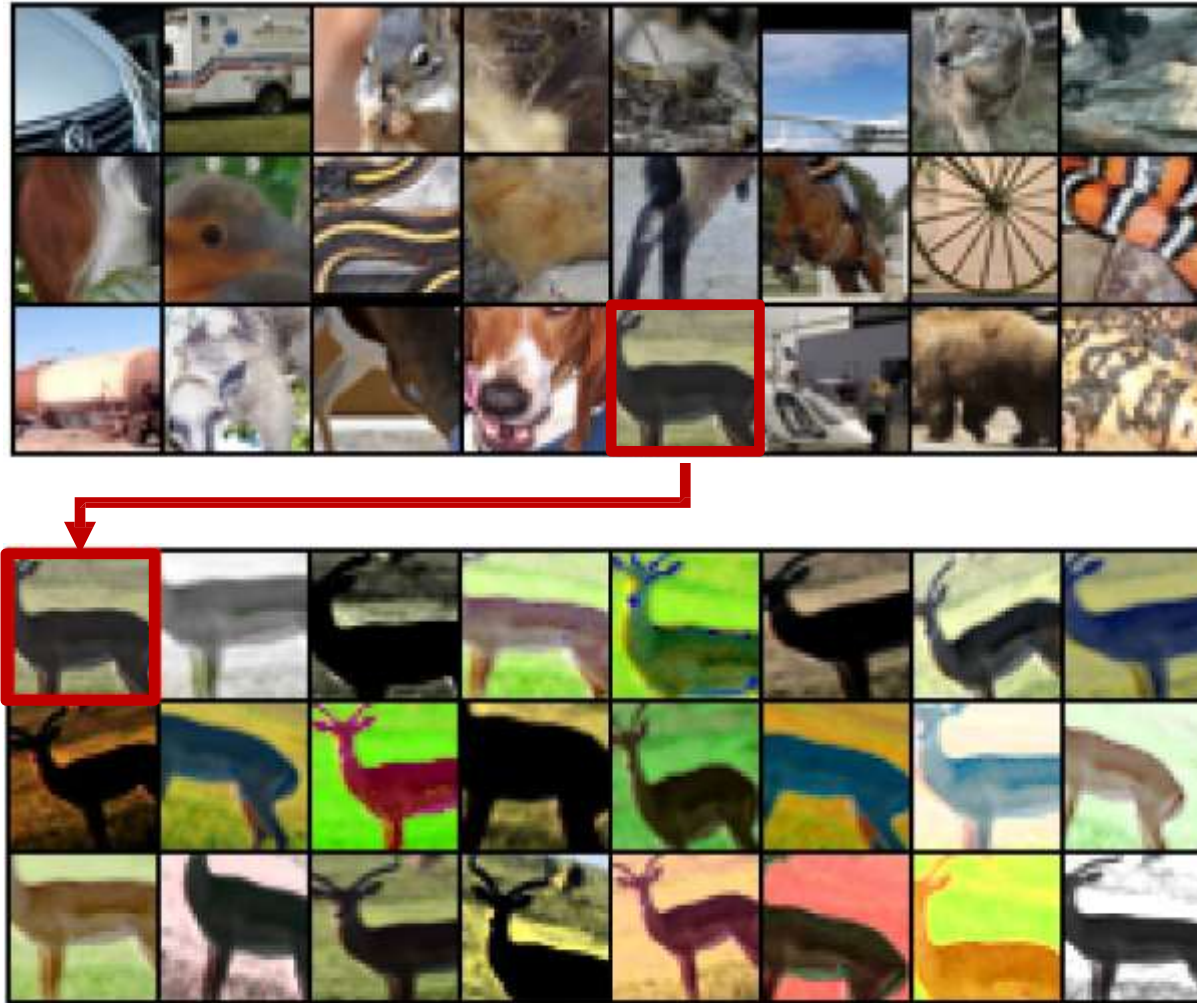
CNN inputs an augmented patch

Exemplar CNN: Invariance to Data Augmentation

Given an initial dataset of N image patches

Problem: number of parameters in final layer depends on N ; hard to scale

Sample K different augmentations for each; now have $K*N$ total patches



Predicts which of the N original images it came from (N -way classification)

CNN inputs an augmented patch

Contrastive representation learning

- Intuition and formulation
- Instance contrastive learning: SimCLR and MOCO

Pretext tasks from image transformations

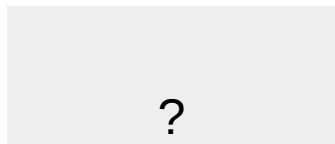
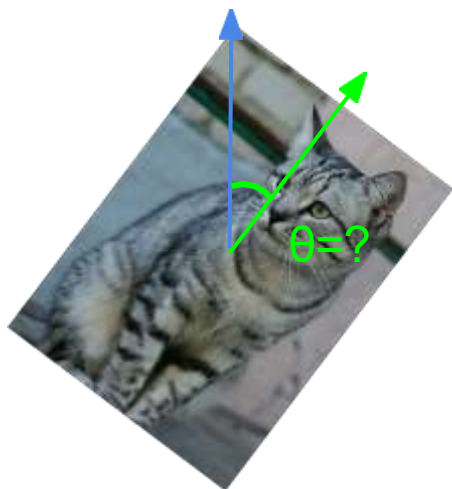
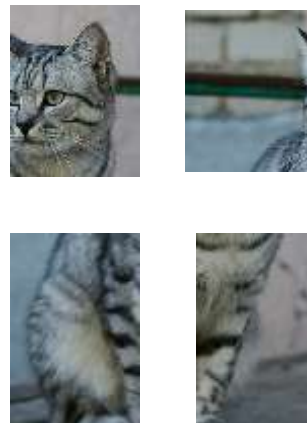


image completion



rotation prediction



“jigsaw puzzle”

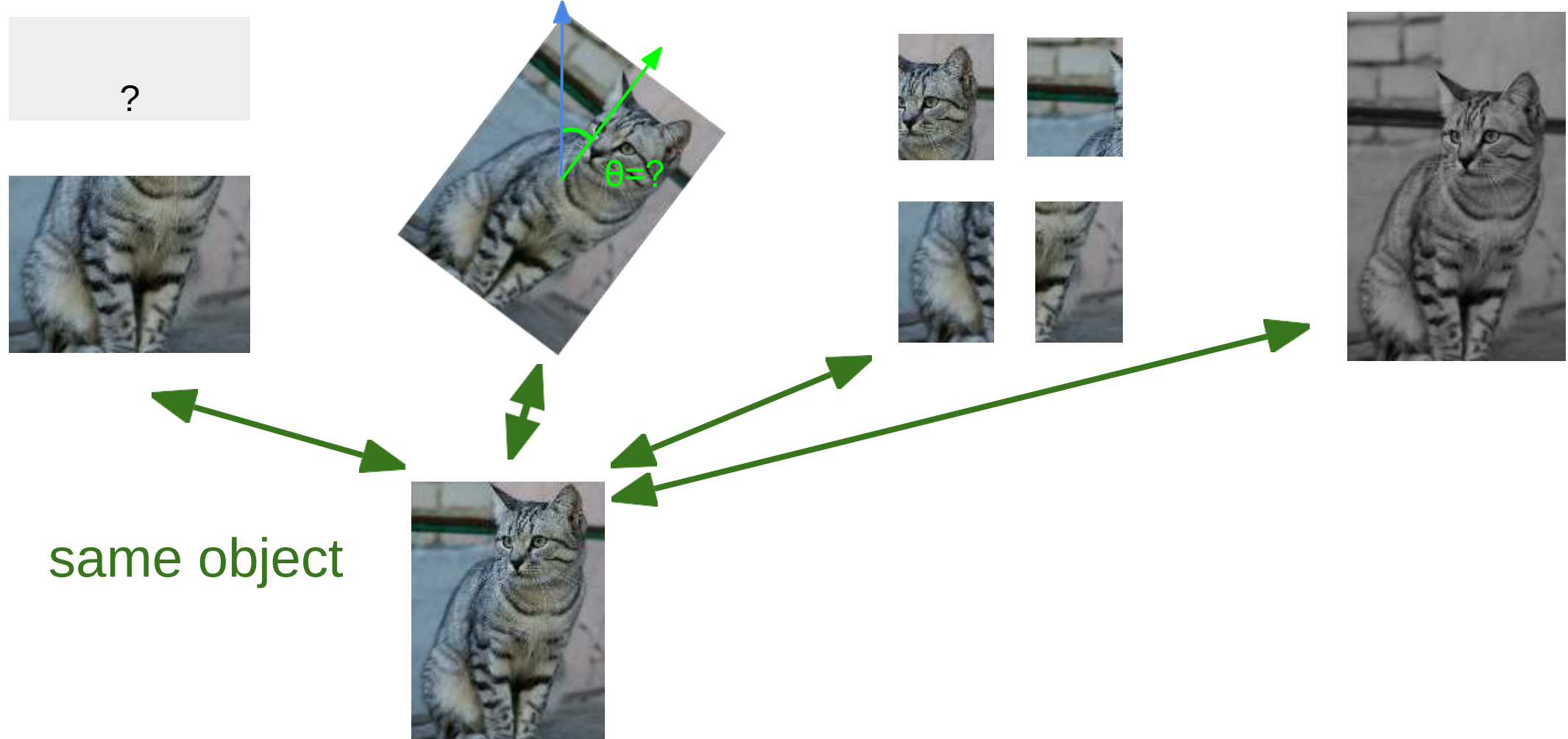


colorization

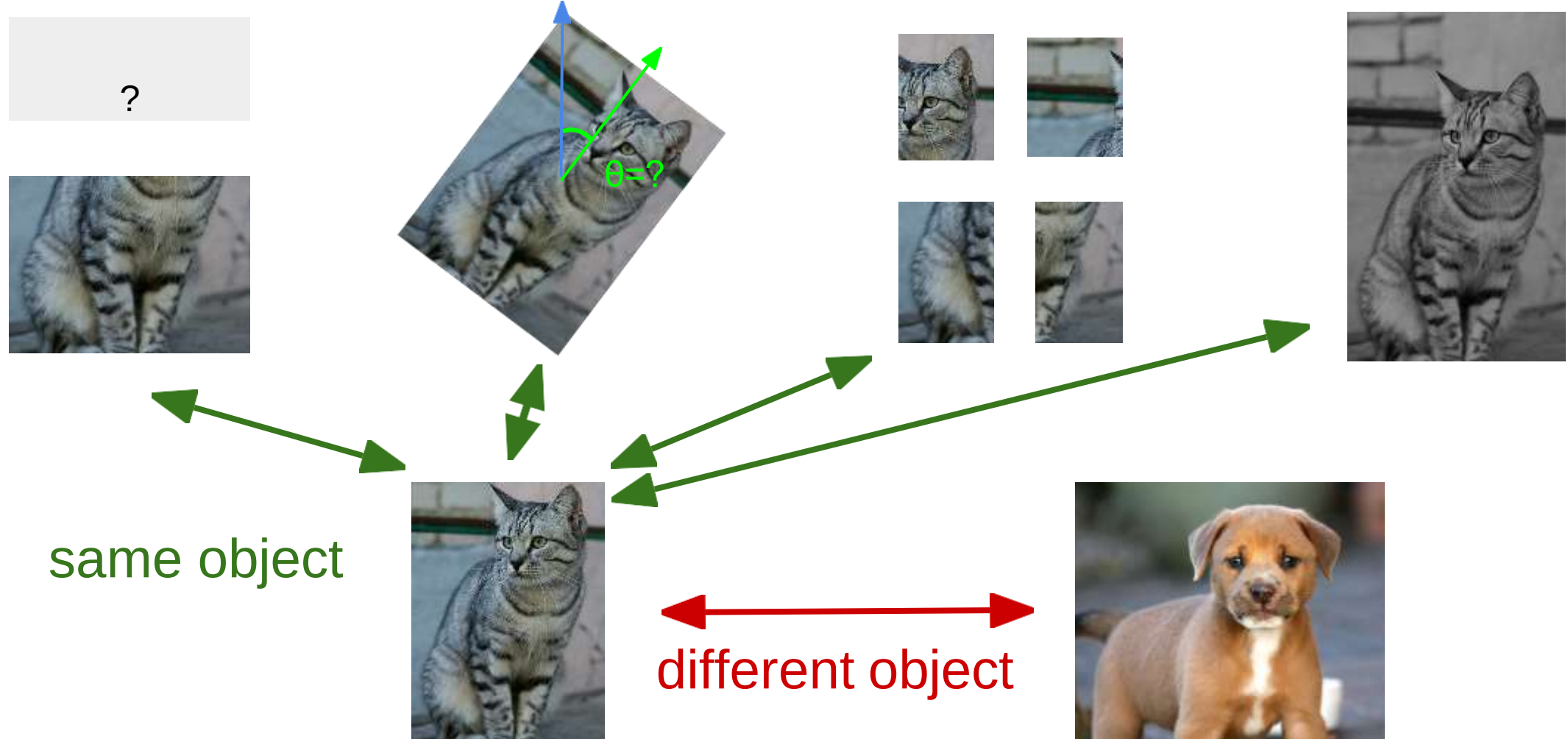
Learned representations may be tied to a specific pretext task!

Can we come up with a more general pretext task?

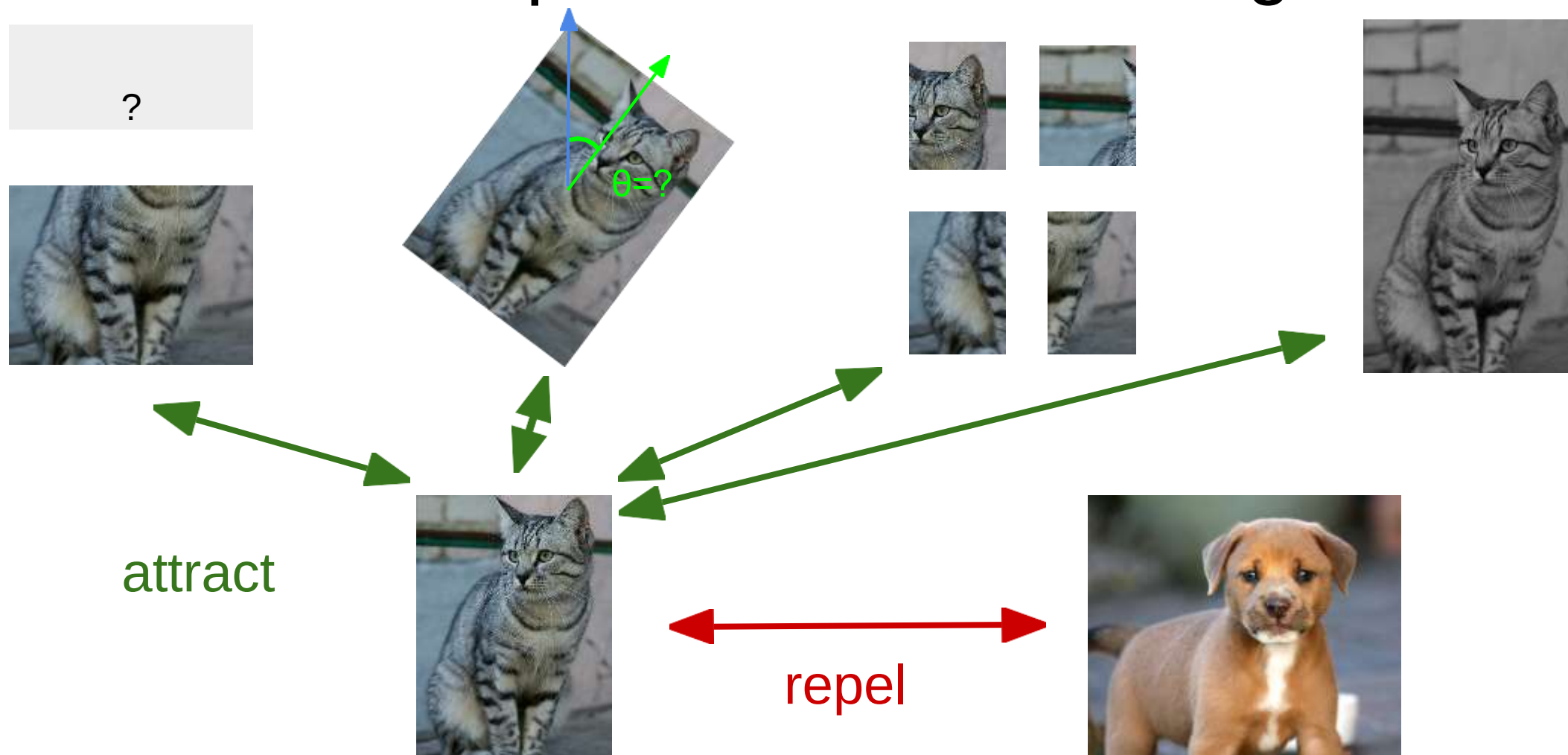
A more general pretext task?



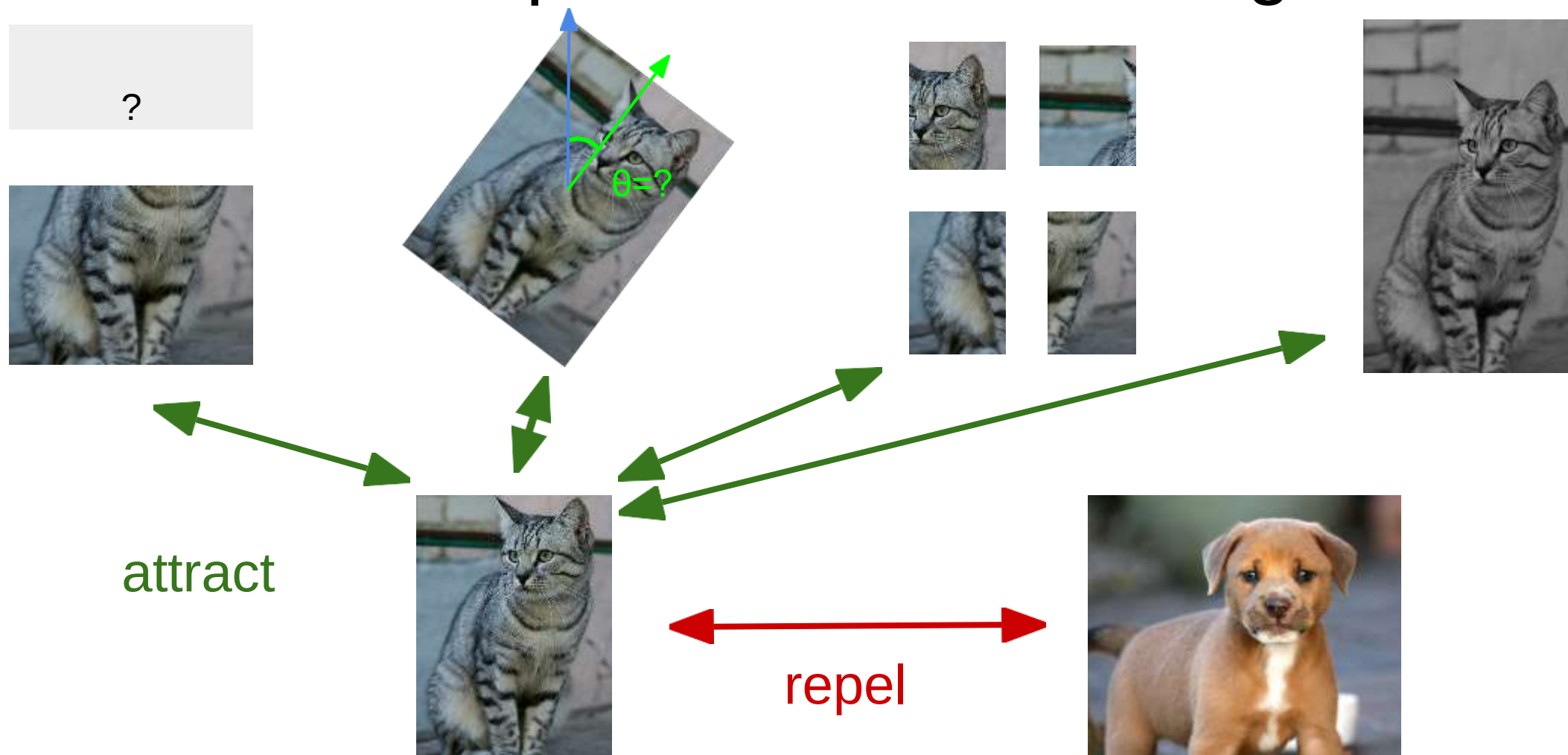
A more general pretext task?



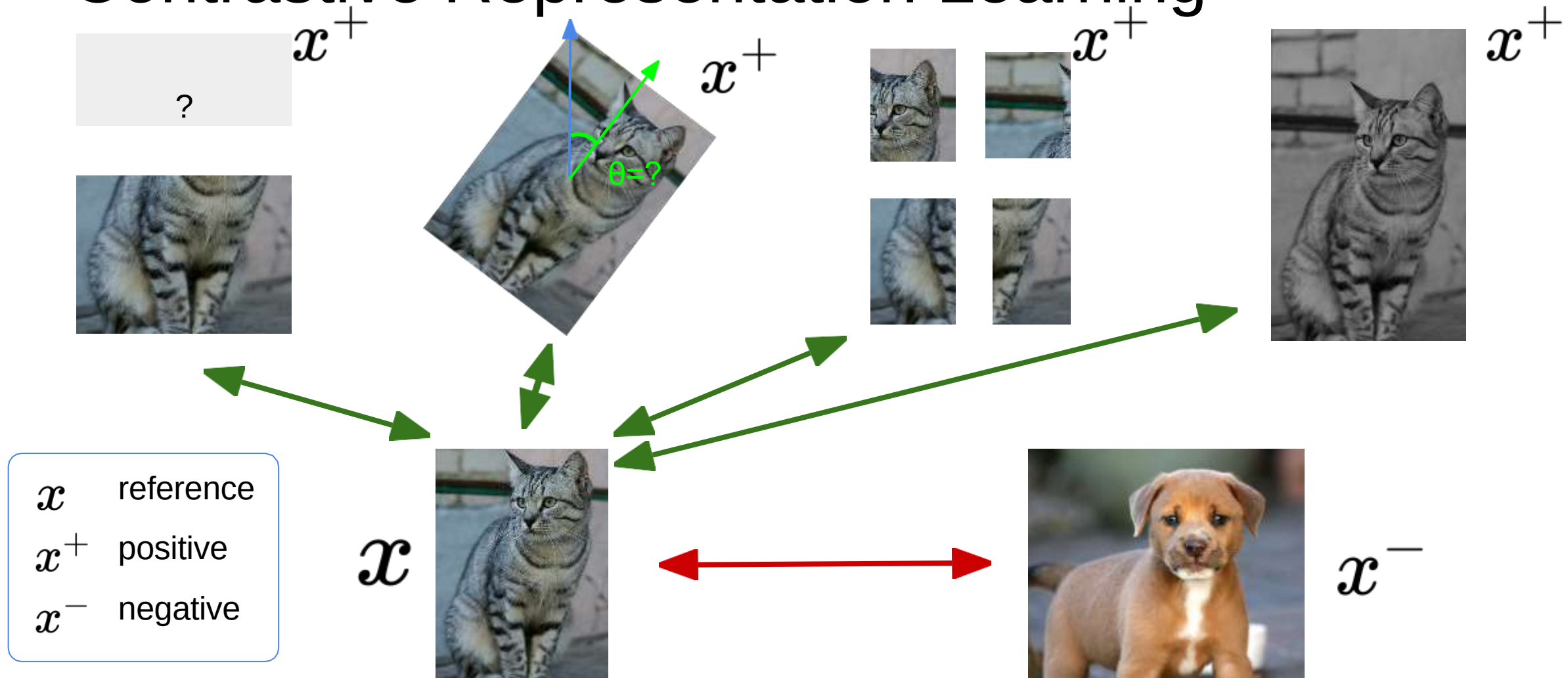
Contrastive Representation Learning



Contrastive Representation Learning



Contrastive Representation Learning



A formulation of contrastive learning

What we want:

$$\text{score}(f(x), f(x^+)) \gg \text{score}(f(x), f(x^-))$$

x : reference sample; x^+ positive sample; x^- negative sample

Given a chosen score function, we aim to learn an **encoder function** f that yields high score for positive pairs (x, x^+) and low scores for negative pairs (x, x^-) .

A formulation of contrastive learning

Loss function given 1 positive sample and $N - 1$ negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$

A formulation of contrastive learning

Loss function given 1 positive sample and $N - 1$ negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\overbrace{\exp(s(f(x), f(x^+)))}}{\underbrace{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} } \right]$$



x



x^+



x



x_1^-



x_2^-



x_3^-

...

A formulation of contrastive learning

Loss function given 1 positive sample and $N - 1$ negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\overbrace{\exp(s(f(x), f(x^+)))}^{\text{score for the positive pair}}}{\underbrace{\exp(s(f(x), f(x^+)))}_{\text{score for the positive pair}} + \underbrace{\sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))}_{\text{score for the N-1 negative pairs}}} \right]$$

This seems familiar ...

A formulation of contrastive learning

Loss function given 1 positive sample and N - 1 negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\overbrace{\exp(s(f(x), f(x^+)))}^{\text{score for the positive pair}}}{\underbrace{\exp(s(f(x), f(x^+)))}_{\text{score for the positive pair}} + \underbrace{\sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))}_{\text{score for the N-1 negative pairs}}} \right]$$

This seems familiar ...

Cross entropy loss for a N-way softmax classifier!

I.e., learn to find the positive sample from the N samples

A formulation of contrastive learning

Loss function given 1 positive sample and $N - 1$ negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$

Commonly known as the InfoNCE loss ([van den Oord et al., 2018](#))

A lower bound on the mutual information between $f(x)$ and $f(x^+)$

$$MI[f(x), f(x^+)] - \log(N) \geq -L$$

The larger the negative sample size (N), the tighter the bound

Detailed derivation: [Poole et al., 2019](#)

SimCLR: A Simple Framework for Contrastive Learning

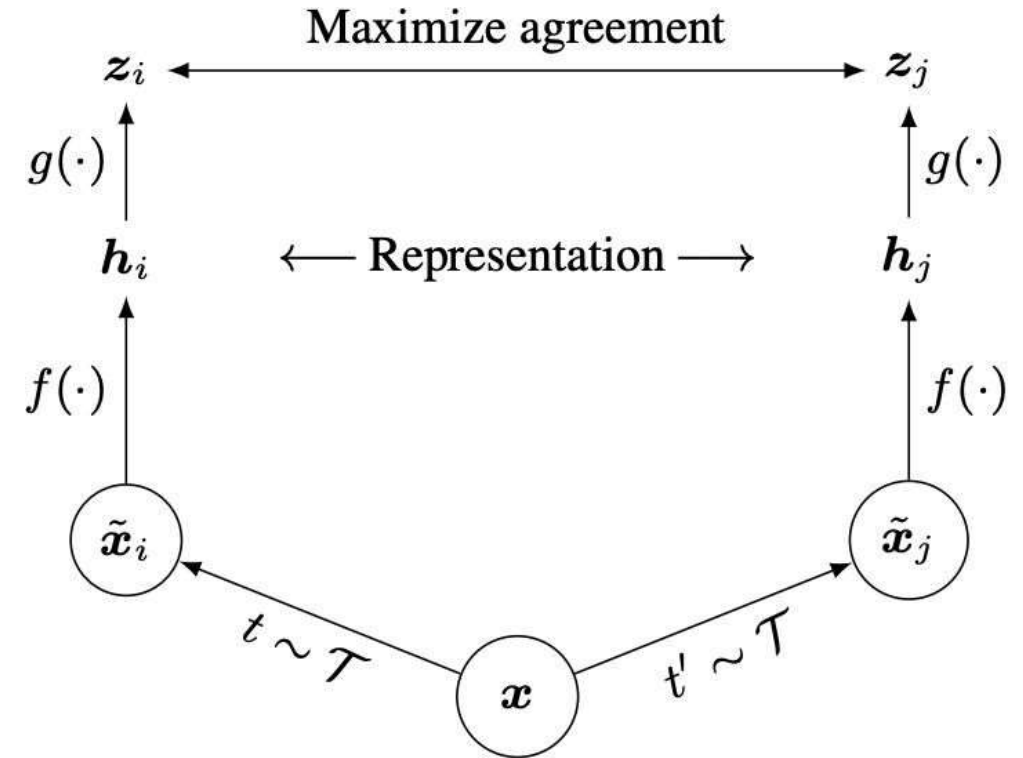
Cosine similarity as the score function:

$$s(u, v) = \frac{u^T v}{||u|| ||v||}$$

Use a projection network $g(\cdot)$ to project features to a space where contrastive learning is applied

Generate positive samples through data augmentation:

- random cropping, random color distortion, and random blur.



SimCLR: generating positive samples from data augmentation



(a) Original



(b) Crop and resize



(c) Crop, resize (and flip)



(d) Color distort. (drop)



(e) Color distort. (jitter)



(f) Rotate $\{90^\circ, 180^\circ, 270^\circ\}$



(g) Cutout



(h) Gaussian noise



(i) Gaussian blur



(j) Sobel filtering

Source: [Chen et al., 2020](#)

SimCLR

Generate a positive pair
by sampling data
augmentation functions

Algorithm 1 SimCLR's main learning algorithm.

```
input: batch size  $N$ , constant  $\tau$ , structure of  $f, g, \mathcal{T}$ .  
for sampled minibatch  $\{\mathbf{x}_k\}_{k=1}^N$  do  
  for all  $k \in \{1, \dots, N\}$  do  
    draw two augmentation functions  $t \sim \mathcal{T}, t' \sim \mathcal{T}$   
    # the first augmentation  
     $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$   
     $\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$  # representation  
     $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$  # projection  
    # the second augmentation  
     $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$   
     $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$  # representation  
     $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$  # projection  
  end for  
  for all  $i \in \{1, \dots, 2N\}$  and  $j \in \{1, \dots, 2N\}$  do  
     $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$  # pairwise similarity  
  end for  
  define  $\ell(i, j)$  as  $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$   
   $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$   
  update networks  $f$  and  $g$  to minimize  $\mathcal{L}$   
end for  
return encoder network  $f(\cdot)$ , and throw away  $g(\cdot)$ 
```

Source: [Chen et al., 2020](#)

SimCLR

Algorithm 1 SimCLR's main learning algorithm.

```
input: batch size  $N$ , constant  $\tau$ , structure of  $f, g, \mathcal{T}$ .  
for sampled minibatch  $\{\mathbf{x}_k\}_{k=1}^N$  do  
  for all  $k \in \{1, \dots, N\}$  do  
    draw two augmentation functions  $t \sim \mathcal{T}, t' \sim \mathcal{T}$   
    # the first augmentation  
     $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$   
     $\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$  # representation  
     $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$  # projection  
    # the second augmentation  
     $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$   
     $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$  # representation  
     $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$  # projection  
  end for  
  for all  $i \in \{1, \dots, 2N\}$  and  $j \in \{1, \dots, 2N\}$  do  
     $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$  # pairwise similarity  
  end for  
  define  $\ell(i, j)$  as  $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$   
   $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$   
  update networks  $f$  and  $g$  to minimize  $\mathcal{L}$   
end for  
return encoder network  $f(\cdot)$ , and throw away  $g(\cdot)$ 
```

Generate a positive pair
by sampling data
augmentation functions

InfoNCE loss:
Use all non-positive
samples in the
batch as $\mathbf{x}^-$

Source: [Chen et al., 2020](#)

SimCLR

Algorithm 1 SimCLR's main learning algorithm.

```
input: batch size  $N$ , constant  $\tau$ , structure of  $f, g, \mathcal{T}$ .  
for sampled minibatch  $\{\mathbf{x}_k\}_{k=1}^N$  do  
  for all  $k \in \{1, \dots, N\}$  do  
    draw two augmentation functions  $t \sim \mathcal{T}, t' \sim \mathcal{T}$   
    # the first augmentation  
     $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$   
     $\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$  # representation  
     $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$  # projection  
    # the second augmentation  
     $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$   
     $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$  # representation  
     $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$  # projection  
  end for  
  for all  $i \in \{1, \dots, 2N\}$  and  $j \in \{1, \dots, 2N\}$  do  
     $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$  # pairwise similarity  
  end for  
  define  $\ell(i, j)$  as  $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$   
   $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$   
  update networks  $f$  and  $g$  to minimize  $\mathcal{L}$   
end for  
return encoder network  $f(\cdot)$ , and throw away  $g(\cdot)$ 
```

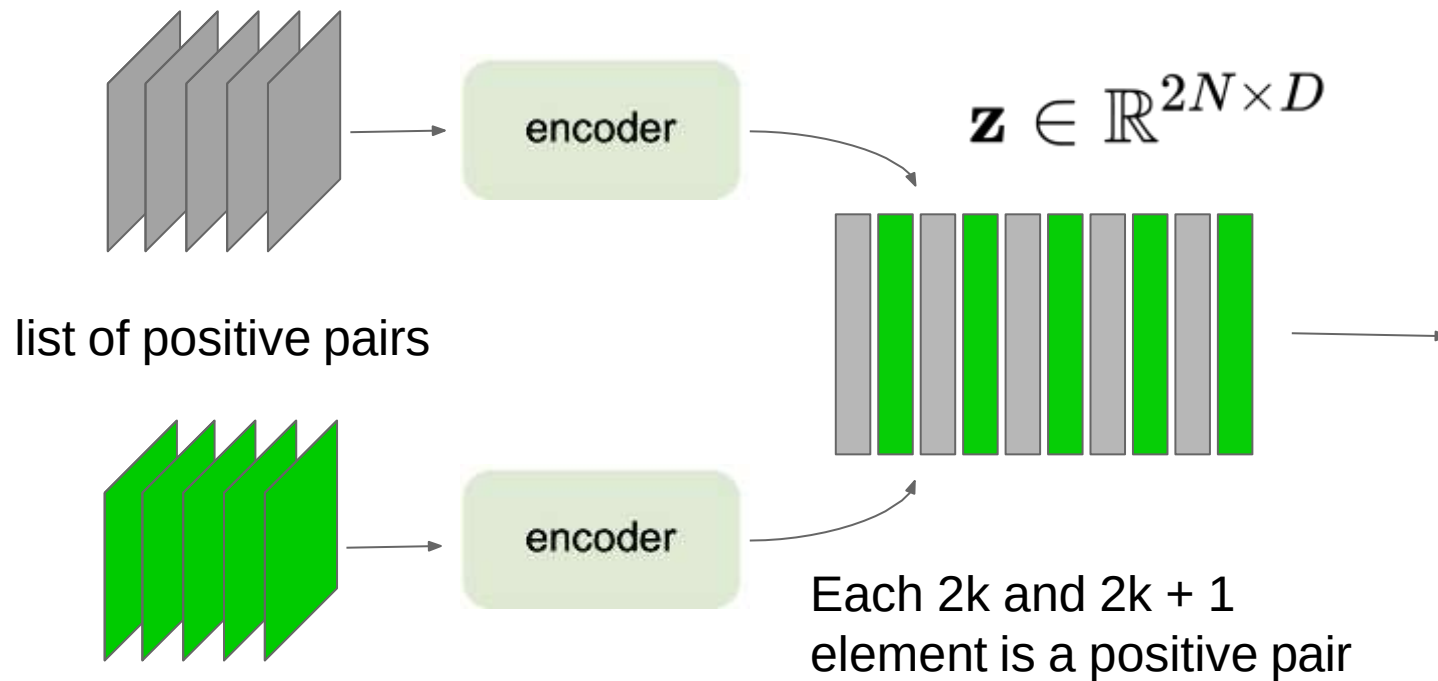
Generate a positive pair
by sampling data
augmentation functions

Iterate through and
use each of the $2N$
sample as reference,
compute average loss

InfoNCE loss:
Use all non-positive
samples in the
batch as $\mathbf{x}^-$

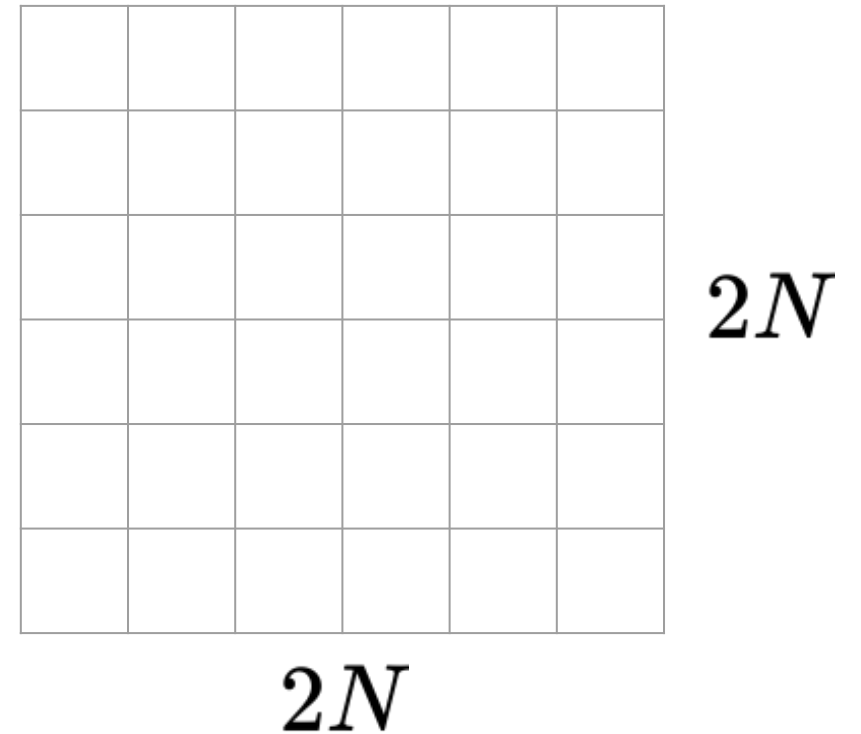
Source: [Chen et al., 2020](#)

SimCLR: mini-batch training

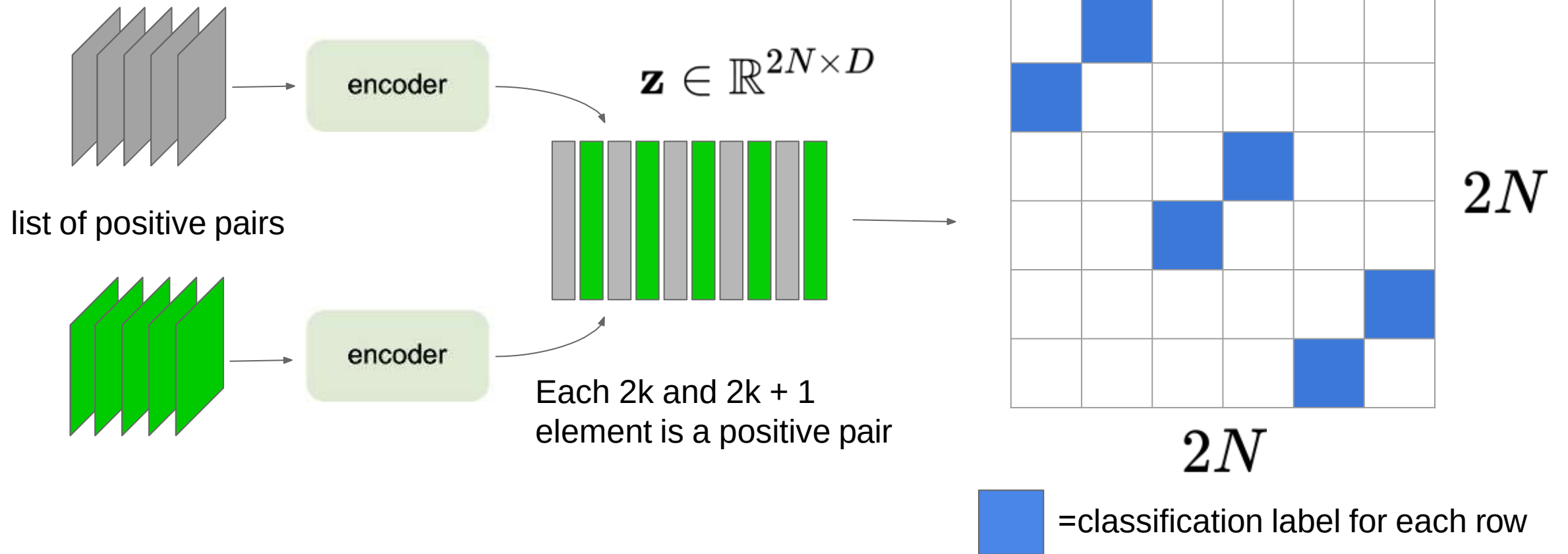


$$s_{i,j} = \frac{z_i^T z_j}{||z_i|| ||z_j||}$$

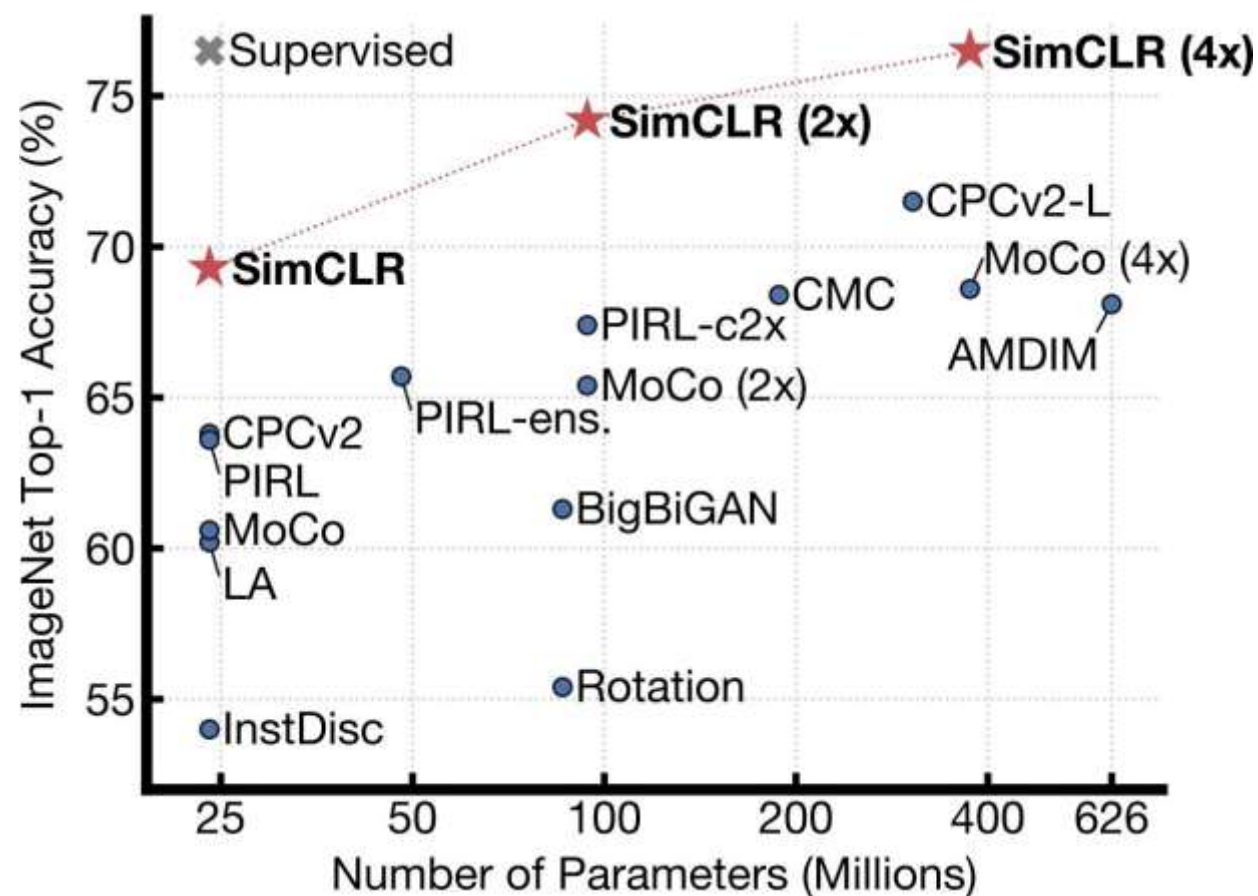
"Affinity matrix"



SimCLR: mini-batch training



Training linear classifier on SimCLR features



Train feature encoder on **ImageNet** (entire training set) using SimCLR.

Freeze feature encoder, train a linear classifier on top with labeled data.

Source: [Chen et al., 2020](#)

Semi-supervised learning on SimCLR features

Method	Architecture	Label fraction	
		1%	10%
		Top 5	
Supervised baseline	ResNet-50	48.4	80.4
<i>Methods using other label-propagation:</i>			
Pseudo-label	ResNet-50	51.6	82.4
VAT+Entropy Min.	ResNet-50	47.0	83.4
UDA (w. RandAug)	ResNet-50	-	88.5
FixMatch (w. RandAug)	ResNet-50	-	89.1
S4L (Rot+VAT+En. M.)	ResNet-50 (4×)	-	91.2
<i>Methods using representation learning only:</i>			
InstDisc	ResNet-50	39.2	77.4
BigBiGAN	RevNet-50 (4×)	55.2	78.8
PIRL	ResNet-50	57.2	83.8
CPC v2	ResNet-161(*)	77.9	91.2
SimCLR (ours)	ResNet-50	75.5	87.8
SimCLR (ours)	ResNet-50 (2×)	83.0	91.2
SimCLR (ours)	ResNet-50 (4×)	85.8	92.6

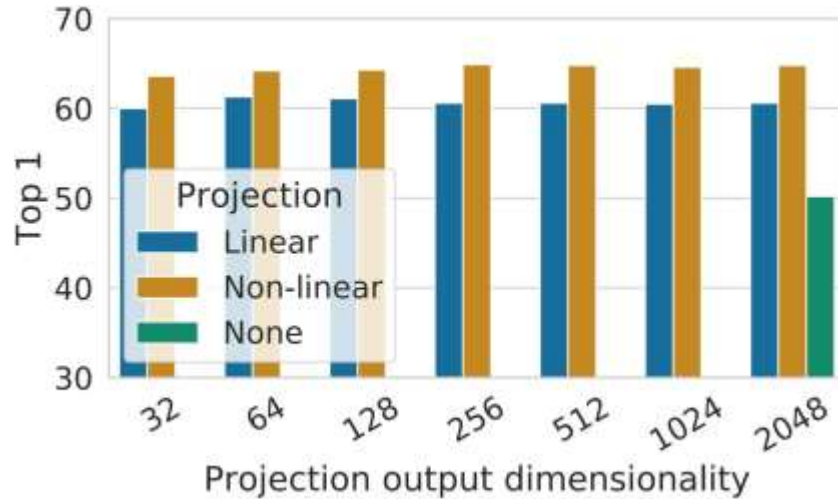
Table 7. ImageNet accuracy of models trained with few labels.

Train feature encoder on **ImageNet** (entire training set) using SimCLR.

Finetune the encoder with 1% / 10% of labeled data on ImageNet.

Source: [Chen et al., 2020](#)

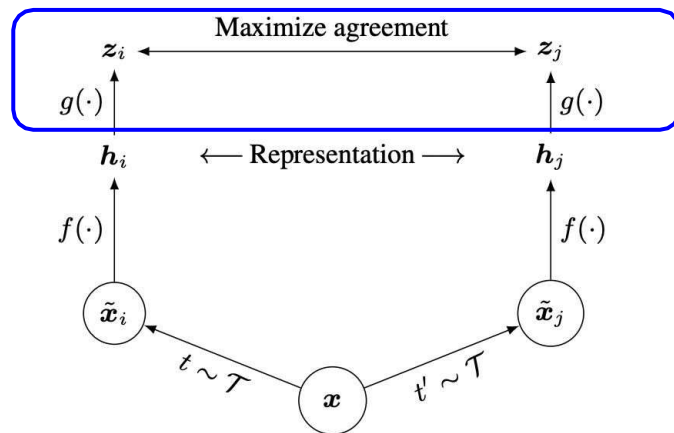
SimCLR design choices: projection head



Linear / non-linear projection heads improve representation learning.

A possible explanation:

- contrastive learning objective may discard useful information for downstream tasks
- representation space $\mathbf{z}$ is trained to be invariant to data transformation.
- by leveraging the projection head $\mathbf{g}(\cdot)$, more information can be preserved in the $\mathbf{h}$ representation space



Source: [Chen et al., 2020](#)

SimCLR design choices: large batch size

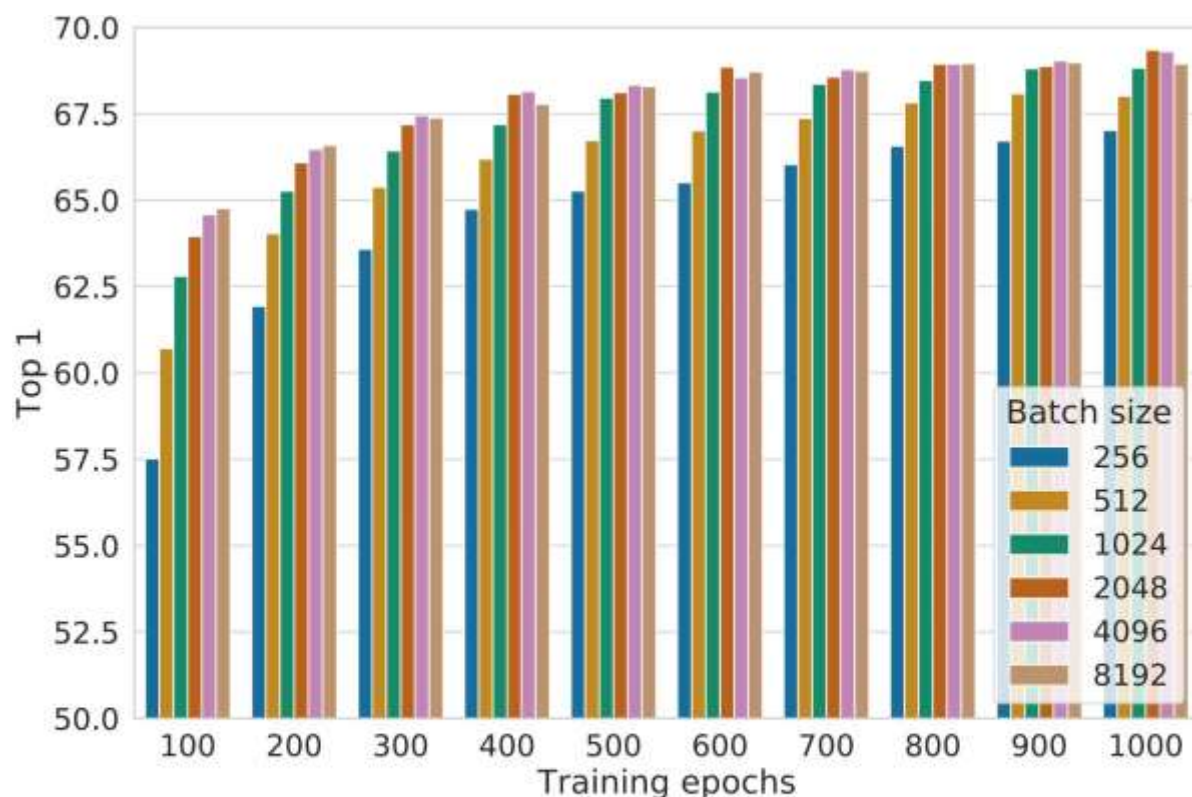


Figure 9. Linear evaluation models (ResNet-50) trained with different batch size and epochs. Each bar is a single run from scratch.¹⁰

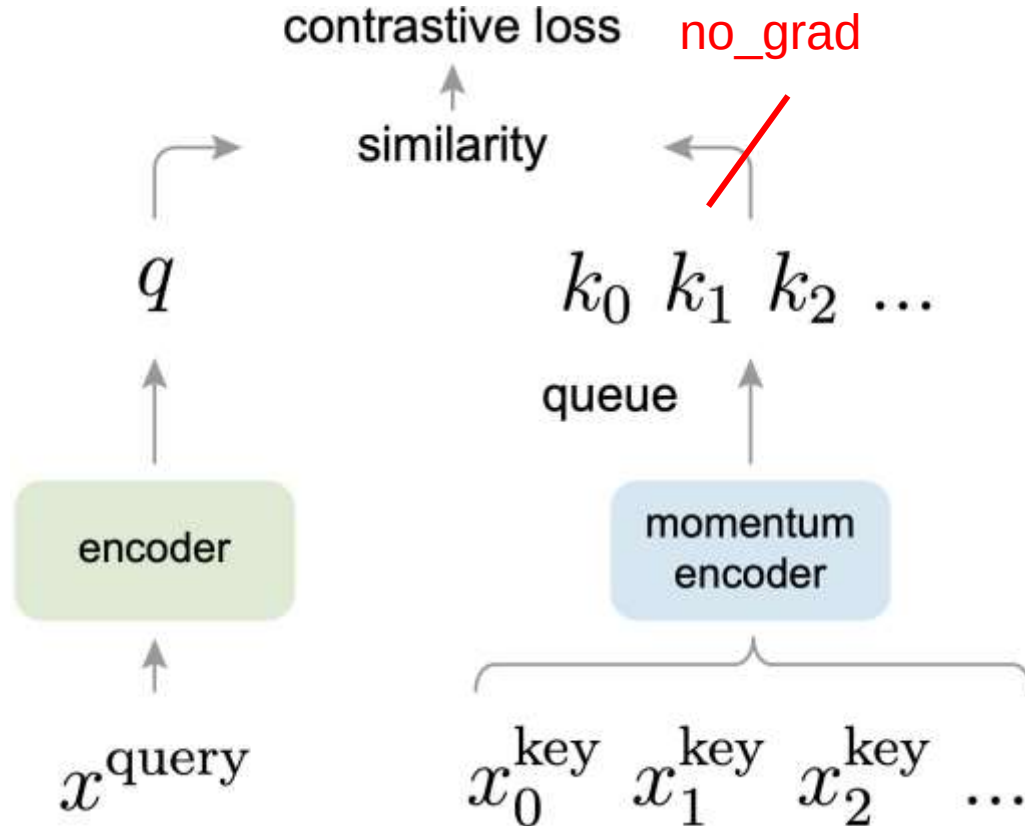
Large training batch size is crucial for SimCLR!

Large batch size causes large memory footprint during backpropagation:
requires distributed training on TPUs
(ImageNet experiments)

Source: [Chen et al., 2020](#)

Momentum Contrastive Learning (MoCo)

Key differences to SimCLR:

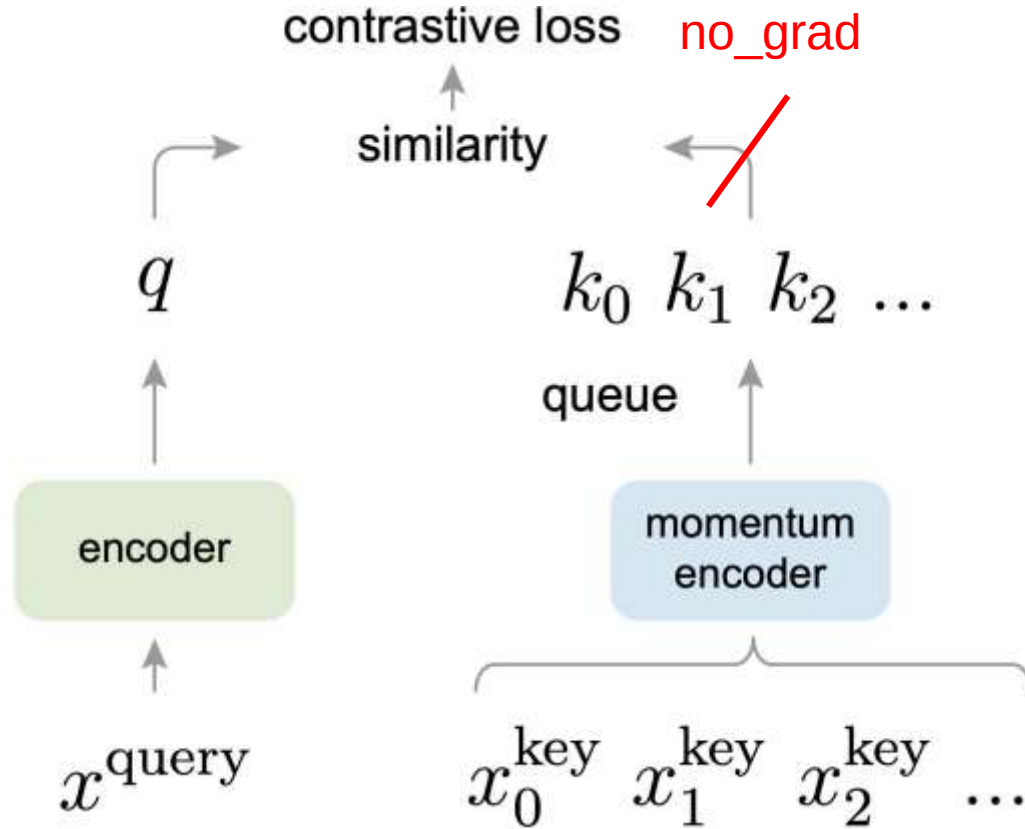


- Keep a running **queue** of keys (negative samples).
- Compute gradients and update the encoder **only through the queries**.
- Decouple min-batch size with the number of keys: can support **a large number of negative samples**.

Source: [He et al., 2020](#)

Momentum Contrastive Learning (MoCo)

Key differences to SimCLR:



- Keep a running **queue** of keys (negative samples).
- Compute gradients and update the encoder **only through the queries**.
- Decouple min-batch size with the number of keys: can support **a large number of negative samples**.
- The key encoder is **slowly progressing** through the momentum update rules:

$$\theta_k \leftarrow m\theta_k + (1 - m)\theta_q$$

Source: [He et al., 2020](#)

MoCo

Generate a positive pair
by sampling data
augmentation functions

No gradient through
the positive sample

Update the FIFO
negative sample queue

Algorithm 1 Pseudocode of MoCo in a PyTorch-like style.

```
# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature

f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
    x_q = aug(x) # a randomly augmented version
    x_k = aug(x) # another randomly augmented version

    q = f_q.forward(x_q) # queries: NxC
    k = f_k.forward(x_k) # keys: NxK
    k = k.detach() # no gradient to keys

    # positive logits: Nx1
    l_pos = bmm(q.view(N, 1, C), k.view(N, C, 1))

    # negative logits: NxK
    l_neg = mm(q.view(N, C), queue.view(C, K))

    # logits: Nx(1+K)
    logits = cat([l_pos, l_neg], dim=1)

    # contrastive loss, Eqn. (1)
    labels = zeros(N) # positives are the 0-th
    loss = CrossEntropyLoss(logits/t, labels)

    # SGD update: query network
    loss.backward()
    update(f_q.params)

    # momentum update: key network
    f_k.params = m*f_k.params+(1-m)*f_q.params

    # update dictionary
    enqueue(queue, k) # enqueue the current minibatch
    dequeue(queue) # dequeue the earliest minibatch
```

bmm: batch matrix multiplication; mm: matrix multiplication; cat: concatenation.

Use the running
queue of keys as the
negative samples

InfoNCE loss

Update f_k through
momentum

Source: [He et al., 2020](#)

“MoCo V2”

Improved Baselines with Momentum Contrastive Learning

Xinlei Chen Haoqi Fan Ross Girshick Kaiming He
Facebook AI Research (FAIR)

A hybrid of ideas from SimCLR and MoCo:

- **From SimCLR:** non-linear projection head and strong data augmentation.
- **From MoCo:** momentum-updated queues that allow training on a large number of negative samples (no TPU required!).

Source: [Chen et al., 2020](#)

MoCo vs. SimCLR vs. MoCo V2

Key takeaways:

- Non-linear projection head and strong data augmentation are crucial for contrastive learning.

case	unsup. pre-train				ImageNet acc.	VOC detection		
	MLP	aug+	cos	epochs		AP ₅₀	AP	AP ₇₅
supervised					76.5	81.3	53.5	58.8
MoCo v1				200	60.6	81.5	55.9	62.6
(a)	✓			200	66.2	82.0	56.4	62.6
(b)		✓		200	63.4	82.2	56.8	63.2
(c)	✓	✓		200	67.3	82.5	57.2	63.9
(d)	✓	✓	✓	200	67.5	82.4	57.0	63.6
(e)	✓	✓	✓	800	71.1	82.5	57.4	64.0

Table 1. **Ablation of MoCo baselines**, evaluated by ResNet-50 for (i) ImageNet linear classification, and (ii) fine-tuning VOC object detection (mean of 5 trials). “**MLP**”: with an MLP head; “**aug+**”: with extra blur augmentation; “**cos**”: cosine learning rate schedule.

MoCo vs. SimCLR vs. MoCo V2

case	unsup. pre-train					ImageNet acc.
	MLP	aug+	cos	epochs	batch	
MoCo v1 [6]				200	256	60.6
SimCLR [2]	✓	✓	✓	200	256	61.9
SimCLR [2]	✓	✓	✓	200	8192	66.6
MoCo v2	✓	✓	✓	200	256	67.5
<i>results of longer unsupervised training follow:</i>						
SimCLR [2]	✓	✓	✓	1000	4096	69.3
MoCo v2	✓	✓	✓	800	256	71.1

Table 2. **MoCo vs. SimCLR**: ImageNet linear classifier accuracy (**ResNet-50, 1-crop 224×224**), trained on features from unsupervised pre-training. “aug+” in SimCLR includes blur and stronger color distortion. SimCLR ablations are from Fig. 9 in [2] (we thank the authors for providing the numerical results).

Key takeaways:

- Non-linear projection head and strong data augmentation are crucial for contrastive learning.
- Decoupling mini-batch size with negative sample size allows MoCo-V2 to outperform SimCLR with smaller batch size (256 vs. 8192).

MoCo vs. SimCLR vs. MoCo V2

mechanism	batch	memory / GPU	time / 200-ep.
MoCo	256	5.0G	53 hrs
end-to-end	256	7.4G	65 hrs
end-to-end	4096	93.0G [†]	n/a

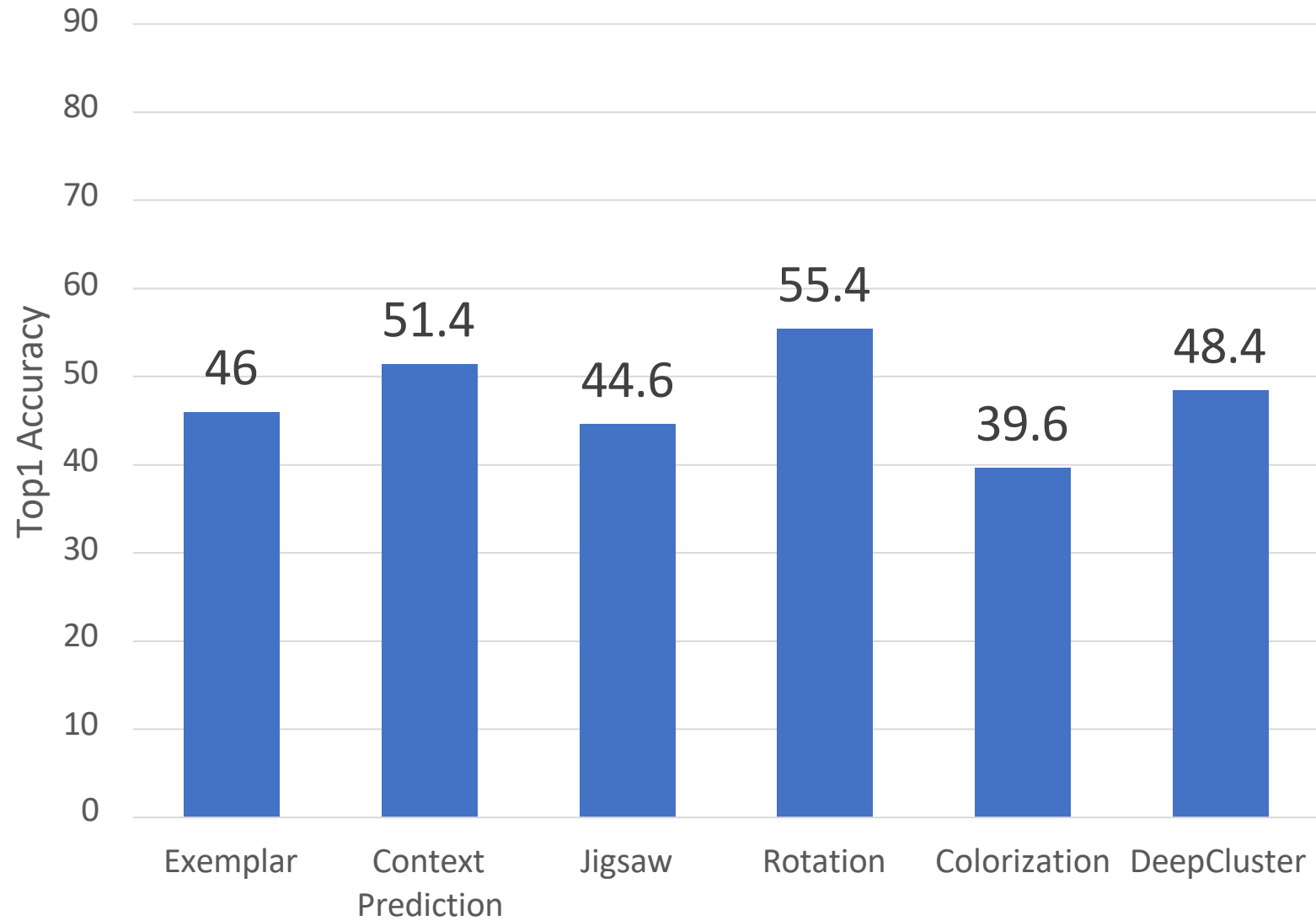
Table 3. **Memory and time cost** in 8 V100 16G GPUs, implemented in PyTorch. [†]: based on our estimation.

Key takeaways:

- Non-linear projection head and strong data augmentation are crucial for contrastive learning.
- Decoupling mini-batch size with negative sample size allows MoCo-V2 to outperform SimCLR with smaller batch size (256 vs. 8192).
- ... all with much smaller memory footprint! (“end-to-end” means SimCLR here)

Source: [Chen et al., 2020](#)

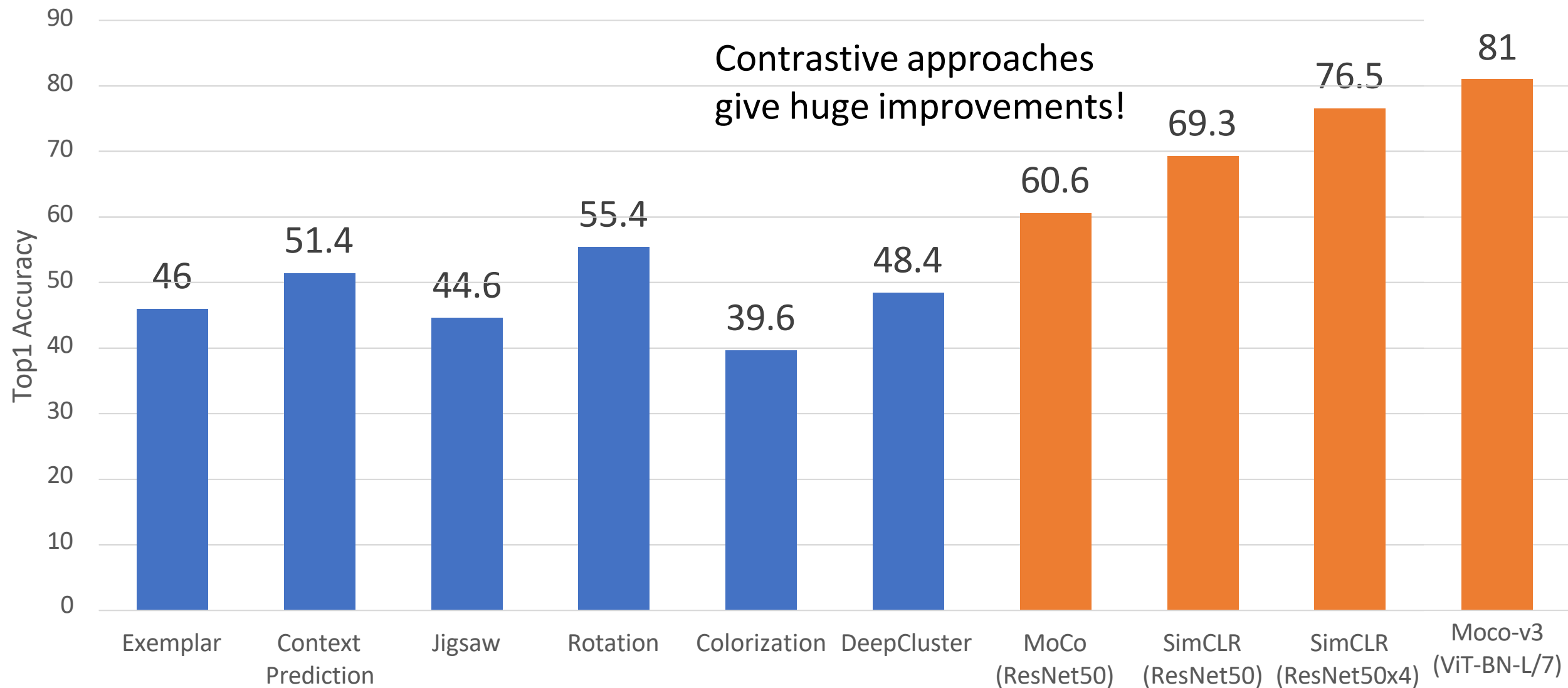
ImageNet Linear Classification from SSL Features



He et al, "Momentum Contrast for Unsupervised Visual Representation Learning", CVPR 2020
Chen et al, "A Simple Framework for Contrastive Learning of Visual Representations", ICML 2020
Chen et al, "An Empirical Study of Training Self-Supervised Vision Transformers", ICCV 2021

(Lots of caveats here ... different architectures, etc)

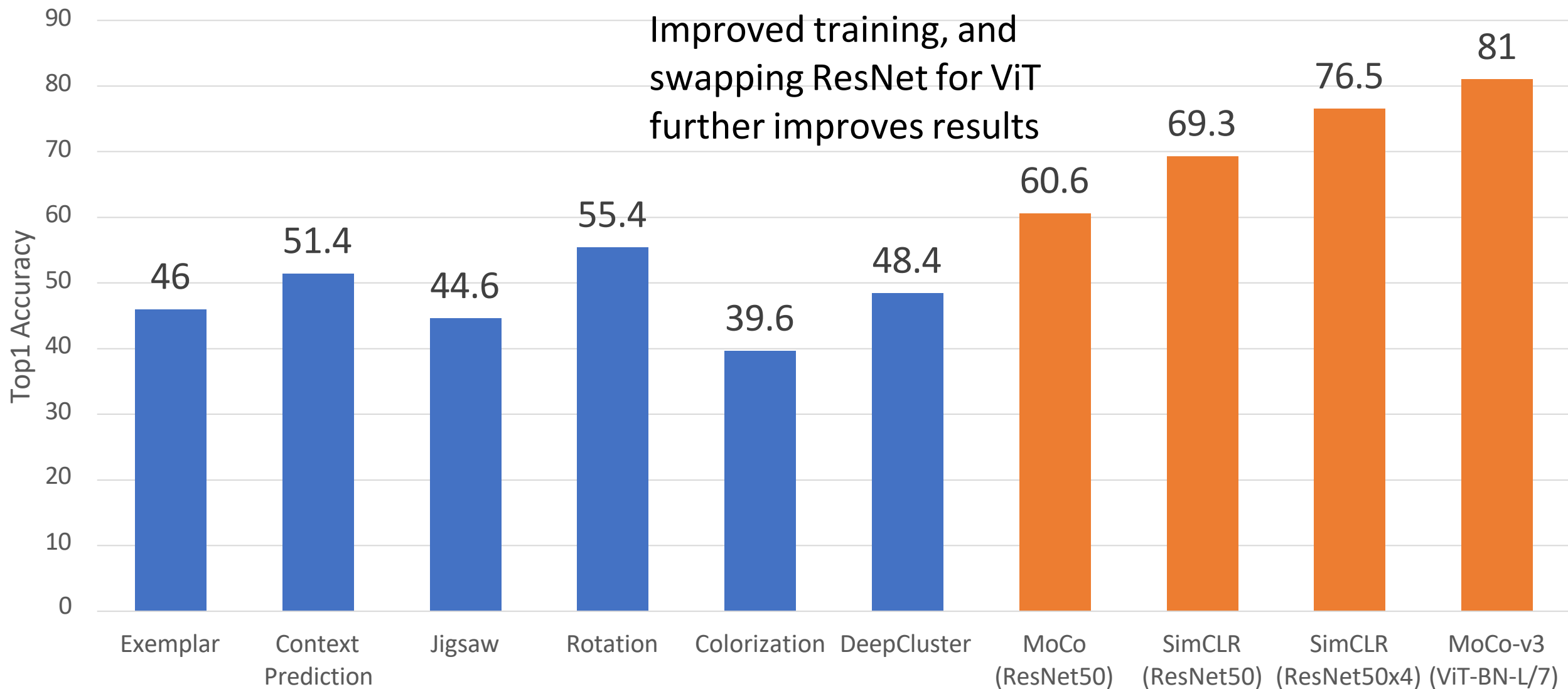
ImageNet Linear Classification from SSL Features



He et al, "Momentum Contrast for Unsupervised Visual Representation Learning", CVPR 2020
Chen et al, "A Simple Framework for Contrastive Learning of Visual Representations", ICML 2020
Chen et al, "An Empirical Study of Training Self-Supervised Vision Transformers", ICCV 2021

(Lots of caveats here ... different architectures, etc)

ImageNet Linear Classification from SSL Features

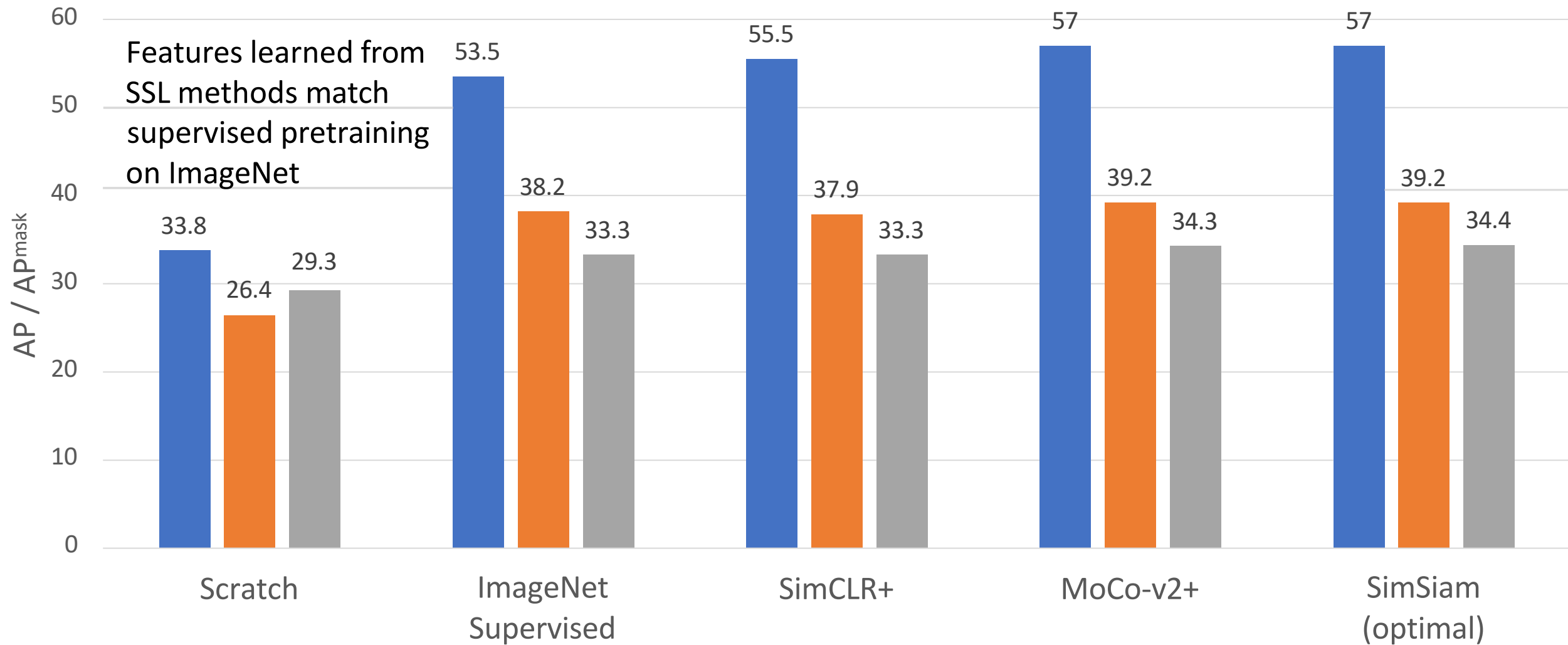


He et al, "Momentum Contrast for Unsupervised Visual Representation Learning", CVPR 2020
Chen et al, "A Simple Framework for Contrastive Learning of Visual Representations", ICML 2020
Chen et al, "An Empirical Study of Training Self-Supervised Vision Transformers", ICCV 2021

(Lots of caveats here ... different architectures, etc)

Contrastive SSL Pretraining then Finetuning on Detection

■ VOC 07+12 Detection ■ COCO Detection ■ COCO Instance Segmentation



He et al, "Momentum Contrast for Unsupervised Visual Representation Learning", CVPR 2020
Chen et al, "A Simple Framework for Contrastive Learning of Visual Representations", ICML 2020

Chen et al, "Improved Baselines with Momentum Contrastive Learning", arXiv 2020
Chen and He, "Exploring simple Siamese representation learning", CVPR 2021

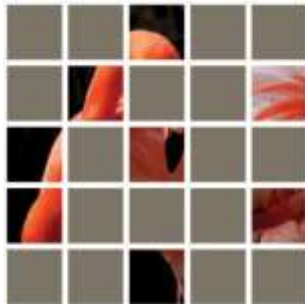
Masked Autoencoders (MAE)

A ~~new~~ old method dethrones contrastive learning? Denoising Autoencoder with Vision Transformer

Masked Autoencoders (MAE)

A ~~new~~ old method dethrones contrastive learning? Denoising Autoencoder with Vision Transformer

Divide image into
nonoverlapping patches,
discard most of them



input

Masked Autoencoders (MAE)

A ~~new~~ old method dethrones contrastive learning? Denoising Autoencoder with Vision Transformer

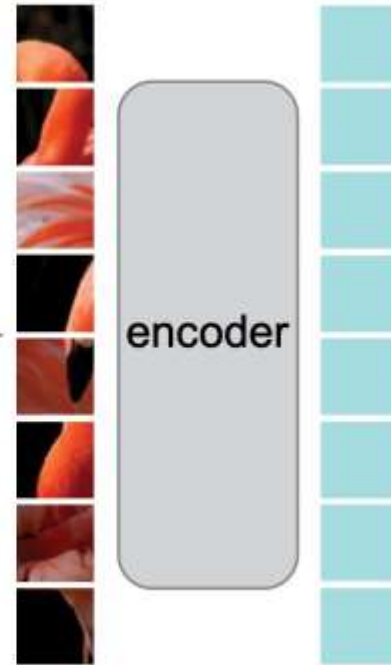
Divide image into
nonoverlapping patches,
discard most of them



input

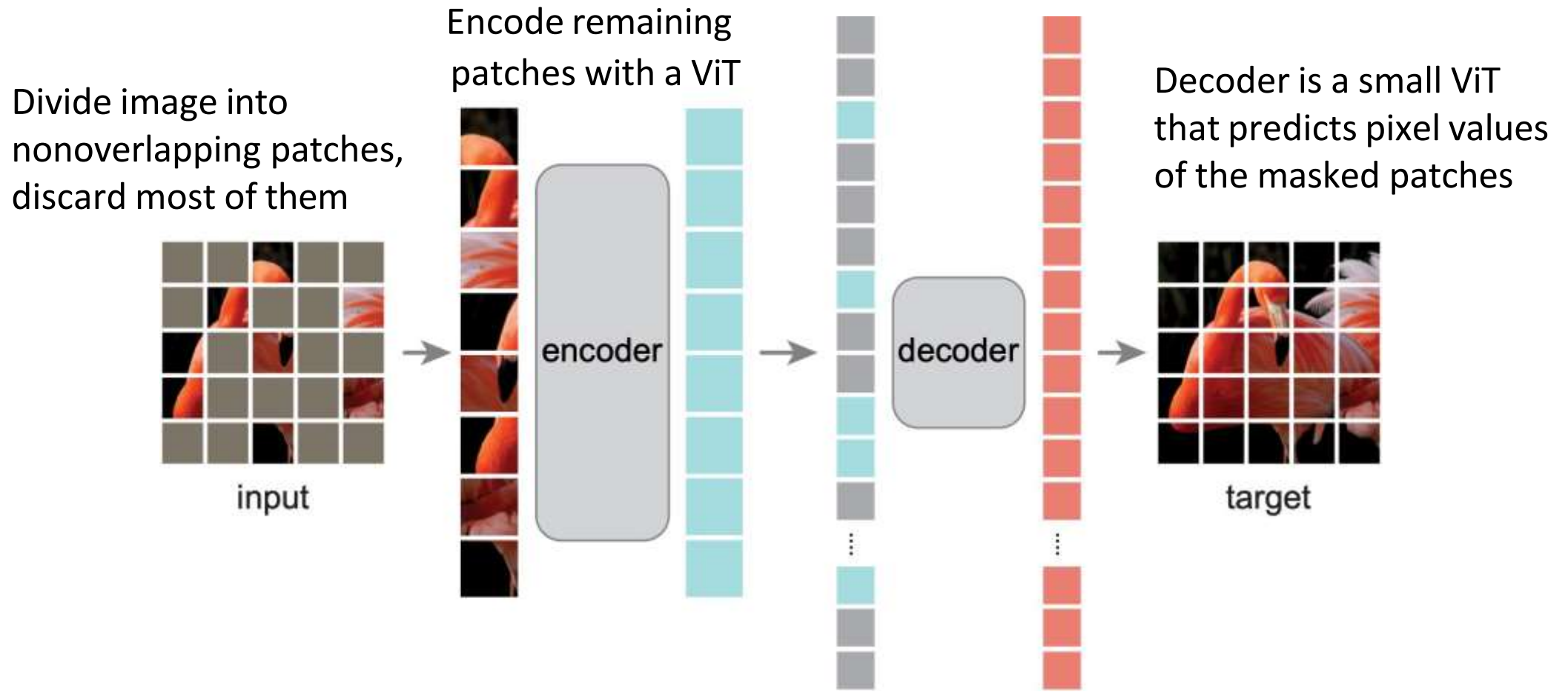


Encode remaining
patches with a ViT



Masked Autoencoders (MAE)

A new old method dethrones contrastive learning? Denoising Autoencoder with Vision Transformer



Masked Autoencoders (MAE): Reconstructions

Input Patches

Prediction

Actual Image



Masked Autoencoders (MAE): Reconstructions

Input Patches



Prediction



Actual Image



Masked Autoencoders (MAE): Reconstructions

Input Patches



Prediction

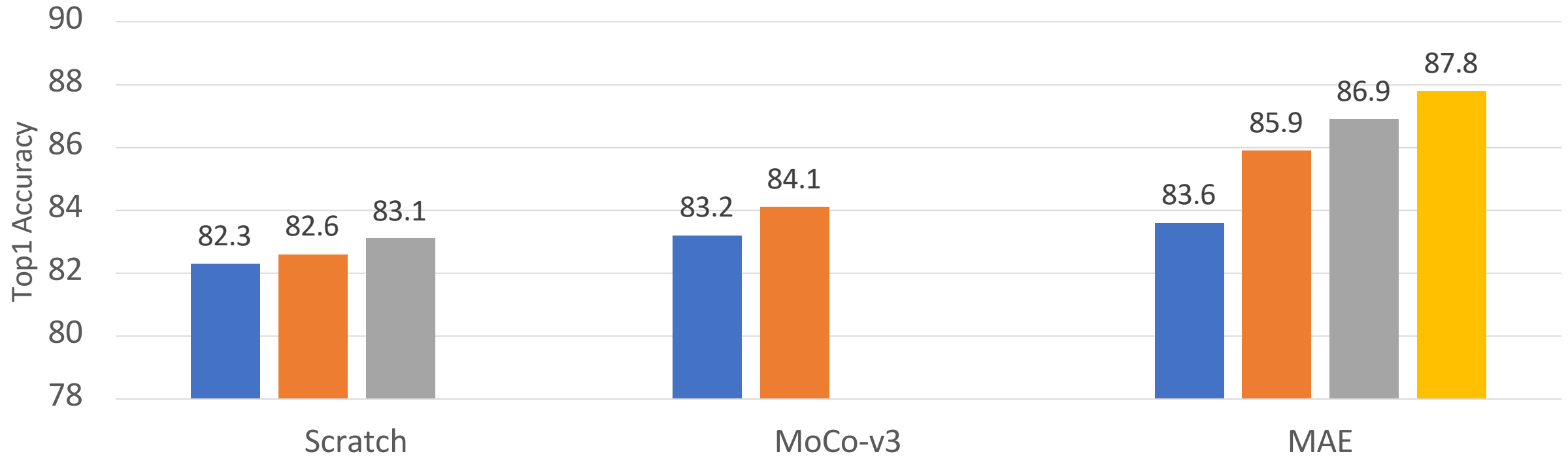


Actual Image



SSL Pretraining, then finetuning for ImageNet Classification

■ ViT-B ■ ViT-L ■ ViT-H ■ ViT-H-448



MAE Pretraining outperforms training from scratch, and allows scaling to larger ViT models

A rant about data...

The motivation of SSL is scaling to large data
that can't be labeled

A rant about data...

The motivation of SSL is scaling to large data that can't be labeled

Most papers pretrain on (unlabeled) ImageNet, then evaluate on ImageNet!

Unlabeled ImageNet is still curated: single object per image, balanced classes

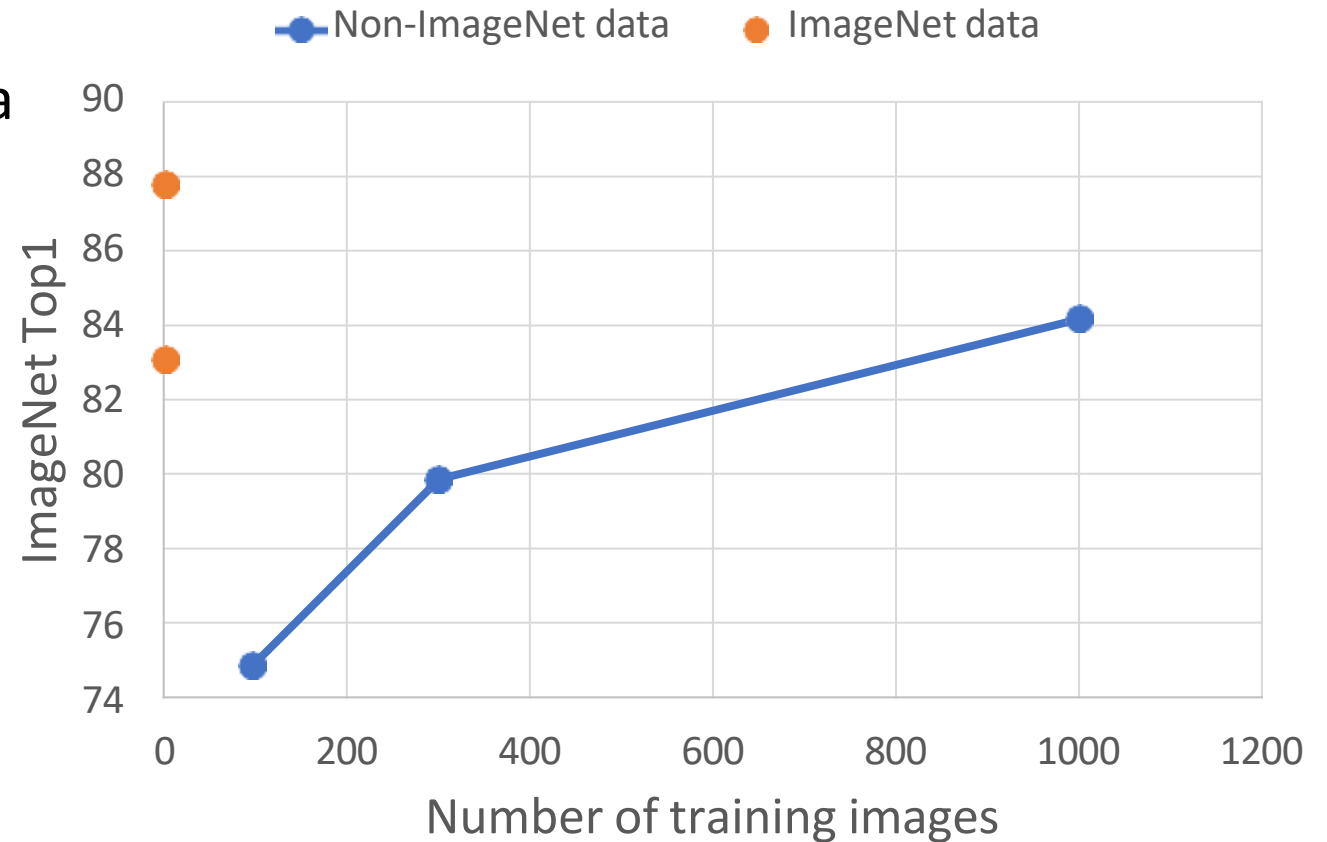
A rant about data...

The motivation of SSL is scaling to large data that can't be labeled

Most papers pretrain on (unlabeled) ImageNet, then evaluate on ImageNet!

Unlabeled ImageNet is still curated: single object per image, balanced classes

Self-Supervised Learning on larger datasets hasn't been as successful as NLP



Caron et al, "Unsupervised pre-training of images features on non-curved data", ICCV 2019

Chen et al, "Big self-supervised models are strong semi-supervised learners", NeurIPS 2020

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

Goyal et al, "Self-supervised Pretraining of Visual Features in the Wild", arXiv 2021

He et al, "Masked Autoencoders are Scalable Vision Learners", arXiv 2021

A rant about data...

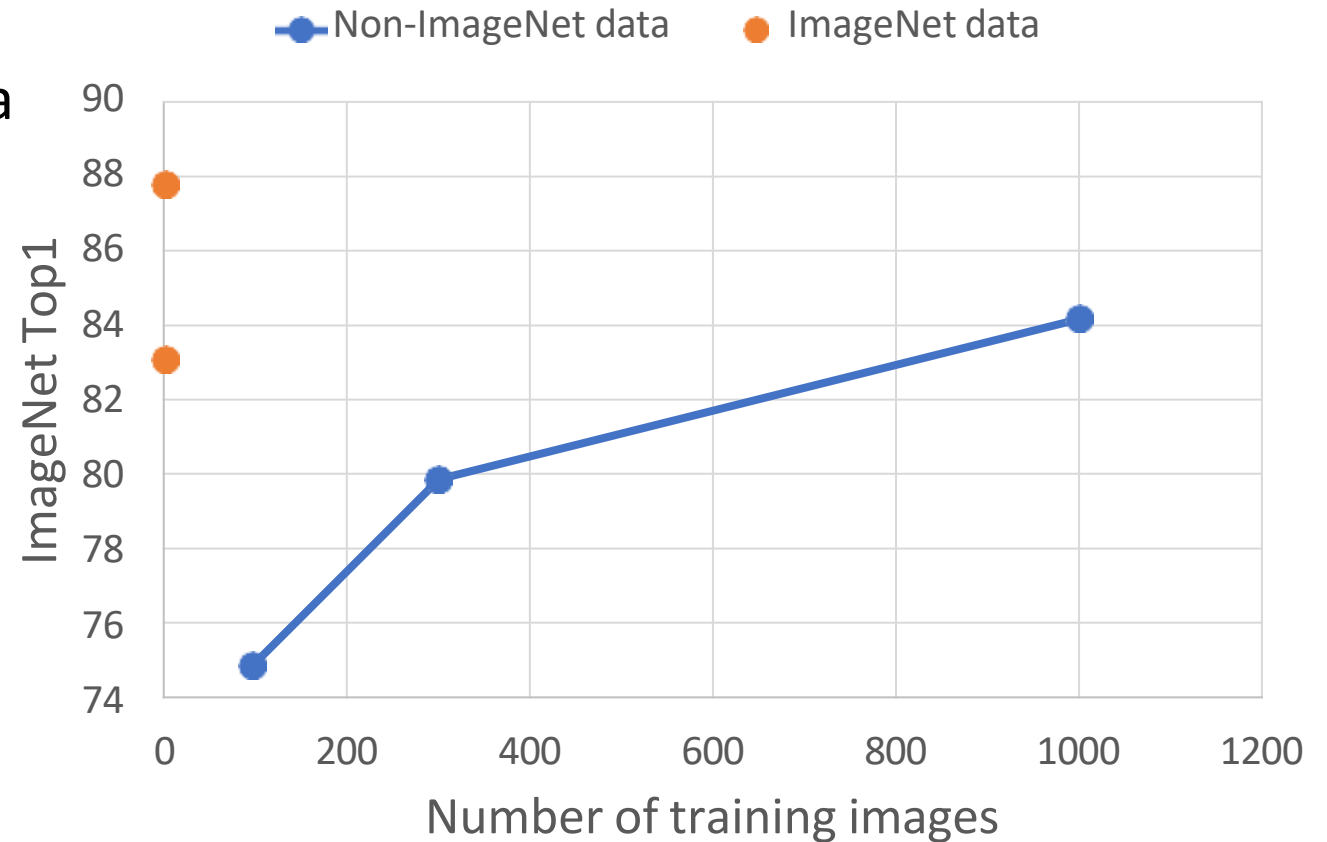
The motivation of SSL is scaling to large data that can't be labeled

Most papers pretrain on (unlabeled) ImageNet, then evaluate on ImageNet!

Unlabeled ImageNet is still curated: single object per image, balanced classes

Self-Supervised Learning on larger datasets hasn't been as successful as NLP

Idea: What if we go beyond isolated images?



Caron et al, "Unsupervised pre-training of images features on non-curved data", ICCV 2019

Chen et al, "Big self-supervised models are strong semi-supervised learners", NeurIPS 2020

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

Goyal et al, "Self-supervised Pretraining of Visual Features in the Wild", arXiv 2021

He et al, "Masked Autoencoders are Scalable Vision Learners", arXiv 2021

Multimodal Self-Supervised Learning

Don't learn from isolated images -- take images together with some **context**

Video: Image together with adjacent video frames

Agrawal et al, "Learning to See by Moving", ICCV 2015

Wang et al, "Unsupervised Learning of Visual Representations using Videos", ICCV 2015

Pathak et al, "Learning Features by Watching Objects Move", CVPR 2017

Multimodal Self-Supervised Learning

Don't learn from isolated images -- take images together with some **context**

Video: Image together with adjacent video frames

Agrawal et al, "Learning to See by Moving", ICCV 2015

Wang et al, "Unsupervised Learning of Visual Representations using Videos", ICCV 2015

Pathak et al, "Learning Features by Watching Objects Move", CVPR 2017

Sound: Image with audio track from video

Owens et al, "Ambient Sound Provides Supervision for Visual Learning", ECCV 2016

Arandjelovic and Zisserman, "Look, Listen and Learn", ICCV 2017

Multimodal Self-Supervised Learning

Don't learn from isolated images -- take images together with some **context**

Video: Image together with adjacent video frames

Agrawal et al, "Learning to See by Moving", ICCV 2015

Wang et al, "Unsupervised Learning of Visual Representations using Videos", ICCV 2015

Pathak et al, "Learning Features by Watching Objects Move", CVPR 2017

Sound: Image with audio track from video

Owens et al, "Ambient Sound Provides Supervision for Visual Learning", ECCV 2016

Arandjelovic and Zisserman, "Look, Listen and Learn", ICCV 2017

3D: Image with depth map or point cloud

Xie et al, "PointContrast: Unsupervised Pre-training for 3D Point Cloud Understanding", ECCV 2020

Zhang et al, "Self-supervised pretraining of 3D features on any point-cloud", CVPR 2021

Multimodal Self-Supervised Learning

Don't learn from isolated images -- take images together with some **context**

Video: Image together with adjacent video frames

Agrawal et al, "Learning to See by Moving", ICCV 2015

Wang et al, "Unsupervised Learning of Visual Representations using Videos", ICCV 2015

Pathak et al, "Learning Features by Watching Objects Move", CVPR 2017

Sound: Image with audio track from video

Owens et al, "Ambient Sound Provides Supervision for Visual Learning", ECCV 2016

Arandjelovic and Zisserman, "Look, Listen and Learn", ICCV 2017

3D: Image with depth map or point cloud

Xie et al, "PointContrast: Unsupervised Pre-training for 3D Point Cloud Understanding", ECCV 2020

Zhang et al, "Self-supervised pretraining of 3D features on any point-cloud", CVPR 2021

Language: Image with natural-language text

Sariyildiz et al, "Learning Visual Representations with Caption Annotations", ECCV 2020

Desai and Johnson, "VirTex: Learning Visual Representations from Textual Annotations", CVPR 2021

Radford et al, "Learning Transferable Visual Models from Natural Language Supervision", ICML 2021

Jia et al, "Scaling up Visual and Vision-Language Representation Learning with Noisy Text Supervision", ICLR 2021

Desai et al, "RedCaps: Web-curated Image-Text data created by the people, for the people", NeurIPS 2021

Why Language?

Large dataset of
(image, caption)



a dog with his
head out the
window of the car



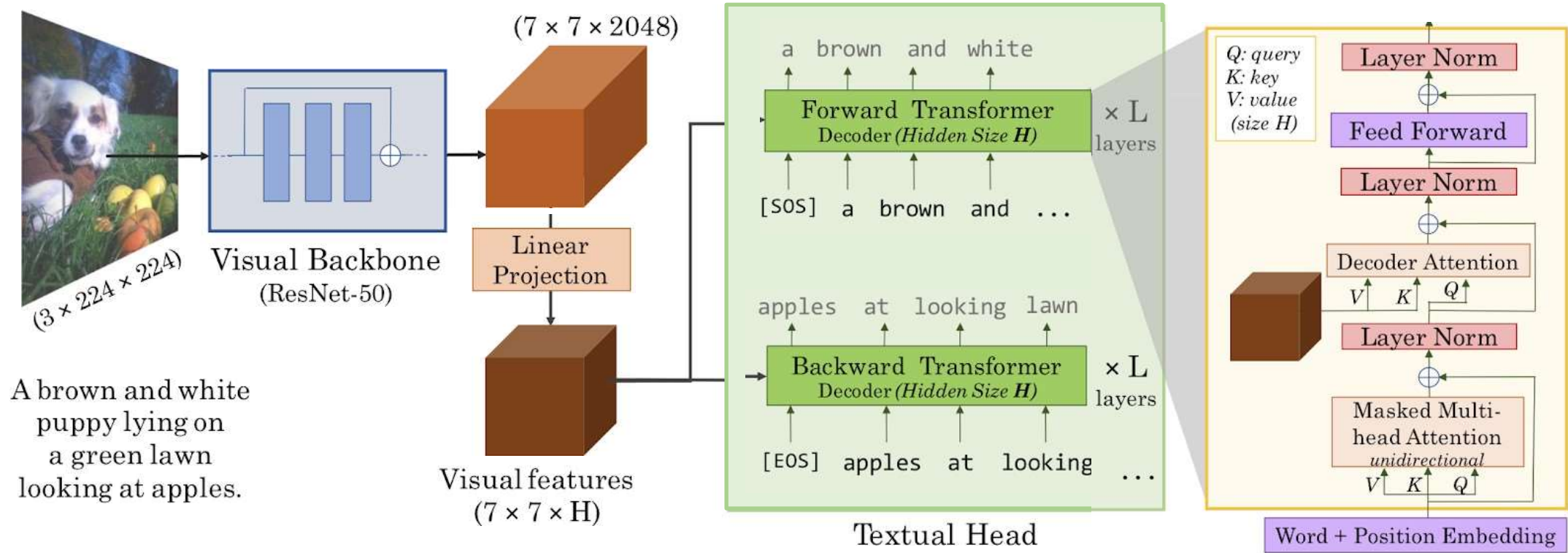
a black and orange
cat is resting on a
keyboard and yellow
back scratcher

1. **Semantic density:** Just a few words give rich information

2. **Universality:** Language can describe any concept

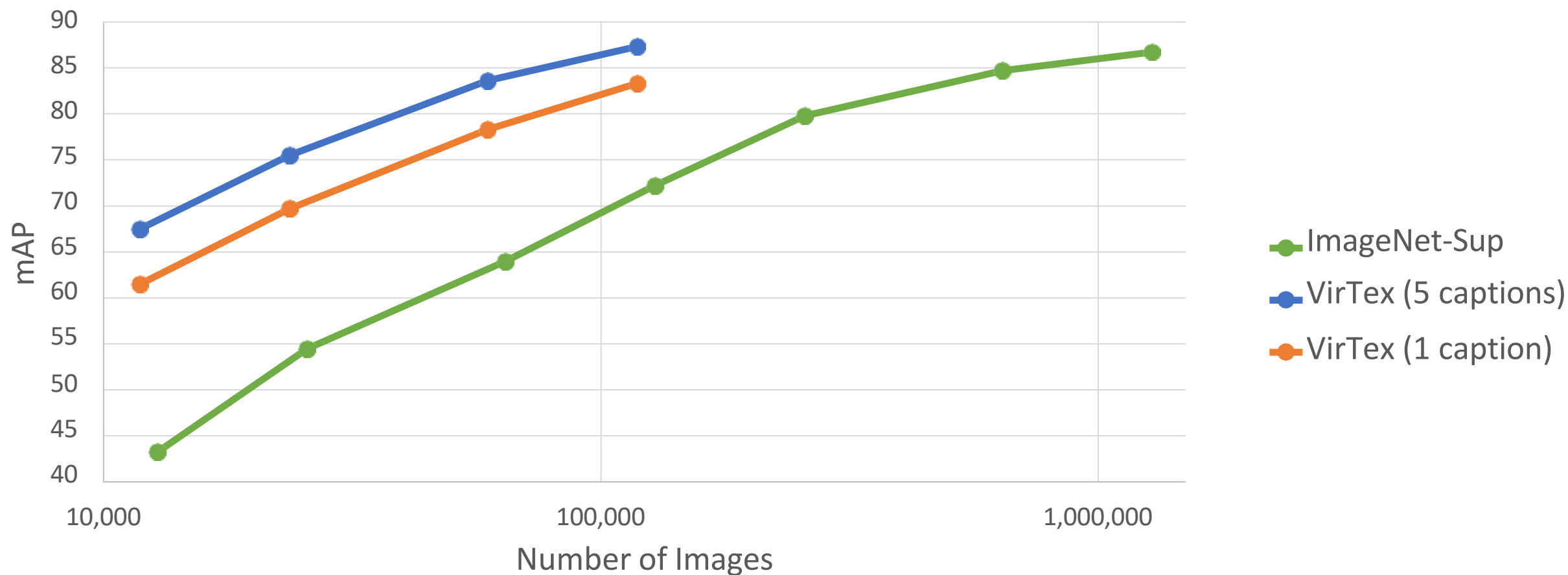
3. **Scalability:** Non-experts can easily caption images; data can also be collected from the web at scale

Generating Captions

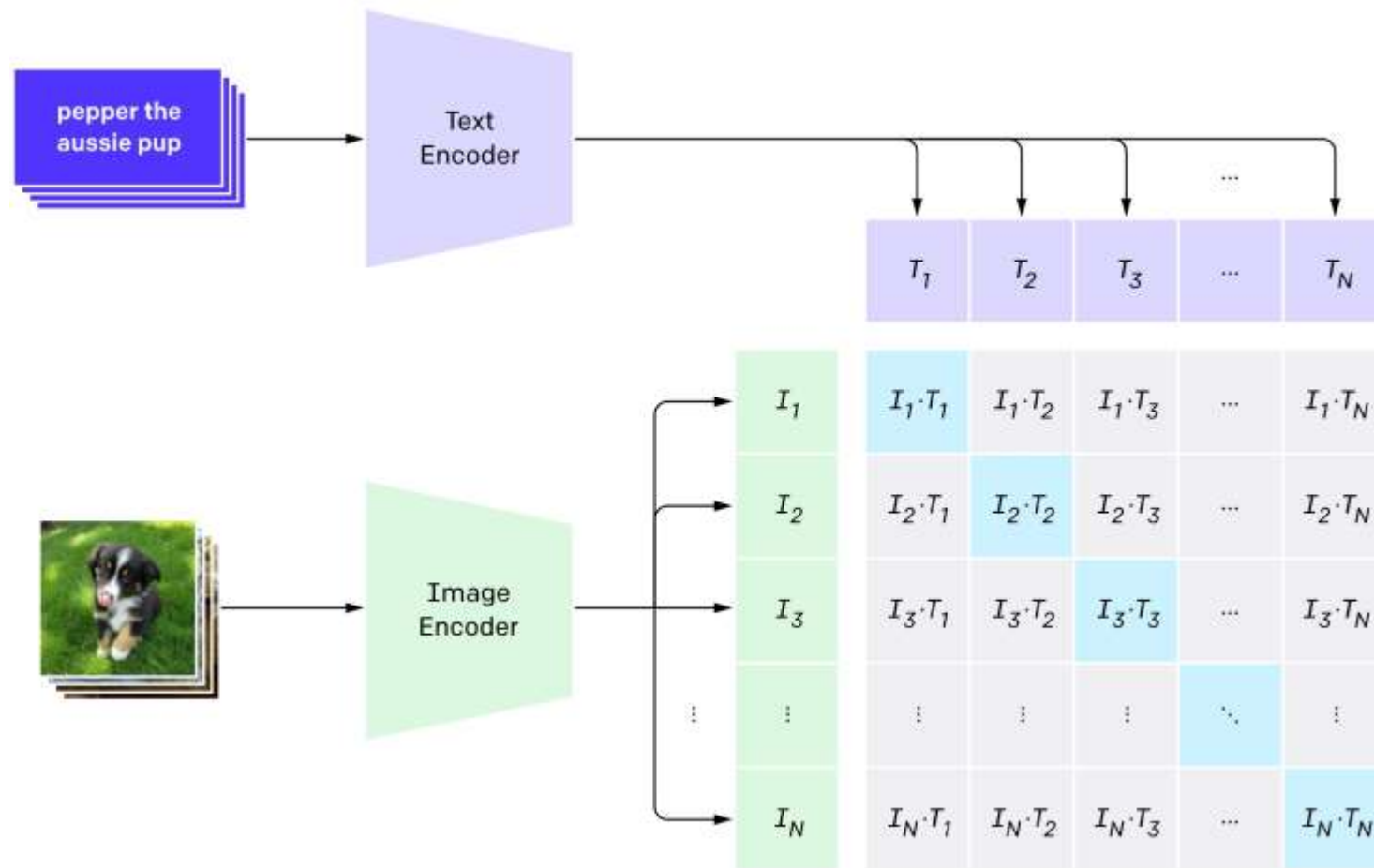


Generating Captions

PASCAL VOC Linear Classification

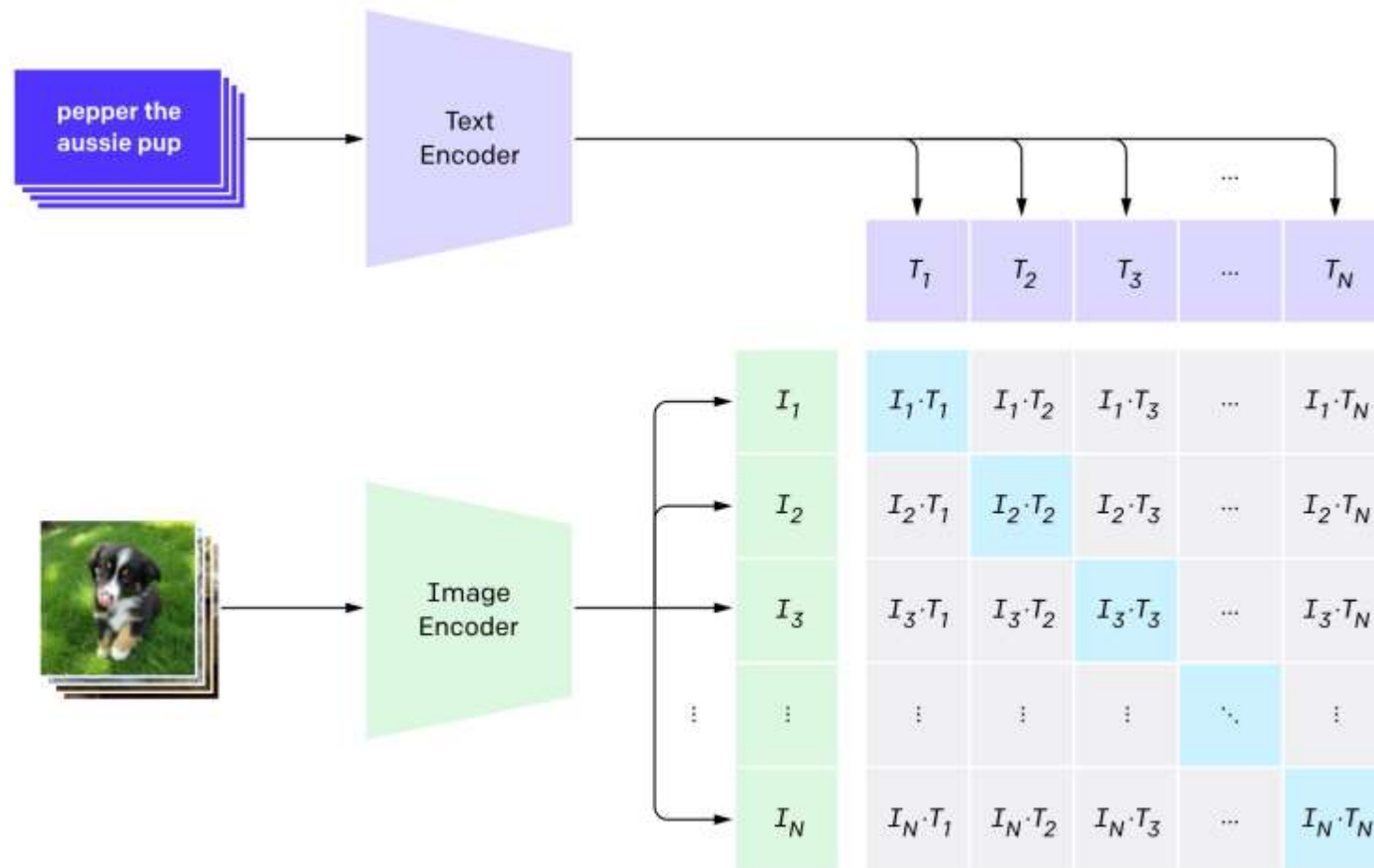


Matching Images and Text



Contrastive loss: Each image predicts which caption matches

Matching Images and Text: CLIP



Contrastive loss: Each image predicts which caption matches

Large-scale training on 400M (image, text) pairs from the internet

Matching Images and Text: CLIP (Pseudocode)

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t            - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

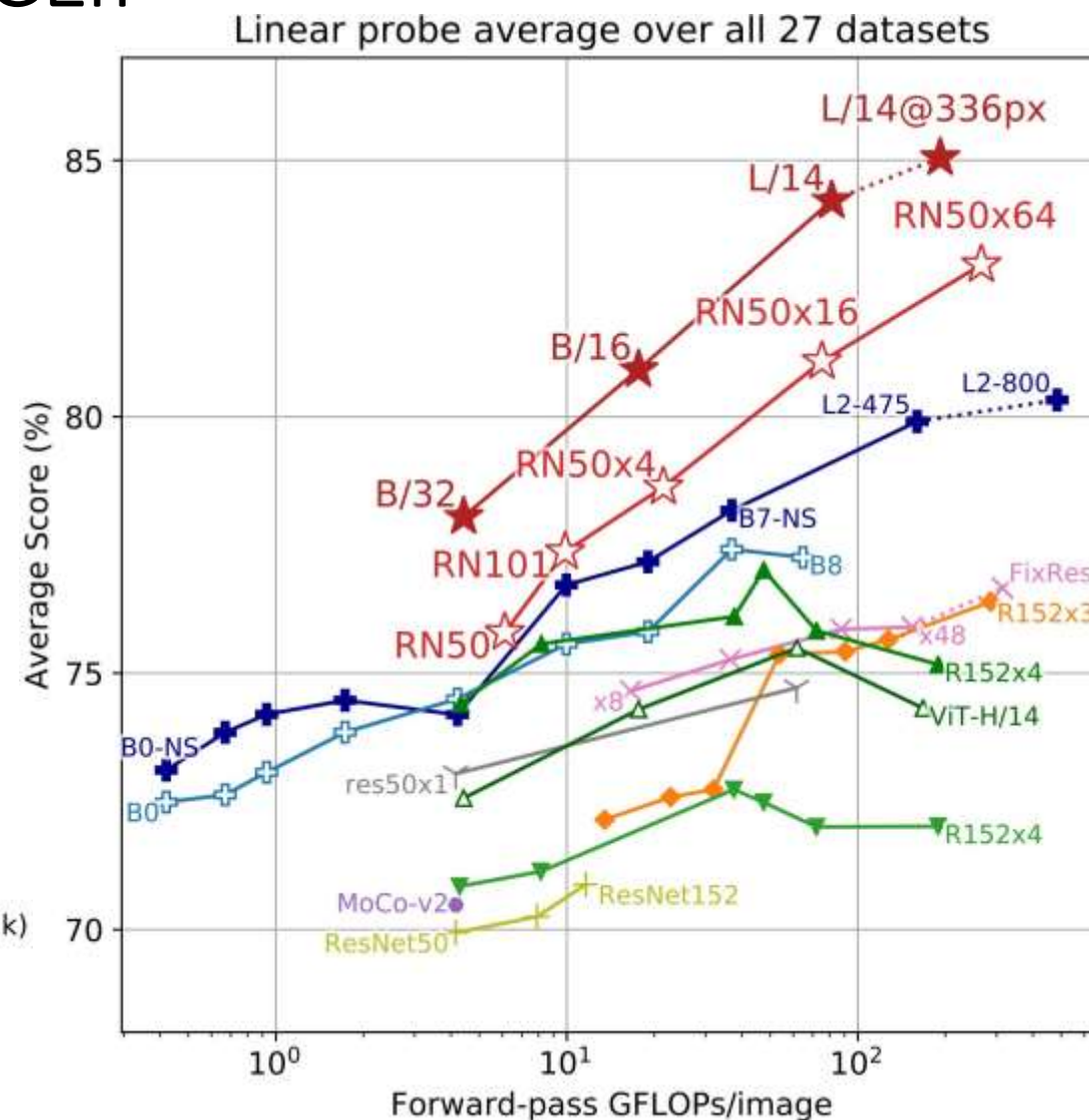
# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

Figure 3. Numpy-like pseudocode for the core of an implementation of CLIP.

Matching Images and Text: CLIP

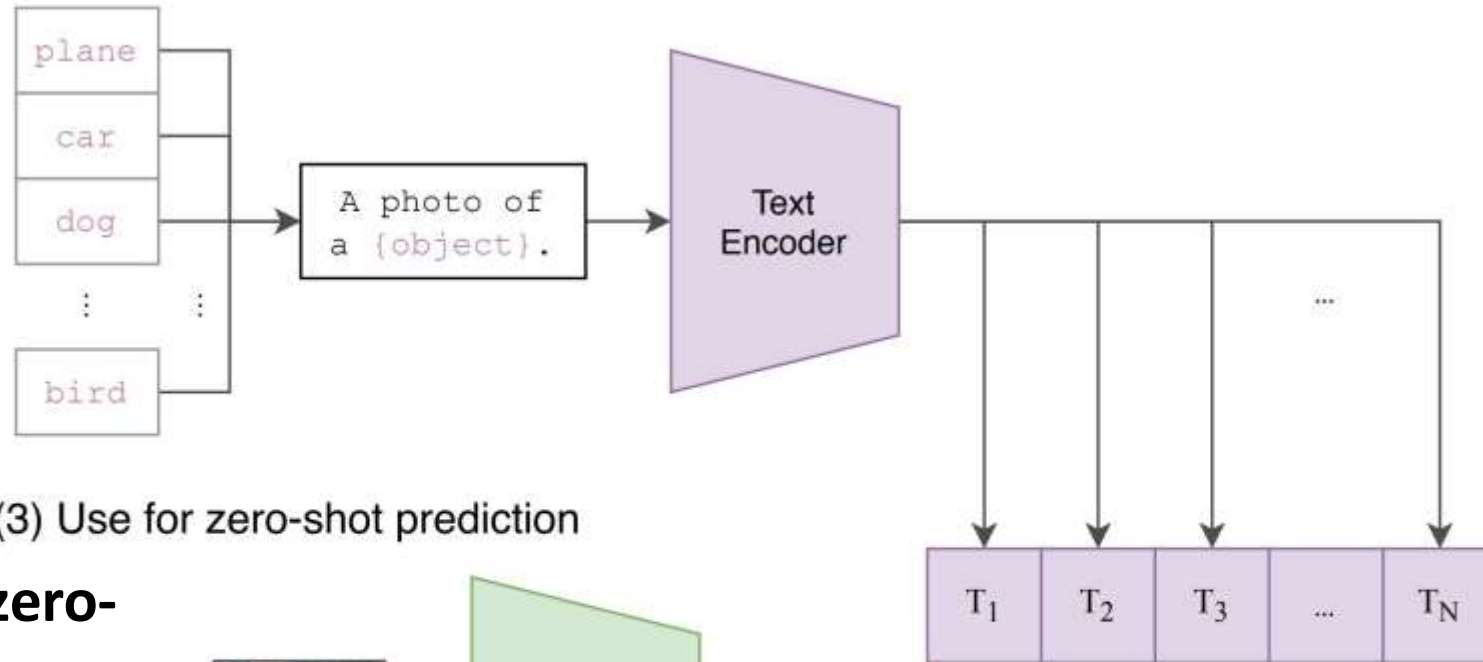
Very strong performance on many downstream vision problems!

Performance continues to improve with larger models

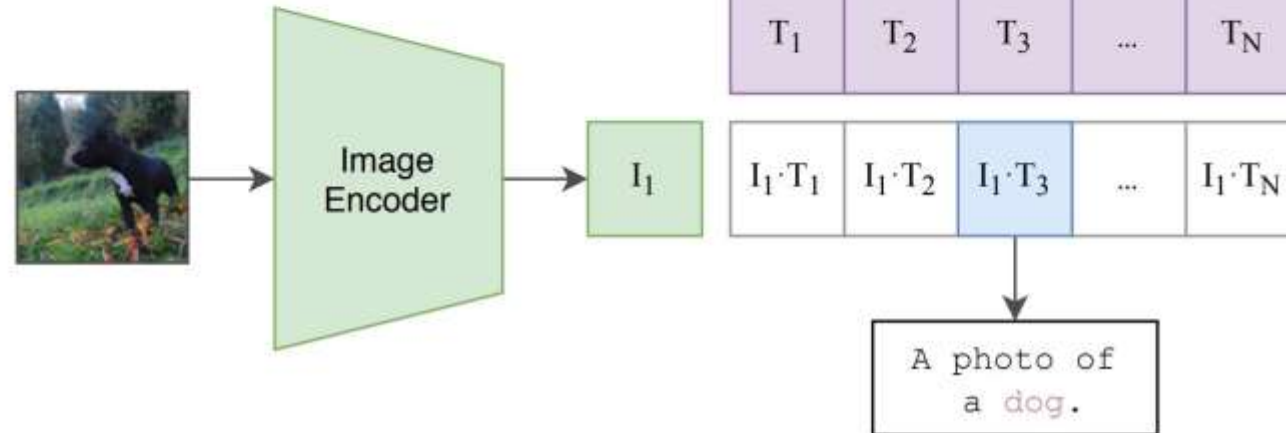


CLIP: Zero-Shot Classification

(2) Create dataset classifier from label text



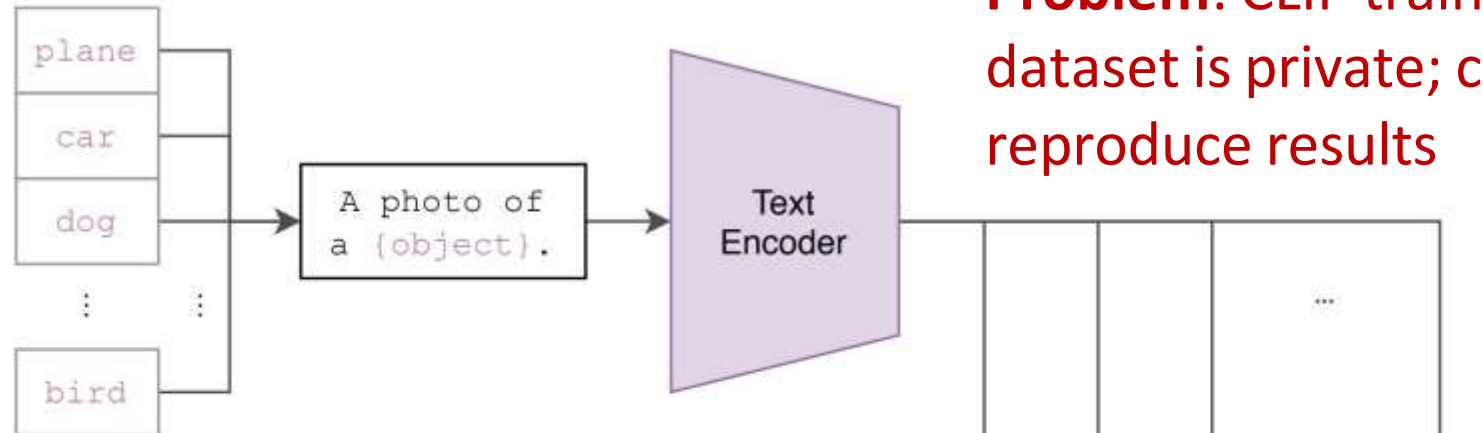
(3) Use for zero-shot prediction



Language enables **zero-shot classification**:
Classify images into categories without any additional training data!

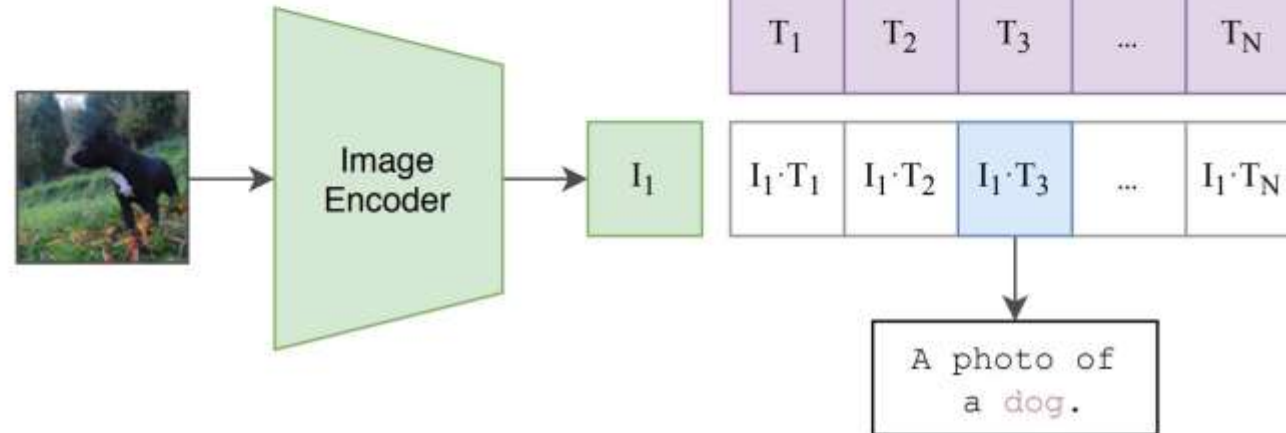
CLIP: Zero-Shot Classification

(2) Create dataset classifier from label text



Problem: CLIP training dataset is private; can't reproduce results

(3) Use for zero-shot prediction



Language enables **zero-shot classification**:
Classify images into categories without any additional training data!

RedCaps: Images and Captions from Reddit



r/birdpics: male
northern cardinal



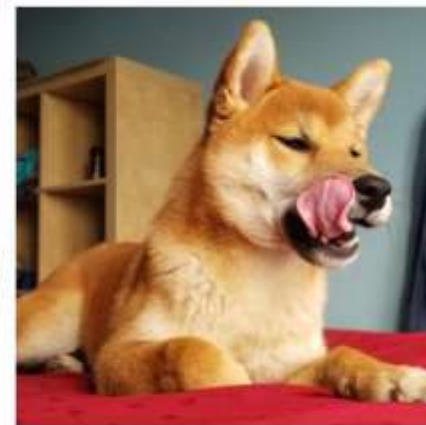
r/crafts: my mom
tied this mouse



r/itookapicture:
itap of the taj mahal



r/perfectfit: this
lemon in my drink



r/shiba: mlem!

Data from 350 manually-chosen subreddits
12M high-quality (image, caption) pairs

Summary

Self-Supervised Learning (SSL) aims to scale up to larger datasets without human annotation

First train for a **pretext** task, then **transfer** to **downstream** tasks

Many pretext tasks: context prediction, jigsaw, colorization, clustering, rotation

SSL has been wildly successful for language

Intense research on SSL in vision; current best are contrastive, masked autoencoding

Multimodal SSL uses images together with additional context

Multimodal SSL with vision + language has been very successful; seems very promising!